

# Package ‘gap’

June 24, 2026

**Type** Package

**Title** Genetic Analysis Package

**Version** 1.15.3

**Date** 2026-06-24

## Description

As first reported [Zhao, J. H. 2007. ``gap: Genetic Analysis Package". J Stat Soft 23(8):1-18. <[doi:10.18637/jss.v023.i08](https://doi.org/10.18637/jss.v023.i08)>], it is designed as an integrated package for genetic data analysis of both population and family data. Currently, it contains functions for sample size calculations of both population-based and family-based designs, probability of familial disease aggregation, kinship calculation, statistics in linkage analysis, and association analysis involving genetic markers including haplotype analysis with or without environmental covariates. Over years, the package has been developed in-between many projects hence also in line with the name (gap).

**License** GPL (>= 2)

**URL** <https://github.com/jinghuazhao/R>

**BugReports** <https://github.com/jinghuazhao/R/issues>

**Depends** R (>= 2.10), gap.datasets (>= 0.0.6)

**Imports** dplyr, ggplot2, plotly, Rdpack

**Suggests** BradleyTerry2, DiagrammeR, MASS, Matrix, MCMCglmm, R2jags, bdsmatrix, bookdown, calibrate, circlize, coda, cowplot, coxme, foreign, genetics, grDevices, haplo.stats, htmltools, htmlwidgets, jsonlite, kinship2, knitr, lattice, magic, matrixStats, meta, metafor, nlme, pedigree, pedigreemm, plotrix, readr, remotes, reshape, rmarkdown, rms, scales, survival, valr

**Enhances** shiny

**LazyData** Yes

**LazyLoad** Yes

**NeedsCompilation** yes

**Encoding** UTF-8

**VignetteBuilder** knitr

**Roxygen** list(markdown=TRUE)

**RdMacros** Rdpack

**Config/roxygen2/version** 8.0.0

**RoxygenNote 7.3.3****Author** Jing Hua Zhao [aut, cre] (ORCID:<<https://orcid.org/0000-0002-1463-5870>>, 0000-0003-4930-3582),

Kurt Hornik [ctb],

Brian Ripley [ctb],

Uwe Ligges [ctb],

Achim Zeileis [ctb]

**Maintainer** Jing Hua Zhao <jinghuazhao@hotmail.com>**Contents**

|                                       |    |
|---------------------------------------|----|
| a2g . . . . .                         | 4  |
| ab . . . . .                          | 5  |
| ACDE . . . . .                        | 7  |
| ACE_CI . . . . .                      | 10 |
| AE3 . . . . .                         | 11 |
| allele.recode . . . . .               | 12 |
| asplot . . . . .                      | 13 |
| b2r . . . . .                         | 14 |
| BFDP . . . . .                        | 15 |
| bt . . . . .                          | 17 |
| ccsize . . . . .                      | 19 |
| chow.test . . . . .                   | 22 |
| chr_pos_a1_a2 . . . . .               | 23 |
| ci2ms . . . . .                       | 24 |
| circos.cis.vs.trans.plot . . . . .    | 26 |
| circos.cnvplot . . . . .              | 27 |
| circos.mhtplot . . . . .              | 27 |
| circos.mhtplot2 . . . . .             | 28 |
| cis.vs.trans.classification . . . . . | 29 |
| cnvplot . . . . .                     | 30 |
| comp.score . . . . .                  | 31 |
| cs . . . . .                          | 33 |
| ESplot . . . . .                      | 34 |
| fbsize . . . . .                      | 35 |
| FPRP . . . . .                        | 37 |
| g2a . . . . .                         | 39 |
| gap . . . . .                         | 40 |
| gc.em . . . . .                       | 43 |
| gc.lambda . . . . .                   | 45 |
| gcontrol . . . . .                    | 45 |
| gcontrol2 . . . . .                   | 47 |
| gcp . . . . .                         | 48 |
| genecounting . . . . .                | 50 |
| geno.recode . . . . .                 | 52 |
| get_b_se . . . . .                    | 52 |
| get_pve_se . . . . .                  | 53 |
| get_sdy . . . . .                     | 54 |
| gif . . . . .                         | 55 |
| grid2d . . . . .                      | 56 |
| h2.jags . . . . .                     | 57 |

|                   |     |
|-------------------|-----|
| h2G               | 59  |
| h2GE              | 59  |
| h2l               | 60  |
| h2_mzdz           | 60  |
| hap               | 62  |
| hap.control       | 64  |
| hap.em            | 65  |
| hap.score         | 67  |
| hg18              | 69  |
| hg19              | 70  |
| hg38              | 70  |
| hmht.control      | 70  |
| htr               | 71  |
| hwe               | 73  |
| hwe.cc            | 75  |
| hwe.hardy         | 77  |
| hwe.jags          | 79  |
| invnormal         | 81  |
| inv_chr_pos_a1_a2 | 82  |
| ixy               | 83  |
| KCC               | 83  |
| kin.morgan        | 84  |
| klem              | 85  |
| labelManhattan    | 86  |
| LD22              | 87  |
| LDkl              | 89  |
| log10p            | 91  |
| log10pvalue       | 91  |
| logp              | 92  |
| makeped           | 92  |
| makeRLEplot       | 93  |
| masize            | 94  |
| MCMCgrm           | 99  |
| METAL_forestplot  | 101 |
| metap             | 103 |
| metareg           | 105 |
| mht.control       | 108 |
| mhtplot           | 109 |
| mhtplot.trunc     | 114 |
| mhtplot2          | 117 |
| mia               | 119 |
| miamipLOT         | 120 |
| miamipLOT2        | 122 |
| mr                | 124 |
| mr_forestplot     | 126 |
| mtdt              | 127 |
| mtdt2             | 129 |
| muvar             | 130 |
| mvmeta            | 132 |
| pbsize            | 134 |
| pbsize2           | 136 |
| pedtodot          | 137 |

|                             |     |
|-----------------------------|-----|
| pedtodot_verbatim . . . . . | 140 |
| pfc . . . . .               | 141 |
| pfc.sim . . . . .           | 142 |
| pgc . . . . .               | 144 |
| plot.hap.score . . . . .    | 145 |
| print.hap.score . . . . .   | 146 |
| pvalue . . . . .            | 147 |
| qqfun . . . . .             | 148 |
| qqunif . . . . .            | 150 |
| qtl2dplot . . . . .         | 151 |
| qtl2dplotly . . . . .       | 153 |
| qtl3dplotly . . . . .       | 154 |
| qtlClassifier . . . . .     | 156 |
| qtlFinder . . . . .         | 157 |
| read.ms.output . . . . .    | 159 |
| ReadGRM . . . . .           | 161 |
| ReadGRMBin . . . . .        | 161 |
| revStrand . . . . .         | 162 |
| runshinygap . . . . .       | 162 |
| s2k . . . . .               | 163 |
| sentinels . . . . .         | 164 |
| snpHWE . . . . .            | 166 |
| snptest_sample . . . . .    | 167 |
| tsc . . . . .               | 168 |
| whscore . . . . .           | 170 |
| WriteGRM . . . . .          | 171 |
| WriteGRMBin . . . . .       | 172 |
| xy . . . . .                | 172 |

|              |            |
|--------------|------------|
| <b>Index</b> | <b>173</b> |
|--------------|------------|

---

|     |                                      |
|-----|--------------------------------------|
| a2g | <i>Allele-to-genotype conversion</i> |
|-----|--------------------------------------|

---

## Description

Allele-to-genotype conversion

## Usage

a2g(a1, a2)

## Arguments

|    |                |
|----|----------------|
| a1 | first allele.  |
| a2 | second allele. |

## Description

This function computes the indirect (mediated) effect in a mediation model and either tests its significance or calculates statistical power. It uses regression coefficients from an independent variable to a mediator (a) and from the mediator to an outcome (b) with their standard errors. Variance can be estimated using Sobel, Aroian, or Goodman formulas.

## Usage

```
ab(
  type = c("power", "test"),
  n = 25000,
  a = 0.15,
  sa = 0.01,
  b = log(1.19),
  sb = 0.01,
  alpha = 0.05,
  fold = 1,
  method = c("sobel", "aroian", "goodman")
)
```

## Arguments

|        |                                                                                 |
|--------|---------------------------------------------------------------------------------|
| type   | Character; "test" to perform a significance test, "power" to estimate power.    |
| n      | Numeric; baseline sample size for power calculation (default 25000).            |
| a      | Numeric; regression coefficient from independent variable to mediator.          |
| sa     | Numeric; standard error of a.                                                   |
| b      | Numeric; regression coefficient from mediator to outcome.                       |
| sb     | Numeric; standard error of b.                                                   |
| alpha  | Numeric; significance level (default 0.05) for power calculation.               |
| fold   | Numeric; multiplicative factor for sample size in power calculation.            |
| method | Character; one of "sobel", "aroian", "goodman" specifying the variance formula. |

## Details

The indirect effect is  $ab$ , representing the portion of the effect of  $X$  on  $Y$  transmitted through mediator  $M$ .

Standard errors of the indirect effect are approximated using the delta method:

- Sobel:  $SE = \sqrt{b^2 s_a^2 + a^2 s_b^2}$
- Aroian:  $SE = \sqrt{b^2 s_a^2 + a^2 s_b^2 + s_a^2 s_b^2}$
- Goodman:  $SE = \sqrt{b^2 s_a^2 + a^2 s_b^2 - s_a^2 s_b^2}$

The z-statistic is  $z = ab/SE(ab)$ , assumed to follow a standard normal distribution under  $H_0 : ab = 0$ .

Two-sided p-value:  $p = 2\Phi(-|z|)$  95% confidence interval:  $ab \pm 1.96 \cdot SE(ab)$

Power is based on a non-central chi-square approximation:  $\lambda = z^2 \cdot \text{fold}$   $Power = P(\chi_1^2(\lambda) > \chi_{1,1-\alpha}^2)$ , where  $\chi_{1,1-\alpha}^2$  is the critical value at significance level  $\alpha$ .

These tests assume large samples. For small samples, bootstrap confidence intervals are recommended.

## Value

If type = "test", a named numeric vector with:

- effect: indirect effect estimate ( $ab$ )
- se: standard error
- z: z-statistic
- p: two-sided p-value
- lcl: lower 95% confidence limit
- ucl: upper 95% confidence limit

If type = "power", a named numeric vector with:

- sample\_size: effective sample size ( $\text{fold} \cdot n$ )
- power: estimated statistical power

## References

Freathy RM, Timpson NJ, Lawlor DA, Pouta A, Ben-Shlomo Y, Ruukonen A, Ebrahim S, Shields B, Zeggini E, Weedon MN, Lindgren CM, Lango H, Melzer D, Ferrucci L, Paolisso G, Neville MJ, Karpe F, Palmer CN, Morris AD, Elliott P, Jarvelin MR, Smith GD, McCarthy MI, Hattersley AT, Frayling TM (2008). "Common variation in the FTO gene alters diabetes-related metabolic traits to the extent expected given its effect on BMI." *Diabetes*, **57**(5), 1419-26. doi:10.2337/db071466.

Kline RB (2005). *Principles and Practice of Structural Equation Modeling*, 2nd edition. The Guilford Press, New York, NY.

MacKinnon DP (2008). *Introduction to Statistical Mediation Analysis*. Taylor & Francis Group, New York, NY.

Preacher KJ, Leonardelli GJ (2001). "Calculation for the Sobel Test—An Interactive Calculation Tool for Mediation Tests." <https://quantpsy.org/sobel/sobel.htm>. Accessed 2026-06-07.

## Examples

```
## Not run:
# Test mediation effect
ab(a = 0.15, sa = 0.01,
  b = log(1.19), sb = 0.01,
  method = "aroian")

# Power calculation for 10% increase in sample size
ab(type = "power", n = 25000,
  a = 0.15, sa = 0.01,
  b = log(1.19), sb = 0.01,
  fold = 1.1)
```

```
## End(Not run)
```

ACDE

*Fit AE, ACE or ADE biometric mixed models to nuclear family data*

## Description

Fits classical biometric variance–decomposition models using a linear mixed model formulation implemented with [nlme::lme](#).

## Usage

```
ACDE(model, data, type = c("AE", "ACE", "ADE"), method = "ML")
```

## Arguments

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <code>model</code>  | Fixed-effects formula.                              |
| <code>data</code>   | Data frame in long format (one row per individual). |
| <code>type</code>   | Model type: "AE", "ACE" or "ADE".                   |
| <code>method</code> | Estimation method: "ML" (default) or "REML".        |

## Details

The function estimates additive genetic (A), shared environmental (C), dominance genetic (D), and unique environmental (E) variance components from **nuclear family data (parents and offspring)** in long format.

Supported family structures:

- Parent-child trios (one offspring)
- Nuclear families with **any number of siblings**

The function automatically detects the family structure and selects the correct additive-genetic parameterisation.

Only AE, ACE and ADE models are fitted.

Phenotypic variance is decomposed as

$$V_P = V_A + V_C + V_D + V_E$$

The model is fitted as a linear mixed model with family-level random effects and individual residual variance.

### Automatic family detection:

The function detects whether families contain one or multiple offspring.

- **Trios:** collapsed additive parameterisation.
- **Siblings:** transmission decomposition parameterisation.

**Trio parameterisation:**

Additive effects represented as:

$$A = 0.5Mother + 0.5Father + 1Child$$

This reproduces the expected parent–offspring covariance:

$$Cov = 1/2V_A$$

**Multi-sibling parameterisation:**

Additive genetic variance is decomposed into:

- maternal transmission ( $A_m$ )
- paternal transmission ( $A_f$ )
- Mendelian sampling ( $M_s$ )

For offspring:

$$A = A_m + A_f + M_s$$

Total additive variance:

$$V_A = 2(\sigma_{A_m}^2 + \sigma_{A_f}^2) + \sigma_{M_s}^2$$

This produces correct covariances:

- Parent–offspring:  $1/2V_A$
- Sibling–sibling:  $1/2V_A$

This formulation generalises to **any number of siblings**.

**Identifiability of C and D:**

Nuclear family data cannot fully separate shared environment (C) and dominance (D). ACE and ADE models should be interpreted jointly.

**Value**

Object of class "ACDEfit" containing:

- fit nlme::lme object
- var Variance components (A,C,D,E)
- h2 Narrow-sense heritability
- c2 Shared environment (ACE only)
- d2 Dominance (ADE only)
- H2 Broad-sense heritability (ADE only)

**Required columns in data**

- familyid Nuclear family identifier.
- var1 Maternal transmission coefficient.
- var2 Paternal transmission coefficient.
- var3 Offspring (Mendelian sampling) indicator.

These variables encode expected genetic transmission and **are not role indicators**.

Coding for a nuclear family:



| role   | var1 | var2 | var3 |
|--------|------|------|------|
| father | 0    | 1    | 0    |
| mother | 1    | 0    | 0    |
| child  | 1    | 1    | 1    |

For **multiple siblings**, each offspring receives identical coding:

| role   | var1 | var2 | var3 |
|--------|------|------|------|
| father | 0    | 1    | 0    |
| mother | 1    | 0    | 0    |
| sib1   | 1    | 1    | 1    |
| sib2   | 1    | 1    | 1    |
| sib3   | 1    | 1    | 1    |

### Note

This complements `pbsize()` and `fbsize()`.

### Author(s)

ChatGPT

### Examples

```
library(nlme)
set.seed(1)

simulate_families <- function(n_fam = 200)
{
  VA <- 0.4; VC <- 0.2; VD <- 0.1; VE <- 0.3
  out <- list()

  for(f in 1:n_fam){
    Cfam <- rnorm(1,0,sqrt(VC))
    Af <- rnorm(1,0,sqrt(VA))
    Am <- rnorm(1,0,sqrt(VA))

    make_child <- function(){
      Mend <- rnorm(1,0,sqrt(0.5*VA))
      A <- 0.5*(Af+Am)+Mend
      D <- rnorm(1,0,sqrt(VD))
      E <- rnorm(1,0,sqrt(VE))
      A + D + Cfam + E
    }

    out[[f]] <- data.frame(
      familyid=f,
      role=c("father","mother","sib1","sib2","sib3"),
      y=c(
        Af + Cfam + rnorm(1,0,sqrt(VE)),
        Am + Cfam + rnorm(1,0,sqrt(VE)),
        make_child(), make_child(), make_child()
      )
    )
  }
}
```

```

    )
  }

  dat <- do.call(rbind,out)

  dat$var1 <- as.integer(dat$role=="mother")
  dat$var2 <- as.integer(dat$role=="father")
  dat$var3 <- as.integer(grepl("sib", dat$role))
  dat
}

dat <- simulate_families()

AE <- ACDE(y~1, dat, "AE")
ACE <- ACDE(y~1, dat, "ACE")
ADE <- ACDE(y~1, dat, "ADE")

anova(AE$fit, ACE$fit, ADE$fit)

#####
# Create rectangular variance table (important!)
#####

summary(AE)
summary(ACE)
summary(ADE)

require(gap.datasets)
model <- bwt ~ male + first + midage + highage + birthyr
AE <- ACDE(model,mfblong)
ACE <- ACDE(model,mfblong,type="ACE")
ADE <- ACDE(model,mfblong,type="ADE")
anova(AE$fit,ACE$fit,ADE$fit)

```

---

ACE\_CI

*Bootstrap ACE confidence intervals*


---

## Description

Estimates ACE components (additive genetic, shared environment, unique environment) using non-parametric bootstrap resampling of twin data (MZ and DZ pairs).

## Usage

```

ACE_CI(
  mzData,
  dzData,
  selV,
  n.boot = 1000,
  conf.level = 0.95,
  use = "complete.obs",
  bounds = FALSE,
  seed = NULL,

```

```
    verbose = interactive()
  )
```

### Arguments

|            |                                                         |
|------------|---------------------------------------------------------|
| mzData     | Data frame for monozygotic twins.                       |
| dzData     | Data frame for dizygotic twins.                         |
| selV       | Character vector of length 2 specifying twin variables. |
| n.boot     | Number of bootstrap replications.                       |
| conf.level | Confidence level for percentile intervals.              |
| use        | Missing-data handling method passed to cor().           |
| bounds     | Logical; constrain ACE estimates to [0, 1].             |
| seed       | Random seed for reproducibility.                        |
| verbose    | Logical; show progress messages.                        |

### Value

A list containing:

- observed: ACE estimates from original data
- bootstrap: bootstrap replicate estimates
- summary: bootstrap summary statistics

### Examples

```
## Not run:
set.seed(123)
res <- ACE_CI(mzData = mzm, dzData = dzm, selV = c("bmi1", "bmi2"))
print(res)

## End(Not run)
```

---

 AE3

*AE model using nuclear family trios*


---

### Description

AE model using nuclear family trios

### Usage

```
AE3(model, random, data, seed = 1234, n.sim = 50000, verbose = TRUE)
```

### Arguments

|         |                                                  |
|---------|--------------------------------------------------|
| model   | a linear mixed model formula, see example below. |
| random  | random effect, see exampe below.                 |
| data    | data to be analyzed.                             |
| seed    | random number seed.                              |
| n.sim   | number of simulations.                           |
| verbose | a flag for printing out results.                 |

**Details**

This function is adapted from example 7.1 of Rabe-Hesketh et al. (2008). It also provides heritability estimate and confidence intervals.

**Value**

The returned value is a list containing:

- lme.result the linear mixed model result.
- h2 the heritability estimate.
- CL confidence intervals.

**Note**

Adapted from f.mbf.R from the paper.

**Author(s)**

Jing Hua Zhao

**References**

Rabe-Hesketh S, Skrondal A, Gjessing HK (2008). “Biometrical modeling of twin and family data using standard mixed model software.” *Biometrics*, **64**(1), 280-8. doi:[10.1111/j.15410420.2007.00803.x](https://doi.org/10.1111/j.15410420.2007.00803.x).

**Examples**

```
## Not run:
require(gap.datasets)
AE3(bwt ~ male + first + midage + highage + birthyr,
    list(familyid = pdIdent(~var1 + var2 + var3 -1)), mfblong)

## End(Not run)
```

---

allele.recode

*Allele recoding*


---

**Description**

Allele recoding

**Usage**

```
allele.recode(a1, a2, miss.val = NA)
```

**Arguments**

|          |                |
|----------|----------------|
| a1       | first allele.  |
| a2       | second allele. |
| miss.val | missing value. |

---

asplot

---

*Regional association plot*

---

**Description**

Regional association plot

**Usage**

```
asplot(  
  locus,  
  map,  
  genes,  
  flanking = 1000,  
  best.pval = NULL,  
  sf = c(4, 4),  
  logpmax = 10,  
  pch = 21  
)
```

**Arguments**

|           |                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------|
| locus     | Data frame with columns c("CHR", "POS", "NAME", "PVAL", "RSQR") containing association results.          |
| map       | Genetic map, i.e. c("POS", "THETA", "DIST").                                                             |
| genes     | Gene annotation with columns c("START", "STOP", "STRAND", "GENE").                                       |
| flanking  | Flanking length.                                                                                         |
| best.pval | Best p value for the locus of interest.                                                                  |
| sf        | scale factors for p values and recombination rates, smaller values are necessary for gene dense regions. |
| logpmax   | Maximum value for $-\log_{10}(p)$ .                                                                      |
| pch       | Plotting character for the SNPs to be highlighted, e.g., 21 and 23 refer to circle and diamond.          |

**Details**

This function obtains regional association plot for a particular locus, based on the information about recombination rates, linkage disequilibria between the SNP of interest and neighbouring ones, and single-point association tests p values.

Note that the best p value is not necessarily within locus in the original design.

**Author(s)**

Paul de Bakker, Jing Hua Zhao, Shengxu Li

## References

Saxena R, Voight BF, Lyssenko V, Burt NP, de Bakker PI, Chen H, Roix JJ, Kathiresan S, Hirschhorn JN, Daly MJ, Hughes TE, Groop L, Altshuler D, Almgren P, Florez JC, Meyer J, Ardlie K, Bengtsson Boström K, Isomaa B, Lettre G, Lindblad U, Lyon HN, Melander O, Newton-Cheh C, Nilsson P, Orho-Melander M, Råstam L, Speliotes EK, Taskinen MR, Tuomi T, Guiducci C, Berglund A, Carlson J, Gianniny L, Hackett R, Hall L, Holmkvist J, Laurila E, Sjögren M, Sterner M, Surti A, Svensson M, Svensson M, Tewhey R, Blumenstiel B, Parkin M, Defelice M, Barry R, Brodeur W, Camarata J, Chia N, Fava M, Gibbons J, Handsaker B, Healy C, Nguyen K, Gates C, Sougnez C, Gage D, Nizzari M, Gabriel SB, Chirn GW, Ma Q, Parikh H, Richardson D, Ricke D, Purcell S (2007). “Genome-wide association analysis identifies loci for type 2 diabetes and triglyceride levels.” *Science*, **316**(5829), 1331-6. doi:10.1126/science.1142358.

## Examples

```
## Not run:
require(gap.datasets)
asplot(CDKNlocus, CDKNmap, CDKNgenes)
title("CDKN2A/CDKN2B Region")
asplot(CDKNlocus, CDKNmap, CDKNgenes, best.pval=5.4e-8, sf=c(3,6))

## End(Not run)
```

---

b2r

---

*Obtain correlation coefficients and their variance-covariances*


---

## Description

Obtain correlation coefficients and their variance-covariances

## Usage

```
b2r(b, s, rho, n)
```

## Arguments

|     |                                               |
|-----|-----------------------------------------------|
| b   | the vector of linear regression coefficients. |
| s   | the corresponding vector of standard errors.  |
| rho | triangular array of between-SNP correlation.  |
| n   | the sample size.                              |

## Details

This function converts linear regression coefficients of phenotype on single nucleotide polymorphisms (SNPs) into Pearson correlation coefficients with their variance-covariance matrix. It is useful as a preliminary step for meta-analyze SNP-trait associations at a given region. Between-SNP correlations (e.g., from HapMap) are required as auxiliary information.

**Value**

The returned value is a list containing:

- `r` the vector of correlation coefficients.
- `V` the variance-covariance matrix of correlations.

**Author(s)**

Jing Hua Zhao

**References**

Elston RC (1975). “On the correlation between correlations.” *Biometrika*, **62**(1), 133-140. doi:10.1093/biomet/62.1.133.

Becker BJ (2000). “Multivariate meta-analysis.” In Tinsley HE, Brown SD (eds.), *Handbook of Applied Multivariate Statistics and Mathematical Modeling*, chapter 17, 499-525. Academic Press, San Diego. ISBN 978-0126913606.

Casella G, Berger RL (2002). *Statistical Inference*, 2 edition. Duxbury. ISBN 978-0-534-24312-8.

**See Also**

[mvmeta](#), [LD22](#)

**Examples**

```
## Not run:
n <- 10
r <- c(1,0.2,1,0.4,0.5,1)
b <- c(0.1,0.2,0.3)
s <- c(0.4,0.3,0.2)
bs <- b2r(b,s,r,n)

## End(Not run)
```

---

BFDP

*Bayesian false-discovery probability*


---

**Description**

Bayesian false-discovery probability

**Usage**

```
BFDP(a, b, pi1, W, logscale = FALSE)
```

**Arguments**

|                       |                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| <code>a</code>        | parameter value at which the power is to be evaluated.                                                         |
| <code>b</code>        | the variance for <code>a</code> , or the upper point ( $RR_{hi}$ ) of a 95%CI if <code>logscale=FALSE</code> . |
| <code>pi1</code>      | the prior probability of a non-null association.                                                               |
| <code>W</code>        | the prior variance.                                                                                            |
| <code>logscale</code> | <code>FALSE</code> =the original scale, <code>TRUE</code> =the log scale.                                      |

## Details

This function calculates BFDP, the approximate  $P(H_0|\hat{\theta})$ , given an estimate of the log relative risk,  $\hat{\theta}$ , the variance of this estimate,  $V$ , the prior variance,  $W$ , and the prior probability of a non-null association. When logscale=TRUE, the function accepts an estimate of the relative risk,  $\hat{R}$ , and the upper point of a 95% confidence interval  $RR_{hi}$ .

## Value

The returned value is a list with the following components: PH0. probability given a,b). PH1. probability given a,b,W). BF. Bayes factor,  $P_{H_0}/P_{H_1}$ . BFDP. Bayesian false-discovery probability. ABF. approximate Bayes factor. ABFDP. approximate Bayesian false-discovery probability.

## Note

Adapted from BFDP functions by Jon Wakefield on 17th April, 2007.

## Author(s)

Jon Wakefield, Jing Hua Zhao

## References

Wakefield J (2007). "A Bayesian measure of the probability of false discovery in genetic epidemiology studies." *Am J Hum Genet*, **81**(2), 208-27. doi:10.1086/519024.

## See Also

[FPRP](#)

## Examples

```
## Not run:
# Example from BDFP.xls by Jon Wakefield and Stephanie Monnier
# Step 1 - Pre-set an BFDP-level threshold for noteworthiness: BFDP values below this
#           threshold are noteworthy
# The threshold is given by R/(1+R) where R is the ratio of the cost of a false
# non-discovery to the cost of a false discovery

T <- 0.8

# Step 2 - Enter up values for the prior that there is an association

pi0 <- c(0.7,0.5,0.01,0.001,0.00001,0.6)

# Step 3 - Enter the value of the OR that is the 97.5% point of the prior, for example
#           if we pick the value 1.5 we believe that the prior probability that the
#           odds ratio is bigger than 1.5 is 0.025.

ORhi <- 3

W <- (log(ORhi)/1.96)^2
W

# Step 4 - Enter OR estimate and 95% confidence interval (CI) to obtain BFDP
```



```

OR <- 1.316
OR_L <- 1.10
OR_U <- 2.50
logOR <- log(OR)
selogOR <- (log(OR_U)-log(OR))/1.96
r <- W/(W+selogOR^2)
r
z <- logOR/selogOR
z
ABF <- exp(-z^2*r/2)/sqrt(1-r)
ABF
FF <- (1-pi0)/pi0
FF
BFDPex <- FF*ABF/(FF*ABF+1)
BFDPex
pi0[BFDPex>T]

## now turn to BFD

pi0 <- c(0.7,0.5,0.01,0.001,0.00001,0.6)
ORhi <- 3
OR <- 1.316
OR_U <- 2.50
W <- (log(ORhi)/1.96)^2
z <- BFD(OR,OR_U,pi0,W)
z

## End(Not run)

```

bt

*Bradley-Terry model for contingency table***Description**

Bradley-Terry model for contingency table

**Usage**

```
bt(x)
```

**Arguments**

x                      the data table.

**Details**

This function calculates statistics under Bradley-Terry model.

**Value**

The returned value is a list containing:

- y A column of 1.
- count the frequency count/weight.

- allele the design matrix.
- bt.glm a glm.fit object.
- etdt.dat a data table that can be used by ETDT.

## Note

Adapted from a SAS macro for data in the example section.

## Author(s)

Jing Hua Zhao

## References

Bradley RA, Terry ME (1952). "Rank analysis of incomplete block designs." *Biometrika*, **39**, 324-335.

Sham PC, Curtis D (1995). "An extended transmission/disequilibrium test (TDT) for multi-allele marker loci." *Ann Hum Genet*, **59**(3), 323-36. doi:[10.1111/j.14691809.1995.tb00751.x](https://doi.org/10.1111/j.14691809.1995.tb00751.x).

Copeman JB, Cucca F, Hearne CM, Cornall RJ, Reed PW, Rønningen KS, Undlien DE, Nisticò L, Buzzetti R, Tosi R, et al. (1995). "Linkage disequilibrium mapping of a type 1 diabetes susceptibility gene (IDDM7) to chromosome 2q31-q33." *Nat Genet*, **9**(1), 80-5. doi:[10.1038/ng019580](https://doi.org/10.1038/ng019580).

## See Also

[mtdt](#)

## Examples

```
## Not run:
x <- matrix(c(0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 3, 0,0, 0, 2, 3, 0, 0, 0, 0,
              2,3,26,35, 7,0, 2,10,11, 3, 4, 1,
              2,3,22,26, 6,2, 4, 4,10, 2, 2, 0,
              0,1, 7,10, 2,0, 0, 2, 2, 1, 1, 0,
              0,0, 1, 4, 0,1, 0, 1, 0, 0, 0, 0,
              0,2, 5, 4, 1,1, 0, 0, 0, 2, 0, 0,
              0,0, 2, 6, 1,0, 2, 0, 2, 0, 0, 0,
              0,3, 6,19, 6,0, 0, 2, 5, 3, 0, 0,
              0,0, 3, 1, 1,0, 0, 0, 1, 0, 0, 0,
              0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 0, 0,0, 0, 0, 0, 0, 0, 0),nrow=12)

# Bradley-Terry model, only deviance is available in glm
# (SAS gives score and Wald statistics as well)
bt.ex<-bt(x)
anova(bt.ex$bt.glm)
summary(bt.ex$bt.glm)

## End(Not run)
```

ccsize

*Power and sample size for case-cohort design***Description**

Power and sample size for case-cohort design

**Usage**

ccsize(n, q, pD, p1, theta, alpha, beta = 0.2, power = TRUE, verbose = FALSE)

**Arguments**

|         |                                                              |
|---------|--------------------------------------------------------------|
| n       | the total number of subjects in the cohort.                  |
| q       | the sampling fraction of the subcohort.                      |
| pD      | the proportion of the failures in the full cohort.           |
| p1      | proportions of the two groups (p2=1-p1).                     |
| theta   | log-hazard ratio for two groups.                             |
| alpha   | type I error – significant level.                            |
| beta    | type II error.                                               |
| power   | if specified, the power for which sample size is calculated. |
| verbose | error messages are explicitly printed out.                   |

**Details**

The power of the test is according to

$$\Phi \left( Z_{\alpha} + m^{1/2} \theta \sqrt{\frac{p_1 p_2 p_D}{q + (1 - q) p_D}} \right)$$

where  $\alpha$  is the significance level,  $\theta$  is the log-hazard ratio for two groups,  $p_j$ ,  $j=1, 2$ , are the proportion of the two groups in the population.  $m$  is the total number of subjects in the subcohort,  $p_D$  is the proportion of the failures in the full cohort, and  $q$  is the sampling fraction of the subcohort.

Alternatively, the sample size required for the subcohort is

$$m = n B p_D / (n - B(1 - p_D))$$

where  $B = (Z_{1-\alpha} + Z_{\beta})^2 / (\theta^2 p_1 p_2 p_D)$ , and  $n$  is the size of cohort.

When infeasible configurations are specified, a sample size of -999 is returned.

**Value**

a value indicating the power or required sample size.

**Note**

Programmed for EPIC study. keywords misc

**Author(s)**

Jing Hua Zhao

**References**

Cai J, Zeng D (2004). "Sample size/power calculation for case-cohort studies." *Biometrics*, **60**(4), 1015-24. doi:10.1111/j.0006341X.2004.00257.x.

**See Also**[pbsize](#)**Examples**

```
## Not run:
# Table 1 of Cai & Zeng (2004).
alpha <- 0.05
table1 <- rbind(
  transform(
    within(expand.grid(
      pD = c(0.10, 0.05),
      p1 = c(0.3, 0.5),
      theta = c(0.5, 1.0),
      q = c(0.1, 0.2)
    ), {
      n <- 1000
      power <- mapply(ccsize,
        n = n, q = q, pD = pD, p1 = p1, theta = theta,
        MoreArgs = list(alpha = alpha)
      )
    }
  ),
  power = signif(power, 3)
),
  transform(
    within(expand.grid(
      pD = c(0.05, 0.01),
      p1 = c(0.3, 0.5),
      theta = c(0.5, 1.0),
      q = c(0.01, 0.02)
    ), {
      n <- 5000
      power <- mapply(ccsize,
        n = n, q = q, pD = pD, p1 = p1, theta = theta,
        MoreArgs = list(alpha = alpha)
      )
    }
  ),
  power = signif(power, 3)
)

# ARIC study
aric <- within(
  data.frame(
    n = 15792,
    pD = 0.03,
    p1 = 0.25,
```

```

    hr = c(1.35, 1.40, 1.45),
    q = c(1463, 722, 468) / 15792
  ), {
    alpha <- 0.05
    beta <- 0.2
    power <- mapply(ccsize,
      n = n, q = q, pD = pD, p1 = p1, theta = log(hr),
      MoreArgs = list(alpha = alpha)
    )
    ssize <- mapply(ccsize,
      n = n, q = q, pD = pD, p1 = p1, theta = log(hr),
      MoreArgs = list(alpha = alpha, beta = beta, power = FALSE)
    )
    power <- signif(power, 3)
  }
)

# EPIC study
epic <- within(
  expand.grid(
    pD = c(0.3, 0.2, 0.1, 0.05),
    p1 = seq(0.1, 0.5, by = 0.1),
    hr = seq(1.1, 1.4, by = 0.1)
  ), {
    n <- 25000
    q <- 0.1
    alpha <- 5e-8
    beta <- 0.2
    ssize <- mapply(ccsize,
      n = n, q = q, pD = pD, p1 = p1, theta = log(hr),
      MoreArgs = list(alpha = alpha, beta = beta, power = FALSE)
    )
  }
)
epic <- subset(epic, !is.na(ssize) & ssize > 0)

# exhaustive search
search <- within(
  expand.grid(
    pD = c(0.3, 0.2, 0.1, 0.05),
    p1 = seq(0.1, 0.5, by = 0.1),
    hr = seq(1.1, 1.4, by = 0.1),
    q = seq(0.01, 0.5, by = 0.01)
  ), {
    n <- 25000
    alpha <- 5e-8
    power <- mapply(ccsize,
      n = n, q = q, pD = pD, p1 = p1, theta = log(hr),
      MoreArgs = list(alpha = alpha)
    )
    nq <- n * q
  }
)

## End(Not run)

```

chow.test

*Chow's test for heterogeneity in two regressions***Description**

Chow's test for heterogeneity in two regressions

**Usage**

```
chow.test(y1, x1, y2, x2, x = NULL)
```

**Arguments**

|    |                                          |
|----|------------------------------------------|
| y1 | a vector of dependent variable.          |
| x1 | a matrix of independent variables.       |
| y2 | a vector of dependent variable.          |
| x2 | a matrix of independent variables.       |
| x  | a known matrix of independent variables. |

**Details**

Chow's test is for differences between two or more regressions. Assuming that errors in regressions 1 and 2 are normally distributed with zero mean and homoscedastic variance, and they are independent of each other, the test of regressions from sample sizes  $n_1$  and  $n_2$  is then carried out using the following steps. 1. Run a regression on the combined sample with size  $n = n_1 + n_2$  and obtain within group sum of squares called  $S_1$ . The number of degrees of freedom is  $n_1 + n_2 - k$ , with  $k$  being the number of parameters estimated, including the intercept. 2. Run two regressions on the two individual samples with sizes  $n_1$  and  $n_2$ , and obtain their within group sums of square  $S_2 + S_3$ , with  $n_1 + n_2 - 2k$  degrees of freedom. 3. Conduct an  $F_{(k, n_1 + n_2 - 2k)}$  test defined by

$$F = \frac{[S_1 - (S_2 + S_3)]/k}{[(S_2 + S_3)/(n_1 + n_2 - 2k)]}$$

If the  $F$  statistic exceeds the critical  $F$ , we reject the null hypothesis that the two regressions are equal.

In the case of haplotype trend regression, haplotype frequencies from combined data are known, so can be directly used.

**Value**

The returned value is a vector containing (please use subscript to access them):

- F the F statistic.
- df1 the numerator degree(s) of freedom.
- df2 the denominator degree(s) of freedom.
- p the p value for the F test.

**Note**

adapted from chow.R.

**Author(s)**

Shigenobu Aoki, Jing Hua Zhao

**References**

Chow GC (1960). "Tests of Equality Between Sets of Coefficients in Two Linear Regressions." *Econometrica*, **28**(3), 591-605. doi:[10.2307/1910133](https://doi.org/10.2307/1910133).

**See Also**[htr](#)**Examples**

```
## Not run:
dat1 <- matrix(c(
  1.2, 1.9, 0.9,
  1.6, 2.7, 1.3,
  3.5, 3.7, 2.0,
  4.0, 3.1, 1.8,
  5.6, 3.5, 2.2,
  5.7, 7.5, 3.5,
  6.7, 1.2, 1.9,
  7.5, 3.7, 2.7,
  8.5, 0.6, 2.1,
  9.7, 5.1, 3.6), byrow=TRUE, ncol=3)

dat2 <- matrix(c(
  1.4, 1.3, 0.5,
  1.5, 2.3, 1.3,
  3.1, 3.2, 2.5,
  4.4, 3.6, 1.1,
  5.1, 3.1, 2.8,
  5.2, 7.3, 3.3,
  6.5, 1.5, 1.3,
  7.8, 3.2, 2.2,
  8.1, 0.1, 2.8,
  9.5, 5.6, 3.9), byrow=TRUE, ncol=3)

y1<-dat1[,3]
y2<-dat2[,3]
x1<-dat1[,1:2]
x2<-dat2[,1:2]
chow.test.r<-chow.test(y1,x1,y2,x2)
# from http://aoki2.si.gunma-u.ac.jp/R/

## End(Not run)
```

**Description**

SNP id by chr:pos+a1/a2

**Usage**

```
chr_pos_a1_a2(  
  chr,  
  pos,  
  a1,  
  a2,  
  prefix = "chr",  
  seps = c(":", "_", "-"),  
  uppercase = TRUE  
)
```

**Arguments**

- chr           Chromosome.
- pos           Position.
- a1            Allele 1.
- a2            Allele 2.
- prefix        Prefix of the identifier.
- seps          Delimiters.
- uppercase     A flag to return in upper case.

**Details**

This function generates unique identifiers for variants

**Value**

Identifier.

**Examples**

```
# rs12075  
chr_pos_a1_a2(1,159175354,"A","G",prefix="chr",seps=c(":", "_", "-"),uppercase=TRUE)
```

---

|       |                                                                |
|-------|----------------------------------------------------------------|
| ci2ms | <i>Effect size and standard error from confidence interval</i> |
|-------|----------------------------------------------------------------|

---

**Description**

Effect size and standard error from confidence interval

**Usage**

```
ci2ms(ci, logscale = TRUE, alpha = 0.05)
```



## Arguments

|          |                                                                                                              |
|----------|--------------------------------------------------------------------------------------------------------------|
| ci       | confidence interval (CI). The delimiter between lower and upper limit is either a hyphen (-) or en dash (–). |
| logscale | a flag indicating the confidence interval is based on a log-scale.                                           |
| alpha    | Type 1 error.                                                                                                |

## Details

Effect size is a measure of strength of the relationship between two variables in a population or parameter estimate of that population. Without loss of generality, denote  $m$  and  $s$  to be the mean and standard deviation of a sample from  $N(\mu, \sigma^2)$ . Let  $z \sim N(0, 1)$  with cutoff point  $z_\alpha$ , confidence limits  $L, U$  in a CI are defined as follows,

$$\begin{aligned} L &= m - z_\alpha s \\ U &= m + z_\alpha s \end{aligned}$$

$\Rightarrow U + L = 2m, U - L = 2z_\alpha s$ . Consequently,

$$\begin{aligned} m &= \frac{U + L}{2} \\ s &= \frac{U - L}{2z_\alpha} \end{aligned}$$

Effect size in epidemiological studies on a binary outcome is typically reported as odds ratio from a logistic regression or hazard ratio from a Cox regression,  $L \equiv \log(L), U \equiv \log(U)$ .

## Value

Based on CI, the function provides a list containing estimates

- $m$  effect size ( $\log(\text{OR})$ )
- $s$  standard error
- direction a decrease/increase (-/+) sign such that  $\text{sign}(m) = -1, 0, 1$ , is labelled "-", "0", "+", respectively as in PhenoScanner.

## Examples

```
# rs3784099 and breast cancer recurrence/mortality
ms <- ci2ms("1.28-1.72")
print(ms)
# Vector input
ci2 <- c("1.28-1.72", "1.25-1.64")
ms2 <- ci2ms(ci2)
print(ms2)
```

---

circos.cis.vs.trans.plot

*Circos plot of cis vs trans protein associations*


---

## Description

Visualizes cis- and trans-protein associations along chromosomes using a circular genome plot (Circos plot).

## Usage

```
circos.cis.vs.trans.plot(f, panel, id, radius = 1e+06)
```

## Arguments

|                     |                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>f</code>      | Character string. Path to a tab-delimited file containing pQTL hits. Must include columns: CHR, BP, SNP, and prot. |
| <code>panel</code>  | Data frame. Annotation panel containing at least: prot, uniprot, chr, start, end, and gene.                        |
| <code>id</code>     | Character identifier passed to <code>cis.vs.trans.classification()</code> .                                        |
| <code>radius</code> | Numeric distance threshold defining the cis region (default: 1e6 base pairs).                                      |

## Details

The function classifies protein-QTL (pQTL) hits into cis and trans associations and displays genomic links between protein loci and target genes.

This version is robust to differences in column naming conventions (e.g. dot vs underscore formats).

This function relies on **circulize** for visualization and assumes that `cis.vs.trans.classification()` returns a list containing a data frame data with cis/trans annotations.

Column names may vary depending on package version. The implementation attempts flexible matching of:

- `p.chr` or `p_chr`
- `cis.start` or `cis_start`
- `cis.end` or `cis_end`
- `p.gene` or `p_gene`
- `p.prot` or `p_prot` or `prot`

## Value

A Circos plot is drawn to the current graphics device. The function returns invisibly.

## Examples

```
## Not run:
f <- file.path(find.package("pQTLtools"), "tests", "INF1.merge")
circos.cis.vs.trans.plot(f, inf1, id = "uniprot")

## End(Not run)
```

---

|                             |                             |
|-----------------------------|-----------------------------|
| <code>circos.cnvplot</code> | <i>circos plot of CNVs.</i> |
|-----------------------------|-----------------------------|

---

### Description

circos plot of CNVs.

### Usage

```
circos.cnvplot(data)
```

### Arguments

|                   |                                                      |
|-------------------|------------------------------------------------------|
| <code>data</code> | CNV data containing chromosome, start, end and freq. |
|-------------------|------------------------------------------------------|

### Details

The function plots frequency of CNVs.

### Value

None.

### Examples

```
## Not run:
circos.cnvplot(cnv)

## End(Not run)
```

---

|                             |                                                   |
|-----------------------------|---------------------------------------------------|
| <code>circos.mhtplot</code> | <i>circos Manhattan plot with gene annotation</i> |
|-----------------------------|---------------------------------------------------|

---

### Description

circos Manhattan plot with gene annotation

### Usage

```
circos.mhtplot(data, glist)
```

### Arguments

|                    |                  |
|--------------------|------------------|
| <code>data</code>  | Data to be used. |
| <code>glist</code> | A gene list.     |

### Details

The function generates circos Manhattan plot with gene annotation.

**Value**

None.

**Examples**

```
## Not run:
require(gap.datasets)
glist <- c("IRS1", "SPRY2", "FTO", "GRIK3", "SNED1", "HTR1A", "MARCH3", "WISP3",
           "PPP1R3B", "RP1L1", "FDFT1", "SLC39A14", "GFRA1", "MC4R")
circos.mhtplot(mhtdata, glist)

## End(Not run)
```

---

circos.mhtplot2

*Another circos Manhattan plot*

---

**Description**

Another circos Manhattan plot

**Usage**

```
circos.mhtplot2(dat, labs, species = "hg18", ticks = 0:3 * 10, ymax = 30)
```

**Arguments**

|                      |                                                     |
|----------------------|-----------------------------------------------------|
| <code>dat</code>     | Data to be plotted with variables chr, pos, log10p. |
| <code>labs</code>    | Data on labels.                                     |
| <code>species</code> | Genome build.                                       |
| <code>ticks</code>   | Tick positions.                                     |
| <code>ymax</code>    | maximum value for y-axis.                           |

**Details**

This is adapted from work for a recent publication. It enables a y-axis to the  $-\log_{10}(P)$  for association statistics

**Value**

There is no return value but a plot.

**Examples**

```
## Not run:
require(gap.datasets)
library(dplyr)
glist <- c("IRS1", "SPRY2", "FTO", "GRIK3", "SNED1", "HTR1A", "MARCH3", "WISP3",
           "PPP1R3B", "RP1L1", "FDFT1", "SLC39A14", "GFRA1", "MC4R")
testdat <- mhtdata[c("chr", "pos", "p", "gene", "start", "end")] %>%
  rename(log10p=p) %>%
  mutate(chr=paste0("chr", chr), log10p=-log10(log10p))
```

```

dat <- mutate(testdat,start=pos,end=pos) %>%
  select(chr,start,end,log10p)
labs <- subset(testdat,gene %in% glist) %>%
  group_by(gene,chr,start,end) %>%
  summarize() %>%
  mutate(cols="blue") %>%
  select(chr,start,end,gene,cols)
labs[2,"cols"] <- "red"
ticks <- 0:3*5
circos.mhtplot2(dat,labs,ticks=ticks,ymax=max(ticks))
# https://www.rapidtables.com/web/color/RGB_Color.html

## End(Not run)

```

---

cis.vs.trans.classification

*Cis/trans classification of pQTL signals*

---

## Description

Classify genetic association signals as cis or trans relative to the gene encoding the target protein.

## Usage

```

cis.vs.trans.classification(
  hits,
  panel,
  id = "uniprot",
  radius = 1e+06,
  verbose = TRUE
)

```

## Arguments

|         |                                                                                                                                                                                                               |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hits    | Data frame of association signals containing at least: <ul style="list-style-type: none"> <li>• identifier column (specified by id)</li> <li>• chromosome column Chr</li> <li>• position column bp</li> </ul> |
| panel   | Annotation data frame containing: <ul style="list-style-type: none"> <li>• uniprot (or other id column)</li> <li>• gene, chr, start, end</li> </ul>                                                           |
| id      | Join key (default: "uniprot").                                                                                                                                                                                |
| radius  | Cis window size in base pairs (default: 1e6).                                                                                                                                                                 |
| verbose | Logical; print summary if TRUE.                                                                                                                                                                               |

## Details

A variant is classified as cis if it lies on the same chromosome as the gene and within a specified window around the gene boundaries.

**Value**

A list containing:

- **data**: annotated data frame with cis/trans classification
  - p.gene, p.chr, p.start, p.end
  - cis (logical)
  - cis.trans ("cis" / "trans")
  - cis.start, cis.end
- **table**: gene × cis/trans contingency table
- **total**: named vector of total cis and trans counts

**Examples**

```
#-----
# minimal reproducible example
#-----

hits <- data.frame(
  uniprot = c("P1", "P2", "P3", "P1"),
  Chr = c("1", "1", "2", "1"),
  bp = c(100000, 2000000, 500000, 105000)
)

panel <- data.frame(
  uniprot = c("P1", "P2", "P3"),
  gene = c("G1", "G2", "G3"),
  chr = c("1", "1", "2"),
  start = c(90000, 1500000, 400000),
  end = c(110000, 2500000, 600000)
)

ct <- cis.vs.trans.classification(hits, panel)

ct$total
head(ct$data)
with(ct$data, table(p.gene, cis.trans))
```

---

cnvplot

*genomewide plot of CNVs*


---

**Description**

genomewide plot of CNVs

**Usage**

```
cnvplot(data)
```

**Arguments**

data                      Data to be used.

## Details

The function generates a plot containing genomewide copy number variants (CNV) chr, start, end, freq(uencies).

## Value

None.

## Examples

```
knitr::kable(cnv,caption="A CNV dataset")
cnvplot(cnv)
```

---

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| comp.score | <i>score statistics for testing genetic linkage of quantitative trait</i> |
|------------|---------------------------------------------------------------------------|

---

## Description

score statistics for testing genetic linkage of quantitative trait

## Usage

```
comp.score(
  ibddata = "ibd_dist.out",
  phenotype = "pheno.dat",
  mean = 0,
  var = 1,
  h2 = 0.3
)
```

## Arguments

|           |                                                                                                                                                                                                                                                                                             |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ibddata   | The output file from GENEHUNTER using command "dump ibd". The default file name is <i>ibd_dist.out</i> .                                                                                                                                                                                    |
| phenotype | The file of pedigree structure and trait value. The default file name is "pheno.dat". Columns (no headings) are: family ID, person ID, father ID, mother ID, gender, trait value, where Family ID and person ID must be numbers, not characters. Use character "NA" for missing phenotypes. |
| mean      | (population) mean of the trait, with a default value of 0.                                                                                                                                                                                                                                  |
| var       | (population) variance of the trait, with a default value of 1.                                                                                                                                                                                                                              |
| h2        | heritability of the trait, with a default value of 0.3.                                                                                                                                                                                                                                     |

## Details

The function empirically estimate the variance of the score functions. The variance-covariance matrix consists of two parts: the additive part and the part for the individual-specific environmental effect. Other reasonable decompositions are possible.

This program has the following improvement over "score.r":

1. It works with selected nuclear families

2. Trait data on parents (one parent or two parents), if available, are utilized.
3. Besides a statistic assuming no locus-specific dominance effect, it also computes a statistic that allows for such effect. It computes two statistics instead of one.

Function "merge" is used to merge the IBD data for a pair with the transformed trait data (i.e.,  $w_k w_l$ ).

### Value

a matrix with each row containing the location and the statistics and their p-values.

### Note

Adapt from score2.r.

### Author(s)

Yingwei Peng, Kai Wang

### References

- Kruglyak L, Daly MJ, Reeve-Daly MP, Lander ES (1996). "Parametric and nonparametric linkage analysis: a unified multipoint approach." *Am J Hum Genet*, **58**(6), 1347-63.
- Kruglyak L, Lander ES (1998). "Faster multipoint linkage analysis using Fourier transforms." *J Comput Biol*, **5**(1), 1-7. doi:10.1089/cmb.1998.5.1.
- Wang K (2005). "A likelihood approach for quantitative-trait-locus mapping with selected pedigrees." *Biometrics*, **61**(2), 465-73. doi:10.1111/j.15410420.2005.031213.x.

### Examples

```
## Not run:
# An example based on GENEHUNTER version 2.1, with quantitative trait data in file
# "pheno.dat" generated from the standard normal distribution. The following
# example shows that it is possible to automatically call GENEHUNTER using R
# function "system".

cwd <- getwd()
cs.dir <- file.path(find.package("gap.examples"), "tests/comp.score")
setwd(cs.dir)
dir()
# system("gh < gh.inp")
cs.default <- comp.score()
setwd(cwd)

## End(Not run)
```



cs

*Credible set from summary statistics***Description**

Computes a Bayesian credible set from GWAS summary statistics using Wakefield-style approximate Bayes factors.

**Usage**

```
cs(tbl, b = "Effect", se = "StdErr", log_p = NULL, cutoff = 0.95)
```

**Arguments**

|        |                                                                                                                                            |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------|
| tbl    | A data.frame containing summary statistics.                                                                                                |
| b      | Name of column containing effect sizes.                                                                                                    |
| se     | Name of column containing standard errors.                                                                                                 |
| log_p  | Optional column name containing log p-values. If supplied, z-scores are derived from p-values instead of effect sizes and standard errors. |
| cutoff | Cumulative posterior probability threshold. Default is 0.95 (95% credible set).                                                            |

**Details**

Credible set is often used in fine-mapping.

Posterior probabilities are computed from z-statistics using a numerically stable log-sum-exp implementation from matrixStats.

**Value**

A subset of tbl containing variants in the credible set, ordered by decreasing posterior probability of association.

**Examples**

```
## Not run:
\preformatted{
  zcat ~/rds/results/private/proteomics/scallop-inf1/4E.BP1-1.tbl.gz | \
  awk 'NR==1 || ($1==4 && $2 >= 187158034 - 1e6 && $2 < 187158034 + 1e6)' > 4E.BP1.z
}
tbl <- within(read.delim("4E.BP1.z"),{logp <- logp(Effect/StdErr)})
z <- cs(tbl)
l <- cs(tbl, log_p="logp")

## End(Not run)
```

## ESplot

*Effect-size / Odds-ratio forest plot***Description**

The function accepts parameter estimates and their standard errors from one or more models and produces a horizontal forest plot with confidence intervals.

Two plotting modes are supported:

- `transform="none"` (default): plots estimates on the linear scale (typical for GWAS or Mendelian randomisation beta coefficients).
- `transform="exp"`: plots exponentiated estimates on a log10 axis (typical for odds ratios or hazard ratios). Confidence intervals are computed on the log scale and back-transformed.

**Usage**

```
ESplot(
  ESdat,
  alpha = 0.05,
  fontsize = 12,
  transform = c("none", "exp"),
  xlab = NULL
)
```

**Arguments**

|           |                                                                                                                                                                                                                        |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ESdat     | Data frame with <b>three columns</b> : <ul style="list-style-type: none"> <li>• id Model or trait label</li> <li>• b Effect estimate (beta or log(OR)/log(HR))</li> <li>• se Standard error of the estimate</li> </ul> |
| alpha     | Type-I error rate for the confidence interval (default 0.05 for 95% CI).                                                                                                                                               |
| fontsize  | Base font size used in the plot.                                                                                                                                                                                       |
| transform | Either "none" (linear scale) or "exp" (exponentiated scale).                                                                                                                                                           |
| xlab      | Optional x-axis label. If NULL, a sensible default is used.                                                                                                                                                            |

**Details**

Create a publication-ready forest plot for model effect estimates. The function supports both linear effect sizes (e.g. regression betas) and exponentiated effects (e.g. odds ratios or hazard ratios).

Confidence intervals are computed as

$$estimate \pm z_{\alpha/2} \times SE$$

When `transform="exp"`, estimates are interpreted as log(OR) or log(HR) and are exponentiated before plotting. The x-axis is displayed on a log10 scale and the reference line is placed at 1.

This function replaces an earlier base-R implementation and provides a consistent interface for GWAS, Mendelian randomisation, and epidemiological regression analyses.

**Value**

A ggplot2 plot object.

**Author(s)**

Jing Hua Zhao

**Examples**

```
## Example 1: Linear effect sizes (GWAS / MR)
rs12075 <- data.frame(
  id=c("CCL2", "CCL7", "CCL8", "CCL11", "CCL13", "CXCL6", "Monocytes"),
  b=c(0.1694, -0.0899, -0.0973, 0.0749, 0.189, 0.0816, 0.0338387),
  se=c(0.0113, 0.013, 0.0116, 0.0114, 0.0114, 0.0115, 0.00713386)
)
ESplot(rs12075)

## Example 2: Odds ratios
dat <- data.frame(
  id=c("Basic", "Adjusted", "Moderate", "Heavy", "Other"),
  b=log(c(4.5, 3.5, 2.5, 1.5, 1)),
  se=c(0.2, 0.1, 0.2, 0.3, 0.2)
)
ESplot(dat, transform="exp")
```

fbsize

*Sample size for family-based linkage and association design***Description**

Sample size for family-based linkage and association design

**Usage**

```
fbsize(
  gamma,
  p,
  alpha = c(1e-04, 1e-08, 1e-08),
  beta = 0.2,
  debug = 0,
  error = 0
)
```

**Arguments**

|       |                                                       |
|-------|-------------------------------------------------------|
| gamma | genotype relative risk assuming multiplicative model. |
| p     | frequency of disease allele.                          |
| alpha | Type I error rates for ASP linkage, TDT and ASP-TDT.  |
| beta  | Type II error rate.                                   |
| debug | verbose output.                                       |
| error | 0=use the correct formula, 1=the original paper.      |

## Details

This function implements Risch and Merikangas (1996) statistics evaluating power for family-based linkage (affected sib pairs, ASP) and association design. They are potentially useful in the prospect of genome-wide association studies.

The function calls auxiliary functions `sn()` and `strlen`; `sn()` contains the necessary thresholds for power calculation while `strlen()` evaluates length of a string (generic).

## Value

The returned value is a list containing:

- gamma input gamma.
- p input p.
- n1 sample size for ASP.
- n2 sample size for TDT.
- n3 sample size for ASP-TDT.
- lambdao lambda o.
- lambdas lambda s.

## Note

extracted from `rm.c`.

## Author(s)

Jing Hua Zhao

## References

Risch N, Merikangas K (1996). "The Future of Genetic Studies of Complex Human Diseases." *Science*, **273**(5281), 1516-1517. doi:[10.1126/science.273.5281.1516](https://doi.org/10.1126/science.273.5281.1516). Risch N, Merikangas K (1996). "The Future of Genetic Studies of Complex Human Diseases." *Science*, **273**(5281), 1516-1517. doi:[10.1126/science.273.5281.1516](https://doi.org/10.1126/science.273.5281.1516). Risch N, Merikangas K (1997). "Reply to Scott et al." *Science*, **275**, 1329-1330. Scott WK, Pericak-Vance MA, Haines JL (1997). "Genetic analysis of complex diseases." *Science*, **275**(5304), 1327; author reply 1329-30.

## See Also

[pbsize](#)

## Examples

```
models <- matrix(c(
  4.0, 0.01,
  4.0, 0.10,
  4.0, 0.50,
  4.0, 0.80,
  2.0, 0.01,
  2.0, 0.10,
  2.0, 0.50,
  2.0, 0.80,
  1.5, 0.01,
```

```

1.5, 0.10,
1.5, 0.50,
1.5, 0.80), ncol=2, byrow=TRUE)
outfile <- "fbsize.txt"
cat("gamma","p","Y","N_asp","P_A","H1","N_tdt","H2","N_asp/tdt","L_o","L_s\n",
    file=outfile,sep="\t")
for(i in 1:12) {
  g <- models[i,1]
  p <- models[i,2]
  z <- fbsize(g,p)
  cat(z$gamma,z$p,z$y,z$n1,z$pA,z$h1,z$n2,z$h2,z$n3,z$lambdao,z$lambdas,file=outfile,
      append=TRUE,sep="\t")
  cat("\n",file=outfile,append=TRUE)
}
table1 <- read.table(outfile,header=TRUE,sep="\t")
nc <- c(4,7,9)
table1[,nc] <- ceiling(table1[,nc])
dc <- c(3,5,6,8,10,11)
table1[,dc] <- round(table1[,dc],2)
unlink(outfile)
# APOE-4, Scott WK, Pericak-Vance, MA & Haines JL
# Genetic analysis of complex diseases 1327
g <- 4.5
p <- 0.15
cat("\nAlzheimer's:\n\n")
fbsize(g,p)
# note to replicate the Table we need set alpha=9.961139e-05,4.910638e-08 and
# beta=0.2004542 or reset the quantiles in fbsize.R

```

---

FPRP

---

False-positive report probability

---

## Description

False-positive report probability

## Usage

```
FPRP(a, b, pi0, ORlist, logscale = FALSE)
```

## Arguments

|          |                                                                      |
|----------|----------------------------------------------------------------------|
| a        | parameter value at which the power is to be evaluated.               |
| b        | the variance for a, or the upper point of a 95%CI if logscale=FALSE. |
| pi0      | the prior probability that $H_0$ is true.                            |
| ORlist   | a vector of ORs that is most likely.                                 |
| logscale | FALSE=a,b in original scale, TRUE=a, b in log scale.                 |

## Details

The function calculates the false positive report probability (FPRP), the probability of no true association between a genetic variant and disease given a statistically significant finding, which depends not only on the observed P value but also on both the prior probability that the association is real and the statistical power of the test. An associate result is the false negative reported probability (FNRP). See example for the recommended steps.

The FPRP and FNRP are derived as follows. Let  $H_0$ =null hypothesis (no association),  $H_A$ =alternative hypothesis (association). Since classic frequentist theory considers they are fixed, one has to resort to Bayesian framework by introducing prior,  $\pi = P(H_0 = TRUE) = P(association)$ . Let  $T$ =test statistic, and  $P(T > z_\alpha | H_0 = TRUE) = P(rejecting H_0 | H_0 = TRUE) = \alpha$ ,  $P(T > z_\alpha | H_0 = FALSE) = P(rejecting H_0 | H_A = TRUE) = 1 - \beta$ . The joint probability of test and truth of hypothesis can be expressed by  $\alpha$ ,  $\beta$  and  $\pi$ .

Joint probability of significance of test and truth of hypothesis

| Truth of $H_A$ | significant                        | nonsignificant                     | Total     |
|----------------|------------------------------------|------------------------------------|-----------|
| TRUE           | $(1 - \beta)\pi$                   | $\beta\pi$                         | $\pi$     |
| FALSE          | $\alpha(1 - \pi)$                  | $(1 - \alpha)(1 - \pi)$            | $1 - \pi$ |
| Total          | $(1 - \beta)\pi + \alpha(1 - \pi)$ | $\beta\pi + (1 - \alpha)(1 - \pi)$ | 1         |

We have  $FPRP = P(H_0 = TRUE | T > z_\alpha) = \alpha(1 - \pi) / [\alpha(1 - \pi) + (1 - \beta)\pi] = \{1 + \pi / (1 - \pi) [(1 - \beta) / \alpha]\}^{-1}$  and similarly  $FNRP = \{1 + [(1 - \alpha) / \beta] [(1 - \pi) / \pi]\}^{-1}$ .

## Value

The returned value is a list with components, p p value corresponding to a,b. power the power corresponding to the vector of ORs. FPRP False-positive report probability. FNRP False-negative report probability.

## Author(s)

Jing Hua Zhao

## References

Wacholder S, Chanock S, Garcia-Closas M, El Ghormli L, Rothman N (2004). "Assessing the probability that a positive report is false: an approach for molecular epidemiology studies." *J Natl Cancer Inst*, **96**(6), 434-42. doi:10.1093/jnci/djh075.

## See Also

[BFDP](#)

## Examples

```
## Not run:
# Example by Laure El ghormli & Sholom Wacholder on 25-Feb-2004
# Step 1 - Pre-set an FPRP-level criterion for noteworthiness

T <- 0.2

# Step 2 - Enter values for the prior that there is an association
```

```

pi0 <- c(0.25,0.1,0.01,0.001,0.0001,0.00001)

# Step 3 - Enter values of odds ratios (OR) that are most likely, assuming that
#           there is a non-null association

ORlist <- c(1.2,1.5,2.0)

# Step 4 - Enter OR estimate and 95% confidence interval (CI) to obtain FPRP

OR <- 1.316
ORlo <- 1.08
ORhi <- 1.60

logOR <- log(OR)
selogOR <- abs(logOR-log(ORhi))/1.96
p <- ifelse(logOR>0,2*(1-pnorm(logOR/selogOR)),2*pnorm(logOR/selogOR))
p
q <- qnorm(1-p/2)
POWER <- ifelse(log(ORlist)>0,1-pnorm(q-log(ORlist)/selogOR),
                pnorm(-q-log(ORlist)/selogOR))
POWER
FPRPex <- t(p*(1-pi0)/(p*(1-pi0)+POWER\
row.names(FPRPex) <- pi0
colnames(FPRPex) <- ORlist
FPRPex
FPRPex>T

## now turn to FPRP
OR <- 1.316
ORhi <- 1.60
ORlist <- c(1.2,1.5,2.0)
pi0 <- c(0.25,0.1,0.01,0.001,0.0001,0.00001)
z <- FPRP(OR,ORhi,pi0,ORlist,logscale=FALSE)
z

## End(Not run)

```

g2a

*Conversion of a genotype identifier to alleles***Description**

Conversion of a genotype identifier to alleles

**Usage**

g2a(g)

**Arguments**

g                    a genotype identifier.

## Description

As is first reported, it is designed as an integrated package for genetic data analysis of both population and family data. Currently, it contains functions for sample size calculations of both population-based and family-based designs, probability of familial disease aggregation, kinship calculation, statistics in linkage analysis, and association analysis involving genetic markers including haplotype analysis with or without environmental covariates. Over years, the package has been developed in-between many projects hence also in line with the name (gap).

## Details

We have incorporated functions for a wide range of problems as shown below.

## ANALYSIS

|              |                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------|
| ACDE         | AE/ACE/ADE models using nuclear families                                                              |
| AE3          | AE model using nuclear family trios                                                                   |
| bt           | Bradley-Terry model for contingency table                                                             |
| ccsize       | Power and sample size for case-cohort design                                                          |
| cs           | Credibel set                                                                                          |
| fbsize       | Sample size for family-based linkage and association design                                           |
| gc.em        | Gene counting for haplotype analysis                                                                  |
| gcontrol     | genomic control                                                                                       |
| gcontrol2    | genomic control based on p values                                                                     |
| gcp          | Permutation tests using GENECOUNTING                                                                  |
| gc.lambda    | Estimation of the genomic control inflation statistic (lambda)                                        |
| genecounting | Gene counting for haplotype analysis                                                                  |
| gif          | Kinship coefficient and genetic index of familiarity                                                  |
| hap          | Haplotype reconstruction                                                                              |
| hap.em       | Gene counting for haplotype analysis                                                                  |
| hap.score    | Score statistics for association of traits with haplotypes                                            |
| htr          | Haplotype trend regression                                                                            |
| h2.jags      | Heritability estimation based on genomic relationship matrix using JAGS                               |
| hwe          | Hardy-Weinberg equilibrium test for a multiallelic marker                                             |
| hwe.cc       | A likelihood ratio test of population Hardy-Weinberg equilibrium                                      |
| hwe.hardy    | Hardy-Weinberg equilibrium test using MCMC                                                            |
| hwe.jags     | Hardy-Weinberg equilibrium test for a multiallelic marker using JAGS                                  |
| invnormal    | inverse Normal transformation                                                                         |
| kin.morgan   | kinship matrix for simple pedigree                                                                    |
| LD22         | LD statistics for two diallelic markers                                                               |
| LDkl         | LD statistics for two multiallelic markers                                                            |
| lambda1000   | A standardized estimate of the genomic inflation scaling to a study of 1,000 cases and 1,000 controls |
| log10p       | log10(p) for a standard normal deviate                                                                |
| log10pvalue  | log10(p) for a P value including its scientific format                                                |
| logp         | log(p) for a normal deviate                                                                           |
| masize       | Sample size calculation for mediation analysis                                                        |



|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| MCMCgrm         | Mixed modeling with genetic relationship matrices                                   |
| mia             | multiple imputation analysis for hap                                                |
| mr              | Mendelian randomization analysis                                                    |
| mtdt            | Transmission/disequilibrium test of a multiallelic marker                           |
| mtdt2           | Transmission/disequilibrium test of a multiallelic marker<br>by Bradley-Terry model |
| mvmeta          | Multivariate meta-analysis based on generalized least squares                       |
| pbsize          | Power for population-based association design                                       |
| pbsize2         | Power for case-control association design                                           |
| pfc             | Probability of familial clustering of disease                                       |
| pfc.sim         | Probability of familial clustering of disease                                       |
| pgc             | Preparing weight for GENECOUNTING                                                   |
| print.hap.score | Print a hap.score object                                                            |
| s2k             | Statistics for 2 by K table                                                         |
| sentinels       | Sentinel identification from GWAS summary statistics                                |
| tsc             | Power calculation for two-stage case-control design                                 |

## GRAPHICS

|                          |                                                              |
|--------------------------|--------------------------------------------------------------|
| asplot                   | Regional association plot                                    |
| ESplot                   | Effect-size plot                                             |
| circos.cis.vs.trans.plot | circos plot of cis/trans classification                      |
| circos.cnvplot           | circos plot of CNVs                                          |
| circos.mhtplot           | circos Manhattan plot with gene annotation                   |
| circos.mhtplot2          | Another circos Manhattan plot                                |
| cnvplot                  | genomewide plot of CNVs                                      |
| labelManhattan           | Annotate Manhattan or Miami Plot                             |
| makeRLEplot              | make relative log expression plot                            |
| METAL_forestplot         | forest plot as R/meta's forest for METAL outputs             |
| mhtplot                  | Manhattan plot                                               |
| mhtplot2                 | Manhattan plot with annotations                              |
| mhtplot.trunc            | truncated Manhattan plot                                     |
| miamipLOT                | Miami plot                                                   |
| miamipLOT2               | Miami plot                                                   |
| mr_forestplot            | Mendelian Randomization forest plot                          |
| pedtodot                 | Converting pedigree(s) to dot file(s)                        |
| pedtodot_verbatim        | Pedigree-drawing with graphviz                               |
| plot.hap.score           | Plot haplotype frequencies versus haplotype score statistics |
| qqfun                    | Quantile-comparison plots                                    |
| qqunif                   | Q-Q plot for uniformly distributed random variable           |
| qtl2dplot                | 2D QTL plot                                                  |
| qtl2dplotly              | 2D QTL plotly                                                |
| qtl3dplotly              | 3D QTL plotly                                                |

## UTILITIES

|           |                                                                |
|-----------|----------------------------------------------------------------|
| SNP       | Functions for single nucleotide polymorphisms (SNPs)           |
| BFDp      | Bayesian false-discovery probability                           |
| FPRP      | False-positive report probability                              |
| ab        | Test/Power calculation for mediating effect                    |
| b2r       | Obtain correlation coefficients and their variance-covariances |
| chow.test | Chow's test for heterogeneity in two regressions               |

|                             |                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| chr_pos_a1_a2               | Form SNPID from chromosome, position and alleles                                                           |
| ci2ms                       | Effect size and standard error from confidence interval                                                    |
| cis.vs.trans.classification | a cis/trans classifier                                                                                     |
| comp.score                  | score statistics for testing genetic linkage of quantitative trait                                         |
| GRM functions               | ReadGRM, ReadGRMBin, ReadGRMPLINK, ReadGRMPCA, WriteGRM, WriteGRMBin, WriteGRMSAS                          |
|                             | handle genomic relationship matrix involving other software                                                |
| get_b_se                    | Get b and se from AF, n, and z                                                                             |
| get_pve_se                  | Get pve and its standard error from n, z                                                                   |
| get_sdy                     | Get sd(y) from AF, n, b, se                                                                                |
| h2G                         | A utility function for heritability                                                                        |
| h2GE                        | A utility function for heritability involving gene-environment interaction                                 |
| h2l                         | A utility function for converting observed heritability to its counterpart under liability threshold model |
| h2_mzdz                     | Heritability estimation according to twin correlations                                                     |
| klem                        | Haplotype frequency estimation based on a genotype table of two multiallelic markers                       |
| makeped                     | A function to prepare pedigrees in post-MAKEPED format                                                     |
| metap                       | Meta-analysis of p values                                                                                  |
| metareg                     | Fixed and random effects model for meta-analysis                                                           |
| muvar                       | Means and variances under 1- and 2- locus (diallelic) QTL model                                            |
| qtlClassifier               | A QTL cis/trans classifier                                                                                 |
| qtlFinder                   | Distance-based signal identification                                                                       |
| read.ms.output              | A utility function to read ms output                                                                       |
| revStrand                   | Allele on the reverse strand                                                                               |
| runshinygap                 | Start shinygap                                                                                             |
| snptest_sample              | A utility to generate SNPTEST sample file                                                                  |
| whscore                     | Whittemore-Halpern scores for allele-sharing                                                               |
| weighted.median             | Weighted median with interpolation                                                                         |

## Usage

Vignettes on package usage:

- Genetic Analysis Package. `vignette("gap")`.
- Shiny for Genetic Analysis Package (gap) Designs. `vignette("shinygap")`.
- JSS paper: Genetic Analysis Package. `vignette("jss")`.

## Author(s)

Jing Hua Zhao in collaboration with other colleagues and with help from Kurt Hornik, Brian Ripley, Uwe Ligges and Achim Zeileis

maintained by Jing Hua Zhao [jinghuazhao@hotmail.com](mailto:jinghuazhao@hotmail.com)

## References

Zhao JH (2007). "gap: genetic analysis package." *Journal of Statistical Software*, **23**(8), 1-18.  
doi:10.18637/jss.v023.i08.

**See Also**

Useful links:

- <https://github.com/jinghuazhao/R>
- Report bugs at <https://github.com/jinghuazhao/R/issues>

---

gc.em

*Gene counting for haplotype analysis*


---

**Description**

Gene counting for haplotype analysis

**Usage**

```
gc.em(
  data,
  locus.label = NA,
  converge.eps = 1e-06,
  maxiter = 500,
  handle.miss = 0,
  miss.val = 0,
  control = gc.control()
)
```

**Arguments**

|              |                                                                                                                                                                                                                                                                            |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data         | Matrix of alleles, such that each locus has a pair of adjacent columns of alleles, and the order of columns corresponds to the order of loci on a chromosome. If there are K loci, then $\text{ncol}(\text{data}) = 2 \times K$ . Rows represent alleles for each subject. |
| locus.label  | Vector of labels for loci, of length K (see definition of data matrix).                                                                                                                                                                                                    |
| converge.eps | Convergence criterion, based on absolute change in log likelihood (lnlike).                                                                                                                                                                                                |
| maxiter      | Maximum number of iterations of EM.                                                                                                                                                                                                                                        |
| handle.miss  | a flag for handling missing genotype data, 0=no, 1=yes.                                                                                                                                                                                                                    |
| miss.val     | missing value.                                                                                                                                                                                                                                                             |
| control      | a function, see <a href="#">genecounting</a> .                                                                                                                                                                                                                             |

**Details**

Gene counting for haplotype analysis with missing data, adapted for hap.score

**Value**

List with components:

- converge Indicator of convergence of the EM algorithm (1=converged, 0 = failed).
- niter Number of iterations completed in the EM algorithm.
- locus.info A list with a component for each locus. Each component is also a list, and the items of a locus- specific list are the locus name and a vector for the unique alleles for the locus.

- locus.label Vector of labels for loci, of length K (see definition of input values).
- haplotype Matrix of unique haplotypes. Each row represents a unique haplotype, and the number of columns is the number of loci.
- hap.prob Vector of mle's of haplotype probabilities. The ith element of hap.prob corresponds to the ith row of haplotype.
- hap.prob.noLD Similar to hap.prob, but assuming no linkage disequilibrium.
- Inlike Value of Inlike at last EM iteration (maximum Inlike if converged).
- lr Likelihood ratio statistic to test no linkage disequilibrium among all loci.
- indx.subj Vector for index of subjects, after expanding to all possible pairs of haplotypes for each person. If indx=i, then i is the ith row of input matrix data. If the ith subject has n possible pairs of haplotypes that correspond to their marker phenotype, then i is repeated n times.
- nreps Vector for the count of haplotype pairs that map to each subject's marker genotypes.
- hap1code Vector of codes for each subject's first haplotype. The values in hap1code are the row numbers of the unique haplotypes in the returned matrix haplotype.
- hap2code Similar to hap1code, but for each subject's second haplotype.
- post Vector of posterior probabilities of pairs of haplotypes for a person, given thier marker phenotypes.
- httable A table which can be used in haplotype trend regression.

### Note

Adapted from GENECOUNTING.

### Author(s)

Jing Hua Zhao

### References

Zhao JH, Lissarrague S, Essioux L, Sham PC (2002). "GENECOUNTING: haplotype analysis with missing genotypes." *Bioinformatics*, **18**(12), 1694-5. doi:10.1093/bioinformatics/18.12.1694.

Zhao JH, Sham PC (2003). "Generic number systems and haplotype analysis." *Comput Methods Programs Biomed*, **70**(1), 1-9. doi:10.1016/s01692607(01)001936.

### See Also

[genecounting](#), [LDkl](#)

### Examples

```
## Not run:
data(hla)
gc.em(hla[,3:8], locus.label=c("DQR", "DQA", "DQB"), control=gc.control(assignment="t"))

## End(Not run)
```

gc.lambda

*Estimation of the genomic control inflation statistic (lambda)***Description**

Estimation of the genomic control inflation statistic (lambda)

**Usage**

```
gc.lambda(x, logscale = FALSE, z = FALSE)
```

**Arguments**

**x** A real vector (p or z).  
**logscale** A logical variable such that x as  $-\log_{10}(p)$ .  
**z** A flag to indicate x as a vector of z values.

**Value**

Estimate of inflation factor.

**Examples**

```
set.seed(12345)
p <- runif(100)
gc.lambda(p)
lp <- -log10(p)
gc.lambda(lp, logscale=TRUE)
z <- qnorm(p/2)
gc.lambda(z, z=TRUE)
```

gcontrol

*genomic control***Description**

genomic control

**Usage**

```
gcontrol(
  data,
  zeta = 1000,
  kappa = 4,
  tau2 = 1,
  epsilon = 0.01,
  ngib = 500,
  burn = 50,
  idum = 2348
)
```

### Arguments

|         |                                                                  |
|---------|------------------------------------------------------------------|
| data    | the data matrix.                                                 |
| zeta    | program constant with default value 1000.                        |
| kappa   | multiplier in prior for mean with default value 4.               |
| tau2    | multiplier in prior for variance with default value 1.           |
| epsilon | prior probability of marker association with default value 0.01. |
| ngib    | number of Gibbs steps, with default value 500.                   |
| burn    | number of burn-ins with default value 50.                        |
| idum    | seed for pseudorandom number sequence.                           |

### Details

The Bayesian genomic control statistics with the following parameters,

|           |                                                                                                                                                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| n         | number of loci under consideration                                                                                                                            |
| lambdahat | median(of the n trend statistics)/0.46<br>Prior for noncentrality parameter $A_i$ is<br>Normal( $\sqrt{\text{lambdahat}}\kappa$ , $\text{lambdahat}*\tau^2$ ) |
| kappa     | multiplier in prior above, set at $1.6 * \sqrt{\log(n)}$                                                                                                      |
| tau2      | multiplier in prior above                                                                                                                                     |
| epsilon   | prior probability a marker is associated, set at $10/n$                                                                                                       |
| ngib      | number of cycles for the Gibbs sampler after burn in                                                                                                          |
| burn      | number of cycles for the Gibbs sampler to burn in                                                                                                             |

Armitage's trend test along with the posterior probability that each marker is associated with the disorder is given. The latter is not a p-value but any value greater than 0.5 (pout) suggests association.

### Value

The returned value is a list containing:

- deltot the probability of being an outlier.
- x2 the  $\chi^2$  statistic.
- A the A vector.

### Note

Adapted from gcontrol by Bobby Jones and Kathryn Roeder, use -Dexecutable for standalone program, function getnum in the original code needs \

### Author(s)

Bobby Jones, Jing Hua Zhao

### References

Devlin B, Roeder K (1999). "Genomic control for association studies." *Biometrics*, **55**(4), 997-1004. doi:10.1111/j.0006341x.1999.00997.x.

**Examples**

```
## Not run:
test<-c(1,2,3,4,5,6, 1,2,1,23,1,2, 100,1,2,12,1,1,
        1,2,3,4,5,61, 1,2,11,23,1,2, 10,11,2,12,1,11)
test<-matrix(test,nrow=6,byrow=T)
gcontrol(test)

## End(Not run)
```

gcontrol2

*genomic control based on p values***Description**

genomic control based on p values

**Usage**

```
gcontrol2(p, col = palette()[4], lcol = palette()[2], ...)
```

**Arguments**

|      |                                               |
|------|-----------------------------------------------|
| p    | a vector of observed p values.                |
| col  | colour for points in the Q-Q plot.            |
| lcol | colour for the diagonal line in the Q-Q plot. |
| ...  | other options for plot.                       |

**Details**

The function obtains 1-df  $\chi^2$  statistics (observed) according to a vector of p values, and the inflation factor (lambda) according to medians of the observed and expected statistics. The latter is based on the empirical distribution function (EDF) of 1-df  $\chi^2$  statistics.

It would be appropriate for genetic association analysis as of 1-df Armitage trend test for case-control data; for 1-df additive model with continuous outcome one has to consider the compatibility with p values based on z-/t- statistics.

**Value**

A list containing:

- x the expected  $\chi^2$  statistics.
- y the observed  $\chi^2$  statistics.
- lambda the inflation factor.

**Author(s)**

Jing Hua Zhao

**References**

Devlin B, Roeder K (1999) Genomic control for association studies. *Biometrics* 55:997-1004

**Examples**

```
## Not run:
x2 <- rchisq(100,1,.1)
p <- pchisq(x2,1,lower.tail=FALSE)
r <- gcontrol2(p)
print(r$lambda)

## End(Not run)
```

gcp

*Permutation tests using GENECOUNTING***Description**

Permutation tests using GENECOUNTING

**Usage**

```
gcp(
  y,
  cc,
  g,
  handle.miss = 1,
  miss.val = 0,
  n.sim = 0,
  locus.label = NULL,
  quietly = FALSE
)
```

**Arguments**

|             |                                                          |
|-------------|----------------------------------------------------------|
| y           | A column of 0/1 indicating cases and controls.           |
| cc          | analysis indicator, 0 = marker-marker, 1 = case-control. |
| g           | the multilocus genotype data.                            |
| handle.miss | a flag with value 1 indicating missing data are allowed. |
| miss.val    | missing value.                                           |
| n.sim       | the number of permutations.                              |
| locus.label | label of each locus.                                     |
| quietly     | a flag if TRUE will suppress the screen output.          |

**Details**

This function is a R port of the GENECOUNTING/PERMUTE program which generates EHPLUS-type statistics including z-tests for individual haplotypes



**Value**

The returned value is a list containing (p.sim and ph when n.sim > 0):

- x2obs the observed chi-squared statistic.
- pobs the associated p value.
- zobs the observed z value for individual haplotypes.
- p.sim simulated p value for the global chi-squared statistic.
- ph simulated p values for individual haplotypes.

**Note**

Built on gcp.c.

**Author(s)**

Jing Hua Zhao

**References**

Zhao JH, Curtis D, Sham PC (2000). "Model-free analysis and permutation tests for allelic associations." *Hum Hered*, **50**(2), 133-9. doi:10.1159/000022901.

Zhao JH (2004). "2LD, GENECOUNTING and HAP: computer programs for linkage disequilibrium analysis." *Bioinformatics*, **20**(8), 1325-6. doi:10.1093/bioinformatics/bth071.

Zhao JH, Qian WD (2003) Association analysis of unrelated individuals using polymorphic genetic markers – methods, implementation and application, Royal Statistical Society, Hassallt-Diepenbeek, Belgium.

**See Also**

[genecounting](#)

**Examples**

```
## Not run:
data(fsnps)
y<-fsnps$y
cc<-1
g<-fsnps[,3:10]

gcp(y,cc,g,miss.val="Z",n.sim=5)
hap.score(y,g,method="hap",miss.val="Z")

## End(Not run)
```

genecounting

*Gene counting for haplotype analysis***Description**

Gene counting for haplotype analysis

**Usage**

```
genecounting(data, weight = NULL, loci = NULL, control = gc.control())
```

**Arguments**

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data    | genotype table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| weight  | a column of frequency weights.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| loci    | an array containing number of alleles at each locus.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| control | is a function with the following arguments: <ul style="list-style-type: none"> <li>• xdata. a flag indicating if the data involves X chromosome, if so, the first column of data indicates sex of each subject: 1=male, 2=female. The marker data are no different from the autosomal version for females, but for males, two copies of the single allele present at a given locus.</li> <li>• convll. set convergence criteria according to log-likelihood, if its value set to 1</li> <li>• handle.miss. to handle missing data, if its value set to 1</li> <li>• eps. the actual convergence criteria, with default value 1e-5</li> <li>• tol. tolerance for genotype probabilities with default value 1e-8</li> <li>• maxit. maximum number of iterations, with default value 50</li> <li>• pl. criteria for trimming haplotypes according to posterior probabilities</li> <li>• assignment. filename containing haplotype assignment</li> <li>• verbose. If TRUE, yields print out from the C routine</li> </ul> |

**Details**

Gene counting for haplotype analysis with missing data.

**Value**

The returned value is a list containing:

- h haplotype frequency estimates under linkage disequilibrium (LD).
- h0 haplotype frequency estimates under linkage equilibrium (no LD).
- prob genotype probability estimates.
- l0 log-likelihood under linkage equilibrium.
- l1 log-likelihood under linkage disequilibrium.
- hapid unique haplotype identifier (defunct, see gc.em).
- npusr number of parameters according user-given alleles.
- npdat number of parameters according to observed.

- htrtable design matrix for haplotype trend regression (defunct, see `gc.em`).
- iter number of iterations used in gene counting.
- converge a flag indicating convergence status of gene counting.
- di0 haplotype diversity under no LD, defined as  $1 - \sum(h_0^2)$ .
- di1 haplotype diversity under LD, defined as  $1 - \sum(h^2)$ .
- resid residuals in terms of frequency weights = o - e.

### Note

adapted from GENECOUNTING.

### Author(s)

Jing Hua Zhao

### References

Zhao JH, Lissarrague S, Essioux L, Sham PC (2002). "GENECOUNTING: haplotype analysis with missing genotypes." *Bioinformatics*, **18**(12), 1694-5. doi:10.1093/bioinformatics/18.12.1694.

Zhao JH, Sham PC (2003). "Generic number systems and haplotype analysis." *Comput Methods Programs Biomed*, **70**(1), 1-9. doi:10.1016/s01692607(01)001936.

Zhao JH (2004). "2LD. GENECOUNTING and HAP: computer programs for linkage disequilibrium analysis." *Bioinformatics*, **20**(8), 1325-6. doi:10.1093/bioinformatics/bth071.

### See Also

[gc.em](#), [LDk1](#)

### Examples

```
## Not run:
require(gap.datasets)
# HLA data
data(hla)
hla.gc <- genecounting(hla[,3:8])
summary(hla.gc)
hla.gc$l0
hla.gc$l1

# ALDH2 data
data(aldh2)
control <- gc.control(handle.miss=1,assignment="ALDH2.out")
aldh2.gc <- genecounting(aldh2[,3:6],control=control)
summary(aldh2.gc)
aldh2.gc$l0
aldh2.gc$l1

# Chromosome X data
# assuming allelic data have been extracted in columns 3-13
# and column 3 is sex
filespec <- system.file("tests/genecounting/mao.dat")
mao2 <- read.table(filespec)
dat <- mao2[,3:13]
```

```

loci <- c(12,9,6,5,3)
contr <- gc.control(xdata=TRUE,handle.miss=1)
mao.gc <- genecounting(dat,loci=loci,control=contr)
mao.gc$npusr
mao.gc$npdat

## End(Not run)

```

---

|             |                          |
|-------------|--------------------------|
| geno.recode | <i>Genotype recoding</i> |
|-------------|--------------------------|

---

### Description

Genotype recoding

### Usage

```
geno.recode(geno, miss.val = 0)
```

### Arguments

|          |                |
|----------|----------------|
| geno     | genotype.      |
| miss.val | missing value. |

---

|          |                                       |
|----------|---------------------------------------|
| get_b_se | <i>Get b and se from AF, n, and z</i> |
|----------|---------------------------------------|

---

### Description

The function obtains effect size and its standard error.

### Usage

```
get_b_se(f, n, z)
```

### Arguments

|   |                   |
|---|-------------------|
| f | Allele frequency. |
| n | Sample size.      |
| z | z-statistics.     |

### Value

b and se.

**Examples**

```
## Not run:
library(dplyr)
# eQTLGen
cis_pQTL <- merge(read.delim('eQTLGen.lz') %>%
  filter(GeneSymbol=="LTBR"),read.delim("eQTLGen.AF"),by="SNP") %>%
  mutate(data.frame(get_b_se(AlleleB_all,NrSamples,Zscore)))
head(cis_pQTL,1)
  SNP      Pvalue SNPChr  SNPPos AssessedAllele OtherAllele Zscore
rs1003563 2.308e-06    12 6424577              A          G 4.7245
      Gene GeneSymbol GeneChr GenePos NrCohorts NrSamples      FDR
ENSG00000111321      LTBR    12 6492472      34    23991 0.006278872
BonferroniP hg19_chr hg19_pos AlleleA AlleleB allA_total allAB_total
      1      12 6424577      A      G      2574      8483
allB_total AlleleB_all      b      se
      7859    0.6396966 0.04490488 0.009504684

## End(Not run)
```

get\_pve\_se

*Get pve and its standard error from n, z***Description**

Get pve and its standard error from n, z

**Usage**

```
get_pve_se(n, z, correction = TRUE)
```

**Arguments**

|            |                                                          |
|------------|----------------------------------------------------------|
| n          | Sample size.                                             |
| z          | z-statistic, i.e., b/se when they are available instead. |
| correction | if TRUE an correction based on t-statistic is applied.   |

**Details**

This function obtains proportion of explained variance of a continuous outcome.

**Value**

pve and its se.

---

|         |                                    |
|---------|------------------------------------|
| get_sdy | <i>Get sd(y) from AF, n, b, se</i> |
|---------|------------------------------------|

---

## Description

Get sd(y) from AF, n, b, se

## Usage

```
get_sdy(f, n, b, se, method = "mean", ...)
```

## Arguments

|        |                                          |
|--------|------------------------------------------|
| f      | Allele frequency.                        |
| n      | Sample size.                             |
| b      | effect size.                             |
| se     | standard error.                          |
| method | method of averaging: "mean" or "median". |
| ...    | argument(s) passed to method             |

## Details

This function obtains standard error of a continuous outcome.

## Value

sd(y).

## Examples

```
## Not run:
set.seed(1)
X1 <- matrix(rbinom(1200,1,0.4),ncol=2)
X2 <- matrix(rbinom(1000,1,0.6),ncol=2)
colnames(X1) <- colnames(X2) <- c("f1","f2")
Y1 <- rnorm(600,apply(X1,1,sum),2)
Y2 <- rnorm(500,2*apply(X2,1,sum),5)
summary(lm1 <- lm(Y1~f1+f2,data=as.data.frame(X1)))
summary(lm2 <- lm(Y2~f1+f2,data=as.data.frame(X2)))
b1 <- coef(lm1)
b2 <- coef(lm2)
v1 <- vcov(lm1)
v2 <- vcov(lm2)
require(coloc)
## Bayesian approach, esp. when only p values are available
abf <- coloc.abf(list(beta=b1, varbeta=diag(v1), N=nrow(X1), sdY=sd(Y1), type="quant"),
                  list(beta=b2, varbeta=diag(v2), N=nrow(X2), sdY=sd(Y2), type="quant"))
abf
# sdY
cat("sd(Y)=",sd(Y1),"==> Estimates:",sqrt(diag(var(X1)*b1[-1]^2+var(X1)*v1[-1,-1]*nrow(X1))),"\n")
for(k in 1:2)
{
```

```

    k1 <- k + 1
    cat("Based on b",k," sd(Y1) = ",sqrt(var(X1[,k])*(b1[k1]^2+nrow(X1)*v1[k1,k1])), "\n", sep="")
  }
  cat("sd(Y)=",sd(Y2),"==> Estimates:",sqrt(diag(var(X2)*b2[-1]^2+var(X2)*v2[-1,-1]*nrow(X2))), "\n")
  for(k in 1:2)
  {
    k1 <- k + 1
    cat("Based on b",k," sd(Y2) = ",sqrt(var(X2[,k])*(b2[k1]^2+nrow(X2)*v2[k1,k1])), "\n", sep="")
  }
  get_sdy(0.6396966,23991,0.04490488,0.009504684)

## End(Not run)

```

gif

*Kinship coefficient and genetic index of familiarity***Description**

Kinship coefficient and genetic index of familiarity

**Usage**

```
gif(data, gifset)
```

**Arguments**

|        |                                 |
|--------|---------------------------------|
| data   | the trio data of a pedigree.    |
| gifset | a subgroup of pedigree members. |

**Details**

The genetic index of familiarity is defined as the mean kinship between all pairs of individuals in a set multiplied by 100,000. Formally, it is defined in (Gholami and Thomas 1994) as

$$100,000 \times \frac{2}{n(n-1)} \sum_{i=1}^{n-1} \sum_{j=i+1}^n k_{ij}$$

where  $n$  is the number of individuals in the set and  $k_{ij}$  is the kinship coefficient between individuals  $i$  and  $j$ .

The scaling is purely for convenience of presentation.

**Value**

The returned value is a list containing:

- gifval the genetic index of familiarity.

**Note**

Adapted from gif.c, testable with -Dexecutable as standalone program, which can be use for any pair of individuals

**Author(s)**

Alun Thomas, Jing Hua Zhao

**References**

Gholami K, Thomas A (1994). "A linear time algorithm for calculation of multiple pairwise kinship coefficients and the genetic index of familiarity." *Comput Biomed Res*, **27**(5), 342-50. doi:[10.1006/cbmr.1994.1026](https://doi.org/10.1006/cbmr.1994.1026).

**See Also**

[pfc](#)

**Examples**

```
## Not run:
test<-c(
  5,      0,      0,
  1,      0,      0,
  9,      5,      1,
  6,      0,      0,
  10,     9,      6,
  15,     9,      6,
  21,    10,     15,
  3,      0,      0,
  18,     3,     15,
  23,    21,     18,
  2,      0,      0,
  4,      0,      0,
  7,      0,      0,
  8,      4,      7,
  11,     5,      8,
  12,     9,      6,
  13,     9,      6,
  14,     5,      8,
  16,    14,      6,
  17,    10,      2,
  19,     9,     11,
  20,    10,     13,
  22,    21,     20)
test<-matrix(test,ncol=3,byrow=TRUE)
gif(test,gifset=c(20,21,22))

# all individuals
gif(test,gifset=1:23)

## End(Not run)
```



**Description**

This function build 2-d grids

**Usage**

```
grid2d(
  chrlen,
  plot = TRUE,
  cex.labels = 0.6,
  xlab = "QTL position",
  ylab = "Gene position"
)
```

**Arguments**

|            |                                                   |
|------------|---------------------------------------------------|
| chrlen     | Lengths of chromosomes; e.g., hg18, hg19 or hg38. |
| plot       | A flag for plot.                                  |
| cex.labels | A scaling factor for labels.                      |
| xlab       | X-axis title.                                     |
| ylab       | Y-axis title.                                     |

**Value**

A list with two variables:

- n Number of chromosomes.
- CM Cumulative lengths starting from 0.

---

h2.jags

---

*Heritability estimation based on genomic relationship matrix using JAGS*


---

**Description**

Heritability estimation based on genomic relationship matrix using JAGS

**Usage**

```
h2.jags(
  y,
  x,
  G,
  eps = 1e-04,
  sigma.p = 0,
  sigma.r = 1,
  parms = c("b", "p", "r", "h2"),
  ...
)
```

## Arguments

|         |                                                              |
|---------|--------------------------------------------------------------|
| y       | outcome vector.                                              |
| x       | covariate matrix.                                            |
| G       | genomic relationship matrix.                                 |
| eps     | a positive diagonal perturbation to G.                       |
| sigma.p | initial parameter values.                                    |
| sigma.r | initial parameter values.                                    |
| parms   | monitored parameters.                                        |
| ...     | parameters passed to jags, e.g., n.chains, n.burnin, n.iter. |

## Details

This function performs Bayesian heritability estimation using genomic relationship matrix.

## Value

The returned value is a fitted model from jags().

## Author(s)

Jing Hua Zhao keywords htest

## References

Zhao JH, Luan JA, Congdon P (2018). “Bayesian Linear Mixed Models with Polygenic Effects.” *Journal of Statistical Software*, **85**(6), 1 - 27. doi:[10.18637/jss.v085.i06](https://doi.org/10.18637/jss.v085.i06).

## Examples

```
## Not run:
require(gap.datasets)
set.seed(1234567)
meyer <- within(meyer,{
  y[is.na(y)] <- rnorm(length(y[is.na(y)]),mean(y,na.rm=TRUE),sd(y,na.rm=TRUE))
  g1 <- ifelse(generation==1,1,0)
  g2 <- ifelse(generation==2,1,0)
  id <- animal
  animal <- ifelse(!is.na(animal),animal,0)
  dam <- ifelse(!is.na(dam),dam,0)
  sire <- ifelse(!is.na(sire),sire,0)
})
G <- kin.morgan(meyer)$kin.matrix*2
library(regress)
r <- regress(y~-1+g1+g2,~G,data=meyer)
r
with(r,h2G(sigma,sigma.cov))
eps <- 0.001
y <- with(meyer,y)
x <- with(meyer,cbind(g1,g2))
ex <- h2.jags(y,x,G,sigma.p=0.03,sigma.r=0.014)
print(ex)
require(coda)
ex.mcmc <- as.mcmc(ex)
```

```

traceplot(ex.mcmc)
densplot(ex.mcmc)

## End(Not run)

```

h2G

*Heritability and its variance***Description**

Heritability and its variance

**Usage**

```
h2G(V, VCOV, verbose = TRUE)
```

**Arguments**

|         |                             |
|---------|-----------------------------|
| V       | Variance estimates.         |
| VCOV    | Variance-covariance matrix. |
| verbose | Detailed output.            |

**Value**

A list of phenotypic variance/heritability estimates and their variances.

h2GE

*Heritability and its variance when there is an environment component***Description**

Heritability and its variance when there is an environment component

**Usage**

```
h2GE(V, VCOV, verbose = TRUE)
```

**Arguments**

|         |                             |
|---------|-----------------------------|
| V       | Variance estimates.         |
| VCOV    | Variance-covariance matrix. |
| verbose | Detailed output.            |

**Value**

A list of phenotypic variance/heritability/GxE interaction estimates and their variances.

---

|     |                                                         |
|-----|---------------------------------------------------------|
| h2l | <i>Heritability under the liability threshold model</i> |
|-----|---------------------------------------------------------|

---

### Description

Heritability under the liability threshold model

### Usage

```
h2l(K = 0.05, P = 0.5, h2, se, verbose = TRUE)
```

### Arguments

|         |                        |
|---------|------------------------|
| K       | Disease prevalence.    |
| P       | Phenotypic variance.   |
| h2      | Heritability estimate. |
| se      | Standard error.        |
| verbose | Detailed output.       |

### Value

A list of the input heritability estimate/standard error and their counterpart under liability threshold model, the normal deviate..

---

|         |                                                                 |
|---------|-----------------------------------------------------------------|
| h2_mzdz | <i>ACE heritability estimation from MZ/DZ twin correlations</i> |
|---------|-----------------------------------------------------------------|

---

### Description

Estimates additive genetic (A), shared environmental (C), and unique environmental (E) variance components using Falconer's equations from monozygotic (MZ) and dizygotic (DZ) twin data.

### Usage

```
h2_mzdz(
  mzDat = NULL,
  dzDat = NULL,
  rmz = NULL,
  rdz = NULL,
  nmz = NULL,
  ndz = NULL,
  selV = NULL,
  use = "complete.obs",
  ci = TRUE,
  bounds = FALSE,
  digits = 3
)
```

## Arguments

|        |                                                                                    |
|--------|------------------------------------------------------------------------------------|
| mzDat  | Optional data.frame containing MZ twin data.                                       |
| dzDat  | Optional data.frame containing DZ twin data.                                       |
| rmz    | Optional numeric scalar. Correlation for MZ twins.                                 |
| rdz    | Optional numeric scalar. Correlation for DZ twins.                                 |
| nmz    | Optional integer. Sample size for MZ twins.                                        |
| ndz    | Optional integer. Sample size for DZ twins.                                        |
| selV   | Character vector of length 2 specifying twin variables.                            |
| use    | Correlation missing-data method passed to stats::cor(). Default is "complete.obs". |
| ci     | Logical; if TRUE, returns approximate 95% confidence intervals.                    |
| bounds | Logical; if TRUE, constrain h2/c2/e2 estimates to [0, 1].                          |
| digits | Integer controlling printed rounding.                                              |

## Details

Falconer's estimators are:

$$h^2 = 2(r_{MZ} - r_{DZ}), c^2 = 2r_{DZ} - r_{MZ}, \text{ and } e^2 = 1 - r_{MZ}.$$

Approximate variances are computed using

$$Var(r) \approx \frac{(1 - r^2)^2}{n - 1}$$

Given MZ/DZ data or their correlations and sample sizes, it obtains heritability and variance estimates under an ACE model as in Gidziela et al. (2023) and Keeping (1995).

## Value

A data.frame containing:

- h2: additive genetic variance estimate.
- c2: shared environmental variance estimate.
- e2: unique environmental variance estimate.
- vh2: estimated variance of h2.
- vc2: estimated variance of c2.
- ve2: estimated variance of e2.
- se\_h2: standard error of h2.
- se\_c2: standard error of c2.
- se\_e2: standard error of e2.
- lc1\_h2: lower 95% confidence limit for h2.
- uc1\_h2: upper 95% confidence limit for h2.
- lc1\_c2: lower 95% confidence limit for c2.
- uc1\_c2: upper 95% confidence limit for c2.
- lc1\_e2: lower 95% confidence limit for e2.
- uc1\_e2: upper 95% confidence limit for e2.

## References

Elks CE, den Hoed M, Zhao JH, Sharp SJ, Wareham NJ, Loos RJ, Ong KK (2012). “Variability in the heritability of body mass index: a systematic review and meta-regression.” *Front Endocrinol (Lausanne)*, **3**, 29. doi:10.3389/fendo.2012.00029.

Gidziela A, Ahmadzadeh YI, Michelini G, Allegrini AG, Agnew-Blais J, Lau LY, Duret M, Procopio F, Daly E, Ronald A, Rimfeld K, Malanchini M (2023). “A meta-analysis of genetic effects associated with neurodevelopmental disorders and co-occurring conditions.” *Nature Human Behaviour*, **7**, 642–656. ISSN 2397-3374. doi:10.1038/s4156202301530y.

Keeping ES (1995). *Introduction to statistical inference*, Dover books on mathematics, Dover edition. Dover Publications, New York. ISBN 9780486685021.

## Examples

```
## Not run:
h2_mzdz(rmz = 0.72,
        rdz = 0.33,
        nmz = 384,
        ndz = 243)
library(mvtnorm)
set.seed(12345)
selVars <- c("bmi1", "bmi2")
mzm <- as.data.frame(
  mvtnorm::rmvnorm(
    195,
    c(22.75, 22.75),
    matrix(2.66^2 * c(1, 0.67, 0.67, 1), 2)
  )
)
dzm <- as.data.frame(
  mvtnorm::rmvnorm(
    130,
    c(23.44, 23.44),
    matrix(2.75^2 * c(1, 0.32, 0.32, 1), 2)
  )
)
names(mzm) <- selVars
names(dzm) <- selVars
res <- ACE_CI(
  mzData = mzm,
  dzData = dzm,
  selV = selVars,
  n.boot = 50,
  seed = 123,
  verbose = FALSE
)
print(res)

## End(Not run)
```

## Description

Haplotype reconstruction

## Usage

```
hap(
  id,
  data,
  nloci,
  loci = rep(2, nloci),
  names = paste("loci", 1:nloci, sep = ""),
  control = hap.control()
)
```

## Arguments

|                      |                                           |
|----------------------|-------------------------------------------|
| <code>id</code>      | a column of subject id.                   |
| <code>data</code>    | genotype table.                           |
| <code>nloci</code>   | number of loci.                           |
| <code>loci</code>    | number of alleles at all loci.            |
| <code>names</code>   | locus names.                              |
| <code>control</code> | is a call to <code>hap.control()</code> . |

## Details

Haplotype reconstruction using sorting and trimming algorithms.

The package can handle much larger number of multiallelic loci. For large sample size with relatively small number of multiallelic loci, genecounting should be used.

## Value

The returned value is a list containing:

- 11 log-likelihood assuming linkage disequilibrium.
- converge convergence status, 0=failed, 1=succeeded.
- niter number of iterations.

## Note

adapted from hap.

## References

Clayton DG (2001) SNPHAP. <https://github.com/chr1swallace/snphap>.

Zhao JH and W Qian (2003) Association analysis of unrelated individuals using polymorphic genetic markers. RSS 2003, Hasselt, Belgium

Zhao JH (2004). "2LD. GENECOUNTING and HAP: computer programs for linkage disequilibrium analysis." *Bioinformatics*, **20**(8), 1325-6. doi:10.1093/bioinformatics/bth071.

**See Also**[genecounting](#)**Examples**

```
## Not run:
require(gap.datasets)
# 4 SNP example, to generate hap.out and assign.out alone
data(fsnps)
hap(id=fsnps[,1],data=fsnps[,3:10],nloci=4)
dir()

# to generate results of imputations
control <- hap.control(ss=1,mi=5,hapfile="h",assignfile="a")
hap(id=fsnps[,1],data=fsnps[,3:10],nloci=4,control=control)
dir()

## End(Not run)
```

hap.control

*Control for haplotype reconstruction***Description**

Control for haplotype reconstruction

**Usage**

```
hap.control(
  mb = 0,
  pr = 0,
  po = 0.001,
  to = 0.001,
  th = 1,
  maxit = 100,
  n = 0,
  ss = 0,
  rs = 0,
  rp = 0,
  ro = 0,
  rv = 0,
  sd = 0,
  mm = 0,
  mi = 0,
  mc = 50,
  ds = 0.1,
  de = 0,
  q = 0,
  hapfile = "hap.out",
  assignfile = "assign.out"
)
```



**Arguments**

|            |                                                       |
|------------|-------------------------------------------------------|
| mb         | Maximum dynamic storage to be allocated, in Mb.       |
| pr         | Prior (ie population) probability threshold.          |
| po         | Posterior probability threshold.                      |
| to         | Log-likelihood convergence tolerance.                 |
| th         | Posterior probability threshold for output.           |
| maxit      | Maximum EM iteration.                                 |
| n          | Force numeric allele coding (1/2) on output (off).    |
| ss         | Tab-delimited spreadsheet file output (off).          |
| rs         | Random starting points for each EM iteration (off).   |
| rp         | Restart from random prior probabilities.              |
| ro         | Loci added in random order (off).                     |
| rv         | Loci added in reverse order (off).                    |
| sd         | Set seed for random number generator (use date+time). |
| mm         | Repeat final maximization multiple times.             |
| mi         | Create multiple imputed datasets. If set >0.          |
| mc         | Number of MCMC steps between samples.                 |
| ds         | Starting value of Dirichlet prior parameter.          |
| de         | Finishing value of Dirichlet prior parameter.         |
| q          | Quiet operation (off).                                |
| hapfile    | a file for haplotype frequencies.                     |
| assignfile | a file for haplotype assignment.                      |

**Value**

A list containing the parameter specifications to the function.

---

hap.em

*Gene counting for haplotype analysis*


---

**Description**

Gene counting for haplotype analysis

**Usage**

```
hap.em(
  id,
  data,
  locus.label = NA,
  converge.eps = 1e-06,
  maxiter = 500,
  miss.val = 0
)
```

**Arguments**

|                           |                                                                                                                                                                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>           | a vector of individual IDs.                                                                                                                                                                                                                                             |
| <code>data</code>         | Matrix of alleles, such that each locus has a pair of adjacent columns of alleles, and the order of columns corresponds to the order of loci on a chromosome. If there are $K$ loci, then $\text{ncol}(\text{data}) = 2 * K$ . Rows represent alleles for each subject. |
| <code>locus.label</code>  | Vector of labels for loci, of length $K$ (see definition of data matrix).                                                                                                                                                                                               |
| <code>converge.eps</code> | Convergence criterion, based on absolute change in log likelihood (lnlike).                                                                                                                                                                                             |
| <code>maxiter</code>      | Maximum number of iterations of EM.                                                                                                                                                                                                                                     |
| <code>miss.val</code>     | missing value.                                                                                                                                                                                                                                                          |

**Details**

Gene counting for haplotype analysis with missing data, adapted for `hap.score`.

**Value**

List with components:

- `converge` Indicator of convergence of the EM algorithm (1=converged, 0 = failed).
- `niter` Number of iterations completed in the EM algorithm.
- `locus.info` A list with a component for each locus. Each component is also a list, and the items of a locus- specific list are the locus name and a vector for the unique alleles for the locus.
- `locus.label` Vector of labels for loci, of length  $K$  (see definition of input values).
- `haplotype` Matrix of unique haplotypes. Each row represents a unique haplotype, and the number of columns is the number of loci.
- `hap.prob` Vector of mle's of haplotype probabilities. The  $i$ th element of `hap.prob` corresponds to the  $i$ th row of haplotype.
- `lnlike` Value of lnlike at last EM iteration (maximum lnlike if converged).
- `indx.subj` Vector for index of subjects, after expanding to all possible pairs of haplotypes for each person. If `indx=i`, then  $i$  is the  $i$ th row of input matrix data. If the  $i$ th subject has  $n$  possible pairs of haplotypes that correspond to their marker phenotype, then  $i$  is repeated  $n$  times.
- `nreps` Vector for the count of haplotype pairs that map to each subject's marker genotypes.
- `hap1code` Vector of codes for each subject's first haplotype. The values in `hap1code` are the row numbers of the unique haplotypes in the returned matrix haplotype.
- `hap2code` Similar to `hap1code`, but for each subject's second haplotype.
- `post` Vector of posterior probabilities of pairs of haplotypes for a person, given thier marker phenotypes.

**Note**

Adapted from HAP.

**Author(s)**

Jing Hua Zhao

**See Also**

[hap](#), [LDk1](#)

**Examples**

```
## Not run:
data(hla)
hap.em(id=1:length(hla[,1]),data=hla[,3:8],locus.label=c("DQR","DQA","DQB"))

## End(Not run)
```

hap.score

*Score statistics for association of traits with haplotypes***Description**

Score statistics for association of traits with haplotypes

**Usage**

```
hap.score(
  y,
  geno,
  trait.type = "gaussian",
  offset = NA,
  x.adj = NA,
  skip.haplo = 0.005,
  locus.label = NA,
  miss.val = 0,
  n.sim = 0,
  method = "gc",
  id = NA,
  handle.miss = 0,
  mloci = NA,
  sexid = NA
)
```

**Arguments**

|             |                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| y           | Vector of trait values. For trait.type = "binomial", y must have values of 1 for event, 0 for no event.                                                                                                                                            |
| geno        | Matrix of alleles, such that each locus has a pair of adjacent columns of alleles, and the order of columns corresponds to the order of loci on a chromosome. If there are K loci, then ncol(geno) = 2*K. Rows represent alleles for each subject. |
| trait.type  | Character string defining type of trait, with values of "gaussian", "binomial", "poisson", "ordinal".                                                                                                                                              |
| offset      | Vector of offset when trait.type = "poisson".                                                                                                                                                                                                      |
| x.adj       | Matrix of non-genetic covariates used to adjust the score statistics. Note that intercept should not be included, as it will be added in this function.                                                                                            |
| skip.haplo  | Skip score statistics for haplotypes with frequencies < skip.haplo.                                                                                                                                                                                |
| locus.label | Vector of labels for loci, of length K (see definition of geno matrix).                                                                                                                                                                            |
| miss.val    | Vector of codes for missing values of alleles.                                                                                                                                                                                                     |

|             |                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------|
| n.sim       | Number of simulations for empirical p-values. If n.sim=0, no empirical p-values are computed. |
| method      | method of haplotype frequency estimation, "gc" or "hap".                                      |
| id          | an added option which contains the individual IDs.                                            |
| handle.miss | flag to handle missing genotype data, 0=no, 1=yes.                                            |
| mloci       | maximum number of loci/sites with missing data to be allowed in the analysis.                 |
| sexid       | flag to indicator sex for data from X chromosome, i=male, 2=female.                           |

## Details

Compute score statistics to evaluate the association of a trait with haplotypes, when linkage phase is unknown and diploid marker phenotypes are observed among unrelated subjects. For now, only autosomal loci are considered. This package haplo.score which this function is based is greatly acknowledged.

This is a version which substitutes haplo.em.

## Value

List with the following components:

- score.global Global statistic to test association of trait with haplotypes that have frequencies  $\geq$  skip.haplo.
- df Degrees of freedom for score.global.
- score.global.p P-value of score.global based on chi-square distribution, with degrees of freedom equal to df.
- score.global.p.sim P-value of score.global based on simulations (set equal to NA when n.sim=0).
- score.haplo Vector of score statistics for individual haplotypes that have frequencies  $\geq$  skip.haplo.
- score.haplo.p Vector of p-values for score.haplo, based on a chi-square distribution with 1 df.
- score.haplo.p.sim Vector of p-values for score.haplo, based on simulations (set equal to NA when n.sim=0).
- score.max.p.sim P-value of maximum score.haplo, based on simulations (set equal to NA when n.sim=0).
- haplotype Matrix of haplotypes analyzed. The ith row of haplotype corresponds to the ith item of score.haplo, score.haplo.p, and score.haplo.p.sim.
- hap.prob Vector of haplotype probabilities, corresponding to the haplotypes in the matrix haplotype.
- locus.label Vector of labels for loci, of length K (same as input argument).
- n.sim Number of simulations.
- n.val.global Number of valid simulated global statistics.
- n.val.haplo Number of valid simulated score statistics (score.haplo) for individual haplotypes.

## References

Schaid DJ, Rowland CM, Tines DE, Jacobson RM, Poland GA (2002). "Score tests for association between traits and haplotypes when linkage phase is ambiguous." *Am J Hum Genet*, **70**(2), 425-34. doi:10.1086/338688.

**Examples**

```
## Not run:
data(hla)
y<-hla[,2]
geno<-hla[,3:8]
# complete data
hap.score(y,geno,locus.label=c("DRB","DQA","DQB"))
# incomplete genotype data
hap.score(y,geno,locus.label=c("DRB","DQA","DQB"),handle.miss=1,mloci=1)
unlink("assign.dat")

### note the differences in p values in the following runs
data(alDH2)
# to subset the data since hap doesn't handle one allele missing
deleted<-c(40,239,256)
alDH2[deleted,]
alDH2<-alDH2[-deleted,]
y<-alDH2[,2]
geno<-alDH2[,3:18]
# only one missing locus
hap.score(y,geno,handle.miss=1,mloci=1,method="hap")
# up to seven missing loci and with 10,000 permutations
hap.score(y,geno,handle.miss=1,mloci=7,method="hap",n.sim=10000)

# hap.score takes considerably longer time and does not handle missing data
hap.score(y,geno,n.sim=10000)

## End(Not run)
```

hg18

*Chromosomal lengths for build 36***Description**

Data are used in other functions.

**Usage**

```
data(hg18)
```

**Format**

A vector containing lengths of chromosomes.

**Details**

generated from GRCh.R.

---

|      |                                         |
|------|-----------------------------------------|
| hg19 | <i>Chromosomal lengths for build 37</i> |
|------|-----------------------------------------|

---

**Description**

Data are used in other functions.

**Usage**

```
data(hg19)
```

**Format**

A vector containing lengths of chromosomes.

---

|      |                                         |
|------|-----------------------------------------|
| hg38 | <i>Chromosomal lengths for build 38</i> |
|------|-----------------------------------------|

---

**Description**

Data are used in other functions.

**Usage**

```
data(hg38)
```

**Format**

A vector containing lengths of chromosomes.

---

|              |                                                    |
|--------------|----------------------------------------------------|
| hmht.control | <i>Controls for highlighted regions in mhtplot</i> |
|--------------|----------------------------------------------------|

---

**Description**

Helper function defining highlighted markers or genomic regions to be emphasised in `mhtplot()`.

**Usage**

```
hmht.control(  
  data = NULL,  
  colors = "red",  
  yoffset = 0.25,  
  cex = 1.2,  
  boxed = FALSE  
)
```

**Arguments**

|         |                                                                                                                |
|---------|----------------------------------------------------------------------------------------------------------------|
| data    | Data frame with <b>four columns</b> : chromosome, position, value and label (e.g. gene name).                  |
| colors  | Character vector of colours used for highlighted regions. Colours are recycled if multiple labels are present. |
| yoffset | Numeric vertical offset added to the highest highlighted point before placing the label.                       |
| cex     | Numeric scaling factor for label text.                                                                         |
| boxed   | Logical. If TRUE, labels are drawn inside a white box with black border.                                       |

**Value**

A list of highlight control parameters for `mhtplot()`.

**See Also**

`mhtplot()`, `mht.control()`

**Examples**

```
## Example highlight specification
hdata <- data.frame(
  chr = c(1,3,5),
  pos = c(10000,50000,90000),
  p    = c(1e-8,1e-7,1e-9),
  gene = c("GENE1","GENE2","GENE3")
)
hmht.control(data = hdata, cex=0.8, colors = "red", boxed = TRUE)
```

---

htr

*Haplotype Trend Regression*


---

**Description**

Haplotype Trend Regression

**Usage**

```
htr(y, x, n.sim = 0L)
```

**Arguments**

|       |                                     |
|-------|-------------------------------------|
| y     | Numeric phenotype vector.           |
| x     | Haplotype dosage matrix.            |
| n.sim | Number of permutations (default 0). |

## Details

Performs haplotype trend regression (HTR) with optional permutation testing. The method follows Zaykin et al. (2002).

The model fits a linear regression of phenotype on haplotype dosage indicators and computes:

- Overall F statistic for joint haplotype effects
- Individual haplotype F statistics
- Optional permutation-based p-values for both

Permutation p-values are computed using  $(b + 1)/(B + 1)$  to avoid zero-valued estimates.

## Value

A list with the following components:

**f** Overall F statistic for global haplotype association.

**p** P-value for the overall test (asymptotic or permutation-based).

**fv** Vector of F statistics for individual haplotypes.

**pi** Vector of p-values for individual haplotypes.

## References

Zaykin DV, Westfall PH, Young SS, Karnoub MA, Wagner MJ, Ehm MG (2002). “Testing association of statistically inferred haplotypes with discrete and continuous traits in samples of unrelated individuals.” *Hum Hered*, **53**(2), 79-91. doi:10.1159/000057986.

Xie R, Stram DO (2005). “Asymptotic equivalence between two score tests for haplotype-specific risk in general linear models.” *Genetic Epidemiology*, **29**(2), 166-170. doi:10.1002/gepi.20087.

## Examples

```
## Not run:
## 26-10-2003
## This example is now part of the demo.
if (!requireNamespace("gap.examples", quietly = TRUE))
  remotes::install_github("jinghuazhao/R/gap.examples")

filespec <- file.path(
  find.package("gap.examples"),
  "tests", "htr", "test2.dat"
)
test2 <- read.table(filespec)
y <- test2[, 1]
x <- test2[, -1]
y <- as.matrix(y)
x <- as.matrix(x)
htr.test2 <- htr(y, x)
htr.test2
htr.test2 <- htr(y, x, n.sim = 10)
htr.test2

## 13-11-2003
library(gap.datasets)
data(apoeapoc)
```



```

apoeapoc.gc <- gc.em(apoeapoc[, 5:8])
y <- apoeapoc$y
y[y == 2] <- 1

htr(y, apoeapoc.gc$htrtable)

## 20-08-2008
## Part of the useR! 2008 tutorial by Andrea Foulkes.
## The approach may be used beyond the generalized linear
## model (GLM) framework.
HaploEM <- haplo.em(Geno, locus.label = SNPnames)
HapMat <- HapDesign(HaploEM)

m1 <- lm(Trait ~ HapMat)
m2 <- lm(Trait ~ 1)

anova(m2, m1)

## End(Not run)

```

hwe

*Hardy-Weinberg Equilibrium Test (Multiallelic, Unified Interface)*

## Description

Unified Hardy-Weinberg equilibrium (HWE) testing procedure for multiallelic loci. All input formats are internally converted to a single genotype count matrix, ensuring identical inference across representations.

## Usage

```

hwe(
  data,
  type = c("alleles", "genotypes", "counts"),
  verbose = TRUE,
  yates.correct = FALSE,
  B = 1e+05,
  seed = 123
)

```

## Arguments

|               |                                                                                                                                                                                                                                              |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data          | genotype data in one of three formats: <ul style="list-style-type: none"> <li>• alleles two-column allele pairs</li> <li>• genotypes integer genotype IDs (triangular encoding)</li> <li>• counts symmetric genotype count matrix</li> </ul> |
| type          | input format: "alleles", "genotypes", or "counts"                                                                                                                                                                                            |
| verbose       | logical; if TRUE, prints full test output                                                                                                                                                                                                    |
| yates.correct | logical; if TRUE applies Yates continuity correction to Pearson chi-square statistic only                                                                                                                                                    |
| B             | number of Monte Carlo replicates for exact test                                                                                                                                                                                              |
| seed          | random seed for Monte Carlo exact test                                                                                                                                                                                                       |

### Details

Let allele frequencies be:

$$p_i = \frac{c_i}{2n}$$

Under Hardy–Weinberg equilibrium:

$$P_{ii} = p_i^2, \quad P_{ij} = 2p_i p_j \quad (i \neq j)$$

Expected genotype counts:

$$E_{ij} = nP_{ij}$$

All input formats are mapped to the same genotype matrix, ensuring algebraic equivalence.

*Pearson chi-square*

$$X^2 = \sum_{i \leq j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

With optional Yates correction:

$$X^2 = \sum \frac{(|O_{ij} - E_{ij}| - 0.5)^2}{E_{ij}}$$

*Likelihood ratio test*

$$G^2 = 2 \sum_{i \leq j} O_{ij} \log(O_{ij}/E_{ij})$$

with convention  $0 \log(0) = 0$ .

*Inbreeding coefficient*

$$F = \frac{H_{obs} - H_{exp}}{1 - H_{exp}}$$

where:

$$H_{obs} = \sum_i O_{ii}/n, \quad H_{exp} = \sum_i p_i^2$$

Under:

$$X \sim \text{Multinomial}(n, P)$$

the p-value is:

$$p = \Pr(\ell(X^{sim}) \leq \ell(X^{obs}))$$

where:

$$\ell(x) = \sum x_i \log p_i + \log \frac{n!}{\prod x_i!}$$

$$\text{alleles} \equiv \text{genotype IDs} \equiv \text{count matrix} \rightarrow M_{ij}$$

Hence all statistics are identical up to Monte Carlo variation.

**Value**

Named list containing:

- source — input representation used ("alleles", "genotypes", or "counts")
- X2 — Pearson chi-square statistic
- p\_X2 — asymptotic p-value of Pearson chi-square test
- LRT — likelihood ratio statistic
- p\_LRT — asymptotic p-value of likelihood ratio test
- p\_exact — Monte Carlo exact test p-value
- freq — allele frequency vector
- rho — inbreeding coefficient

**Note**

Note that

- Zero-frequency alleles are removed automatically
- Exact test is Monte Carlo (fixed seed for reproducibility)
- All statistics are computed on the same standardized genotype matrix

**Examples**

```
## Not run:
a1 <- c(1,1,1,1,2,2,2,3,3,1,2,3,1,2,3,1,2,3)
a2 <- c(1,2,3,1,2,3,2,3,1,2,3,1,1,1,2,3,2,3)
r1 <- hwe(cbind(a1,a2), "alleles", FALSE)
g <- a2g(a1,a2)
r2 <- hwe(g, "genotypes", FALSE)
g_tab <- table(g)
pairs <- g2a(as.integer(names(g_tab)))
k <- max(pairs)
M <- matrix(0, k, k)
for(i in seq_len(nrow(pairs))) {
  M[pairs[i,1], pairs[i,2]] <- g_tab[i]
}
r3 <- hwe(M, "counts", FALSE)
r <- lapply(list(r1, r2, r3), \(x) within(as.data.frame(x), {
  freq <- paste(round(as.numeric(freq), 3), collapse=";")))[1,])
do.call(rbind,r)

## End(Not run)
```

---

hwe.cc

---

*A likelihood ratio test of population Hardy-Weinberg equilibrium for case-control studies*


---

**Description**

A likelihood ratio test of population Hardy-Weinberg equilibrium for case-control studies

**Usage**

```
hwe.cc(model, case, ctrl, k0, initial1, initial2)
```

**Arguments**

|          |                                             |
|----------|---------------------------------------------|
| model    | model specification, dominant, recessive.   |
| case     | a vector of genotype counts in cases.       |
| ctrl     | a vector of genotype counts in controls.    |
| k0       | prevalence of disease in the population.    |
| initial1 | initial values for beta, gamma, and q.      |
| initial2 | initial values for logit(p) and log(gamma). |

**Details**

A likelihood ratio test of population Hardy-Weinberg equilibrium for case-control studies

This is a collection of utility functions. The null hypothesis declares that the proportions of genotypes are according to Hardy-Weinberg law, while under the alternative hypothesis, the expected genotype counts are according to the probabilities that particular genotypes are obtained conditional on the prevalence of disease in the population. In so doing, Hardy-Weinberg equilibrium is considered using both case and control samples but pending on the disease model such that 2-parameter multiplicative model is built on baseline genotype  $\alpha$ ,  $\alpha\beta$  and  $\alpha\gamma$ .

**Value**

The returned value is a list with the following components.

- Cox statistics under a general model.
- t2par under the null hypothesis.
- t3par under the alternative hypothesis.
- lrt.stat the log-likelihood ratio statistic.
- pval the corresponding p value.

**Author(s)**

Chang Yu, Li Wang, Jing Hua Zhao

**References**

Yu C, Zhang S, Zhou C, Sile S (2009). "A likelihood ratio test of population Hardy-Weinberg equilibrium for case-control studies." *Genet Epidemiol*, **33**(3), 275-80. doi:10.1002/gepi.20381.

**See Also**

[hwe](#)

**Examples**

```
## Not run:
### Saba Sile, email of Jan 26, 2007, data always in order of GG AG AA, p=Pr(G),
### q=1-p=Pr(A)
case=c(155,27,4)
ctrl=c(408,55,15)
k0=.2
initial1=c(1.0,0.94,0.0904)
initial2=c(logit(1-0.0904),log(0.94))
hwe.cc("recessive",case,ctrl,k0, initial1, initial2)

### John Phillips III, TGFb1 data codon 10: TT CT CC, CC is abnormal and increasing
### TGFb1 activity
case=c(29,78,13)
ctrl=c(17,28,6)
k0 <- 1e-5
initial1 <- c(2.45,2.45,0.34)
initial2 <- c(logit(1-0.34),log(2.45))
hwe.cc("dominant",case,ctrl,k0,initial1,initial2)

## End(Not run)
```

hwe.hardy

*Hardy-Weinberg equilibrium test using MCMC***Description**

Hardy-Weinberg equilibrium test using MCMC

**Usage**

```
hwe.hardy(a, alleles = 3, seed = 3000, sample = c(1000, 1000, 5000))
```

**Arguments**

|         |                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------|
| a       | an array containing the genotype counts, as integer.                                                      |
| alleles | number of allele at the locus, greater than or equal to 3, as integer.                                    |
| seed    | pseudo-random number seed, as integer.                                                                    |
| sample  | optional, parameters for MCMC containing number of chunks, size of a chunk and burn-in steps, as integer. |

**Details**

Hardy-Weinberg equilibrium test by MCMC

**Value**

The returned value is a list containing:

- method Hardy-Weinberg equilibrium test using MCMC.
- data.name name of used data if x is given.

- p.value Monte Carlo p value.
- p.value.se standard error of Monte Carlo p value.
- switches percentage of switches (partial, full and altogether).

### Note

Codes are commented for taking x a genotype object, as genotype to prepare a and alleles on the fly.

Adapted from HARDY, testable with -Dexecutable as standalone program.

keywords htest

### Author(s)

Sun-Wei Guo, Jing Hua Zhao, Gregor Gorjanc

### Source

<https://sites.stat.washington.edu/thompson/Genepi/pangaea.shtml>

### References

Guo SW, Thompson EA (1992). "Performing the exact test of Hardy-Weinberg proportion for multiple alleles." *Biometrics*, **48**(2), 361-72.

### See Also

[hwe](#), [genetics::HWE.test](#), [genetics::genotype](#)

### Examples

```
## Not run:
# example 2 from hwe.doc:
a<-c(
  3,
  4, 2,
  2, 2, 2,
  3, 3, 2, 1,
  0, 1, 0, 0, 0,
  0, 0, 0, 0, 0, 1,
  0, 0, 1, 0, 0, 0, 0,
  0, 0, 0, 2, 1, 0, 0, 0)
ex2 <- hwe.hardy(a=a,alleles=8)

# example using HLA
data(hla)
x <- hla[,3:4]
y <- pgc(x,handle.miss=0,with.id=1)
n.alleles <- max(x,na.rm=TRUE)
z <- vector("numeric",n.alleles*(n.alleles+1)/2)
z[y$idsave] <- y$wt
hwe.hardy(a=z,alleles=n.alleles)

# with use of class 'genotype'
# this is to be fixed
library(genetics)
```

```

hlagen <- genotype(a1=x$DQR.a1, a2=x$DQR.a2,
                  alleles=sort(unique(c(x$DQR.a1, x$DQR.a2))))
hwe.hardy(hlagen)

# comparison with hwe
hwe(z,data.type="count")

# to create input file for HARDY
print.tri<-function (xx,n) {
  cat(n,"\n")
  for(i in 1:n) {
    for(j in 1:i) {
      cat(xx[i,j], " ")
    }
    cat("\n")
  }
  cat("100 170 1000\n")
}
xx<-matrix(0,n.alleles,n.alleles)
xxx<-lower.tri(xx,diag=TRUE)
xx[xxx]<-z
sink("z.dat")
print.tri(xx,n.alleles)
sink()
# now call as: hwe z.dat z.out

## End(Not run)

```

hwe.jags

*Hardy-Weinberg equilibrium test for a multiallelic marker using JAGS***Description**

Hardy-Weinberg equilibrium test for a multiallelic marker using JAGS

**Usage**

```

hwe.jags(
  k,
  n,
  delta = rep(1/k, k),
  lambda = 0,
  lambdamu = -1,
  lambdasd = 1,
  parms = c("p", "f", "q", "theta", "lambda"),
  ...
)

```

**Arguments**

**k**                      number of alleles.

**n**                      a vector of  $k(k+1)/2$  genotype counts.

|          |                                                              |
|----------|--------------------------------------------------------------|
| delta    | initial parameter values.                                    |
| lambda   | initial parameter values.                                    |
| lambdamu | initial parameter values.                                    |
| lambdasd | initial parameter values.                                    |
| parms    | monitored parameters.                                        |
| ...      | parameters passed to jags, e.g., n.chains, n.burnin, n.iter. |

### Details

Hardy-Weinberg equilibrium test.

This function performs Bayesian Hardy-Weinberg equilibrium test, which mirrors [hwe.hardy](#), another implementation for highly polymorphic markers when asymptotic results do not hold.

### Value

The returned value is a fitted model from jags().

### Author(s)

Jing Hua Zhao, Jon Wakefield

### References

Wakefield J (2010). "Bayesian methods for examining Hardy-Weinberg equilibrium." *Biometrics*, **66**(1), 257-65. doi:[10.1111/j.15410420.2009.01267.x](#).

### See Also

[hwe.hardy](#)

### Examples

```
## Not run:
ex1 <- hwe.jags(4,c(5,6,1,7,11,2,8,19,26,15))
print(ex1)
ex2 <- hwe.jags(2,c(49,45,6))
print(ex2)
ex3 <- hwe.jags(4,c(0,3,1,5,18,1,3,7,5,2),lambda=0.5,lambdamu=-2.95,lambdasd=1.07)
print(ex3)
ex4 <- hwe.jags(9,c(1236,120,3,18,0,0,982,55,7,249,32,1,0,12,0,2582,132,20,1162,29,
1312,6,0,0,4,0,4,0,2,0,0,0,0,0,0,0,115,5,2,53,1,149,0,0,4),
delta=c(1,1,1,1,1,1,1,1,1),lambdamu=-4.65,lambdasd=0.21)
print(ex4)
ex5 <- hwe.jags(8,n=c(
  3,
  4, 2,
  2, 2, 2,
  3, 3, 2, 1,
  0, 1, 0, 0, 0,
  0, 0, 0, 0, 0, 1,
  0, 0, 1, 0, 0, 0, 0,
  0, 0, 0, 2, 1, 0, 0, 0))
print(ex5)
```



```

# Data and code according to the following URL,
# http://darwin.eeb.uconn.edu/eeb348-notes/testing-hardy-weinberg.pdf
hwe.jags.AB0 <- function(n,...)
{
  hwe <- function() {
    # likelihood
    pi[1] <- p.a*p.a + 2*p.a*p.o
    pi[2] <- 2*p.a*p.b
    pi[3] <- p.b*p.b + 2*p.b*p.o
    pi[4] <- p.o*p.o
    n[1:4] ~ dmulti(pi[],N)
    # priors
    a1 ~ dexp(1)
    b1 ~ dexp(1)
    o1 ~ dexp(1)
    p.a <- a1/(a1 + b1 + o1)
    p.b <- b1/(a1 + b1 + o1)
    p.o <- o1/(a1 + b1 + o1)
  }
  hwd <- function() {
    # likelihood
    pi[1] <- p.a*p.a + f*p.a*(1-p.a) + 2*p.a*p.o*(1-f)
    pi[2] <- 2*p.a*p.b*(1-f)
    pi[3] <- p.b*p.b + f*p.b*(1-p.b) + 2*p.b*p.o*(1-f)
    pi[4] <- p.o*p.o + f*p.o*(1-p.o)
    n[1:4] ~ dmulti(pi[],N)
    # priors
    a1 ~ dexp(1)
    b1 ~ dexp(1)
    o1 ~ dexp(1)
    p.a <- a1/(a1 + b1 + o1)
    p.b <- b1/(a1 + b1 + o1)
    p.o <- o1/(a1 + b1 + o1)
    f ~ dunif(0,1)
  }
  N <- sum(n)
  AB0.hwe <- R2jags::jags(list(n=n,N=N),,c("pi","p.a","p.b","p.o"),hwe,...)
  AB0.hwd <- R2jags::jags(list(n=n,N=N),,c("pi","p.a","p.b","p.o","f"),hwd,...)
  invisible(list(hwe=AB0.hwe,hwd=AB0.hwd))
}
hwe.jags.AB0.results <- hwe.jags.AB0(n=c(862, 131, 365, 702))
hwe.jags.AB0.results

## End(Not run)

```

---

invnormal

*Inverse normal transformation*


---

## Description

Inverse normal transformation

## Usage

invnormal(x)

**Arguments**

x                      Data with missing values.

**Value**

Transformed value.

**Examples**

```
x <- 1:10
z <- invnormal(x)
plot(z,x,type="b")
```

---

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| inv_chr_pos_a1_a2 | <i>Retrieval of chr:pos+a1/a2 according to SNP id</i> |
|-------------------|-------------------------------------------------------|

---

**Description**

This function obtains information embedded in unique identifiers.

**Usage**

```
inv_chr_pos_a1_a2(chr_pos_a1_a2, prefix = "chr", seps = c(":", "_", "-"))
```

**Arguments**

chr\_pos\_a1\_a2    SNP id.  
 prefix            Prefix of the identifier.  
 seps             Delimiters of fields.

**Value**

A data.frame with the following variables:

- chr Chromosome.
- pos Position.
- a1 Allele 1.
- a2 Allele 2.

**Examples**

```
# rs12075
inv_chr_pos_a1_a2("chr1:159175354_A_G",prefix="chr",seps=c(":", "_", "-"))
```

ixy

*Conversion of chrosome name from strings***Description**

This function converts 1:22, X, Y back to 1:24.

**Usage**

```
ixy(x)
```

**Arguments**

x                      Chromosome name in strings

**Value**

As indicated.

KCC

*Disease prevalences in cases and controls***Description**

Disease prevalences in cases and controls

**Usage**

```
KCC(model, GRR, p1, K)
```

**Arguments**

|       |                                                                                               |
|-------|-----------------------------------------------------------------------------------------------|
| model | disease model (one of "multiplicative", "additive", "recessive", "dominant", "overdominant"). |
| GRR   | genotype relative risk.                                                                       |
| p1    | disease allele frequency.                                                                     |
| K     | disease prevalence in the whole population.                                                   |

**Details**

KCC calculates disease prevalences in cases and controls for a given genotype relative risk, allele frequency and prevalencen of the disease in the whole population. It is used by tscc and pbsize2.

**Value**

A list of two elements:

- pprime prevlence in cases.
- p prevalence in controls.

---

|            |                                           |
|------------|-------------------------------------------|
| kin.morgan | <i>kinship matrix for simple pedigree</i> |
|------------|-------------------------------------------|

---

**Description**

kinship matrix for simple pedigree

**Usage**

```
kin.morgan(ped, verbose = FALSE)
```

**Arguments**

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| ped     | A matrix with columns on individual's id, father's id and mother's id. |
| verbose | an option to print out the original pedigree.                          |

**Details**

kinship matrix according to Morgan v2.1.

**Value**

The returned value is a list containing:

- kin the kinship matrix in vector form.
- kin.matrix the kinship matrix.

**Note**

The input data is required to be sorted so that parents preceed their children.

**Author(s)**

Morgan development team, Jing Hua Zhao

**References**

Morgan V2.1, <https://faculty.washington.edu/eathomp/Genepi/MORGAN/Morgan.shtml>

**See Also**

[gif](#)

**Examples**

```
## Not run:
# Werner syndrome pedigree
werner<-c(
  1, 0, 0, 1,
  2, 0, 0, 2,
  3, 0, 0, 2,
  4, 1, 2, 1,
  5, 0, 0, 1,
```

```

6, 1, 2, 2,
7, 1, 2, 2,
8, 0, 0, 1,
9, 4, 3, 2,
10, 5, 6, 1,
11, 5, 6, 2,
12, 8, 7, 1,
13,10, 9, 2,
14,12, 11, 1,
15,14, 13, 1)
werner<-t(matrix(werner,nrow=4))
kin.morgan(werner[,1:3])

## End(Not run)

```

klem

*Haplotype frequency estimation based on a genotype table of two multiallelic markers*

### Description

Haplotype frequency estimation using expectation-maximization algorithm based on a table of genotypes of two multiallelic markers.

### Usage

```
klem(obs, k = 2, l = 2)
```

### Arguments

|     |                                |
|-----|--------------------------------|
| obs | a table of genotype counts.    |
| k   | number of alleles at marker 1. |
| l   | number of alleles at marker 2. |

The dimension of the genotype table should be  $k*(k+1)/2 \times l*(l+1)/2$ .  
Modified from 2ld.c.

### Value

The returned value is a list containing:

- h haplotype Frequencies.
- l0 log-likelihood under linkage equilibrium.
- l1 log-likelihood under linkage disequilibrium.

### Author(s)

Jing Hua Zhao

### See Also

[genecounting](#)

Examples

```
## Not run:
# an example with known genotype counts
z <- klem(obs=1:9)
# an example with imputed genotypes at SH2B1
source(file.path(find.package("gap"), "scripts", "SH2B1.R"), echo=TRUE)

## End(Not run)
```

---

|                |                                         |
|----------------|-----------------------------------------|
| labelManhattan | <i>Annotate Manhattan or Miami Plot</i> |
|----------------|-----------------------------------------|

---

Description

Annotate Manhattan or Miami Plot

Usage

```
labelManhattan(
  chr,
  pos,
  name,
  gwas,
  gwasChrLab = "chr",
  gwasPosLab = "pos",
  gwasPLab = "p",
  gwasZLab = "NULL",
  chrmaxpos,
  textPos = 4,
  angle = 0,
  miamiBottom = FALSE
)
```

Arguments

|            |                                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| chr        | A vector of chromosomes for the markers to be labelled.                                                                                                       |
| pos        | A vector of positions on the chromosome for the markers to be labelled. These must correspond to markers in the GWAS dataset used to make the manhattan plot. |
| name       | A vector of labels to be added next to the points specified by chr and pos.                                                                                   |
| gwas       | The same GWAS dataset used to plot the existing Manhattan or Miami plot to be annotated.                                                                      |
| gwasChrLab | The name of the column in gwas containing chromosome number. Defaults to "chr".                                                                               |
| gwasPosLab | The name of the column in gwas containing position. Defaults to "pos".                                                                                        |
| gwasPLab   | The name of the column in gwas containing p-values. Defaults to "p".                                                                                          |
| gwasZLab   | The name of the column in gwas containing z-values. Defaults to "NULL".                                                                                       |

|             |                                                                                                                                                                                                                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| chrmaxpos   | Data frame containing x coordinates for chromosome start positions, generated by <a href="#">labelManhattan</a> .                                                                                                                                                                                                                      |
| textPos     | An integer or vector dictating where the label should be plotted relative to each point. Good for avoiding overlapping labels. Provide an integer to plot all points in the same relative position or use a vector to specify position for each label. Passed to the 'pos' option of <a href="#">graphics::text</a> . Defaults to '4'. |
| angle       | An integer or vector dictating the plot angle of the label for each point. Provide an integer to plot all points in the same relative position or use a vector to specify position for each label. Passed to the 'srt' option of <a href="#">graphics::text</a> . Defaults to '0'.                                                     |
| miamiBottom | If 'TRUE', labels will be plotted on the lower region of a Miami plot. If 'FALSE', labels will be plotted on the upper region. Defaults to 'FALSE'.                                                                                                                                                                                    |

### Details

Add labels beside specified points on a Manhattan or Miami plot. Ideal for adding locus names to peaks. Currently only designed to work with [miamiplot2](#).

### Value

Adds annotation to existing Manhattan or Miami plot

### Note

Extended to handle extreme P values.

### Author(s)

Jonathan Marten

### Examples

```
## Not run:
labelManhattan(c(4,5,11,19),c(9994215,16717922,45538760,51699256),
               c("GENE1","GENE2","GENE3","GENE4"),
               gwas1,chrmaxpos=chrmaxpos)
labelManhattan(geneLabels$chr,geneLabel$pos,geneLabel$geneName,gwas1,chrmaxpos=chrmaxpos)

## End(Not run)
```

### Description

LD statistics for two diallelic markers

### Usage

```
LD22(h, n)
```

## Arguments

|   |                                    |
|---|------------------------------------|
| h | a vector of haplotype frequencies. |
| n | number of haplotypes.              |

## Details

It is possible to perform permutation test of  $r^2$  by re-ordering the genotype through R's sample function, obtaining the haplotype frequencies by [gc.em](#) or [genecounting](#), supplying the estimated haplotype frequencies to the current function and record x2, and comparing the observed x2 and that from the replicates.

## Value

The returned value is a list containing:

- h the original haplotype frequency vector.
- n the number of haplotypes.
- D the linkage disequilibrium parameter.
- VarD the variance of D.
- Dmax the maximum of D.
- VarDmax the variance of Dmax.
- Dprime the scaled disequilibrium parameter.
- VarDprime the variance of Dprime.
- x2 the Chi-squared statistic.
- lor the log(OR) statistic.
- vlor the var(log(OR)) statistic.

## Note

extracted from 2ld.c, worked 28/6/03, tables are symmetric do not fix, see kbyl below

## Author(s)

Jing Hua Zhao

## References

- Zabetian CP, Buxbaum SG, Elston RC, Köhnke MD, Anderson GM, Gelernter J, Cubells JF (2003). "The structure of linkage disequilibrium at the DBH locus strongly influences the magnitude of association between diallelic markers and plasma dopamine beta-hydroxylase activity." *Am J Hum Genet*, **72**(6), 1389-400. doi:10.1086/375499.
- Zapata C, Alvarez G, Carollo C (1997). "Approximate variance of the standardized measure of gametic disequilibrium D'." *Am J Hum Genet*, **61**(3), 771-4. doi:10.1016/s00029297(07)643420.

## See Also

[LDk1](#)



**Examples**

```
## Not run:
h <- c(0.442356,0.291532,0.245794,0.020319)
n <- 481*2
t <- LD22(h,n)

## End(Not run)
```

LDkl

*LD statistics for two multiallelic markers***Description**

LD statistics for two multiallelic loci. For two diallelic makers, the familiar  $r^2$  has standard error  $seX2$ .

**Usage**

```
LDkl(n1 = 2, n2 = 2, h, n, optrho = 2, verbose = FALSE)
```

**Arguments**

|         |                                                                                    |
|---------|------------------------------------------------------------------------------------|
| n1      | number of alleles at marker 1.                                                     |
| n2      | number of alleles at marker 2.                                                     |
| h       | a vector of haplotype frequencies.                                                 |
| n       | number of haplotypes.                                                              |
| optrho  | type of contingency table association, 0=Pearson, 1=Tschuprow, 2=Cramer (default). |
| verbose | detailed output of individual statistics.                                          |

**Value**

The returned value is a list containing:

- n1 the number of alleles at marker 1.
- n2 the number of alleles at marker 2.
- h the haplotype frequency vector.
- n the number of haplotypes.
- Dp D'.
- VarDp variance of D'.
- Dijtable table of Dij.
- VarDijtable table of variances for Dij.
- Dmaxtable table of Dmax.
- Dijptable table of Dij'.
- VarDijptable table of variances for Dij'.
- X2table table of Chi-squares (based on Dij).

- ptable table of p values.
- x2 the Chi-squared statistic.
- seX2 the standard error of x2/n.
- rho the measure of association.
- seR the standard error of rho.
- optrho the method for calculating rho.
- kinfo the Kullback-Leibler information.

### Note

adapted from 2ld.c.

### Author(s)

Jing Hua Zhao

### References

Bishop YMM, Fienberg SE, Holland PW (1975). *Discrete multivariate analysis: theory and practice*. MIT Press, Cambridge, Mass. ISBN 9780262021135.

Cramer H (1946). *Mathematical Methods of Statistics*. Princeton Univ. Press.

Zapata C, Carollo C, Rodriguez S (2001). "Sampling variance and distribution of the D' measure of overall gametic disequilibrium between multiallelic loci." *Ann Hum Genet*, **65**(Pt 4), 395-406. doi:[10.1017/s0003480001008697](https://doi.org/10.1017/s0003480001008697).

Zhao JH (2004). "2LD. GENECOUNTING and HAP: computer programs for linkage disequilibrium analysis." *Bioinformatics*, **20**(8), 1325-6. doi:[10.1093/bioinformatics/bth071](https://doi.org/10.1093/bioinformatics/bth071).

### See Also

[LD22](#)

### Examples

```
## Not run:
# two examples in the C program 2LD:
# two SNPs as in 2by2.dat
# this can be compared with output from LD22

h <- c(0.442356,0.291532,0.245794,0.020319)
n <- 481*2
t <- LDkl(2,2,h,n)
t

# two multiallelic markers as in kbyl.dat
# the two-locus haplotype vector is in file "kbyl.dat"
# The data is now available from 2ld in Haplotype-Analysis

filespec <- system.file("kbyl.dat")
h <- scan(filespec,skip=1)
t <- LDkl(9,5,h,213*2,verbose=TRUE)

## End(Not run)
```

---

|        |                                        |
|--------|----------------------------------------|
| log10p | <i>log10(p) for a normal deviate z</i> |
|--------|----------------------------------------|

---

**Description**

log10(p) for a normal deviate z

**Usage**

log10p(z)

**Arguments**

z                      normal deviate.

**Value**

log10(P)

**Author(s)**

James Peters

**Examples**

log10p(100)

---

|             |                                                               |
|-------------|---------------------------------------------------------------|
| log10pvalue | <i>log10(p) for a P value including its scientific format</i> |
|-------------|---------------------------------------------------------------|

---

**Description**

log10(p) for a P value including its scientific format

**Usage**

log10pvalue(p = NULL, base = NULL, exponent = NULL)

**Arguments**

p                      value.  
base                   base part in scientific format.  
exponent              exponent part in scientific format.

**Value**

log10(P)

**Examples**

log10pvalue(1e-323)  
log10pvalue(base=1,exponent=-323)

---

|      |                                      |
|------|--------------------------------------|
| logp | <i>log(p) for a normal deviate z</i> |
|------|--------------------------------------|

---

**Description**

log(p) for a normal deviate z

**Usage**

logp(z)

**Arguments**

z                      normal deviate.

**Value**

log(P)

**Examples**

logp(100)

---

|         |                                                               |
|---------|---------------------------------------------------------------|
| makeped | <i>A function to prepare pedigrees in post-MAKEPED format</i> |
|---------|---------------------------------------------------------------|

---

**Description**

A function to prepare pedigrees in post-MAKEPED format

**Usage**

```
makeped(
  pifile = "pedfile.pre",
  pofile = "pedfile.ped",
  auto.select = 1,
  with.loop = 0,
  loop.file = NA,
  auto.proband = 1,
  proband.file = NA
)
```

**Arguments**

|             |                                                                                         |
|-------------|-----------------------------------------------------------------------------------------|
| pifile      | input filename.                                                                         |
| pofile      | output filename.                                                                        |
| auto.select | no loops in pedigrees and probands are selected automatically? 0=no, 1=yes.             |
| with.loop   | input data with loops? 0=no, 1=yes.                                                     |
| loop.file   | filename containing pedigree id and an individual id for each loop, set if with.loop=1. |

|              |                                                                                          |
|--------------|------------------------------------------------------------------------------------------|
| auto.proband | probands are selected automatically? 0=no, 1=yes.                                        |
| proband.file | filename containing pedigree id and proband id, set if auto.proband=0 (not implemented). |

### Details

Many computer programs for genetic data analysis requires pedigree data to be in the so-called “post-MAKEPED” format. This function performs this translation and allows for some inconsistencies to be detected.

The first four columns of the input file contains the following information:

pedigree ID, individual ID, father’s ID, mother’s ID, sex

Either father’s or mother’s id is set to 0 for founders, i.e. individuals with no parents. Numeric coding for sex is 0=unknown, 1=male, 2=female. These can be followed by satellite information such as disease phenotype and marker information.

The output file has extra information extracted from data above.

Before invoking makeped, input file, loop file and proband file have to be prepared.

By default, auto.select=1, so translation proceeds without considering loops and proband statuses. If there are loops in the pedigrees, then set auto.select=0, with.loop=1, loop.file="filespec".

There may be several versions of makeped available, but their differences with this port should be minor.

### Note

adapted from makeped.c by W Li and others. keywords datagen

### Examples

```
## Not run:
cwd <- getwd()
cs.dir <- file.path(find.package("gap.examples"), "tests", "kinship")
setwd(cs.dir)
dir()
makeped("ped7.pre", "ped7.ped", 0, 1, "ped7.lop")
setwd(cwd)
# https://lab.rockefeller.edu/ott/

## End(Not run)
```

### Description

Generates a Relative Log Expression (RLE) plot for quality assessment of gene expression data. For each feature (row), the median expression across samples is subtracted, and boxplots of the resulting relative expression values are displayed for each sample.

**Usage**

```
makeRLEplot(
  E,
  log2.data = TRUE,
  groups = NULL,
  col.group = NULL,
  showTitle = FALSE,
  title = "Relative log expression (RLE) plot",
  ...
)
```

**Arguments**

|           |                                                                                       |
|-----------|---------------------------------------------------------------------------------------|
| E         | A numeric matrix or data frame with features in rows and samples in columns.          |
| log2.data | Logical; if TRUE, expression values are log2-transformed before computing RLE values. |
| groups    | Optional factor or vector defining sample groups. Must have length equal to ncol(E).  |
| col.group | Optional named vector of colours matching group levels.                               |
| showTitle | Logical; if TRUE, display a plot title.                                               |
| title     | Character string specifying the plot title.                                           |
| ...       | Additional arguments passed to graphics::boxplot().                                   |

**Value**

Invisibly returns the matrix of relative log expression values.

**Examples**

```
## Not run:
set.seed(123)
E <- matrix(rlnorm(987*200, meanlog=5, sdlog=1), nrow=987, ncol=200)
colnames(E) <- paste0("Sample_", seq_len(200))
rownames(E) <- paste0("Gene_", seq_len(987))
group <- rep(c("Control", "Treatment"), each=100)
col.group <- c(Control="steelblue", Treatment="tomato")
par(mar = c(10,4,4,2))
makeRLEplot(E, log2.data=TRUE, groups=group, col.group=col.group, cex=0.4,
  cex.axis=0.5, showTitle=TRUE, title="RLE plot (987 genes×200 samples)")

## End(Not run)
```

**Description**

Computes sample size requirements for regression models in which the effect of a primary predictor ( $X_1$ ) is evaluated in the presence of a mediator ( $X_2$ ). The calculation is based on the expected information matrix of the full model and a Wald test under the alternative hypothesis.

**Usage**

```
masize(model, opts, alpha = 0.025, gamma = 0.2)
```

**Arguments**

|       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| model | <p>A character string specifying the model type:</p> <ul style="list-style-type: none"> <li>• Linear models: "linear1", "linear2", "linear3", "linear4"</li> <li>• Logistic models: "logistic1"–"logistic5"</li> <li>• Poisson models: "poisson1"–"poisson9"</li> <li>• Cox models: "cox1"–"cox9"</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| opts  | <p>Named list of model-specific parameters, which may include:</p> <ul style="list-style-type: none"> <li>• b1 regression coefficient for X1.</li> <li>• b1star coefficient for X1 in the reduced model.</li> <li>• b2 regression coefficient for X2.</li> <li>• PTE proportion of total effect explained.</li> <li>• rho correlation between X1 and X2.</li> <li>• sdx1, sdx2 standard deviations of X1 and X2.</li> <li>• f1, f2 prevalences for binary X1 and X2.</li> <li>• sdy residual SD (linear models).</li> <li>• p marginal outcome prevalence (logistic models).</li> <li>• m marginal outcome mean (Poisson models).</li> <li>• f proportion of uncensored observations (Cox models).</li> <li>• fc early censoring proportion (Cox extensions).</li> <li>• ns simulation size for simulation-based methods.</li> <li>• seed random seed.</li> </ul> |
| alpha | One-sided type I error rate.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| gamma | Type-II error rate.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

**Details**

The framework applies to linear, logistic, Poisson, and Cox regression models and supports four predictor–mediator configurations:

- CpCm: continuous X1 and continuous X2.
- BpCm: binary X1 and continuous X2.
- CpBm: continuous X1 and binary X2.
- BpBm: binary X1 and binary X2.

The naming convention is:

- "cc" continuous-continuous.
- "bc" binary-continuous.
- "cb" continuous-binary.
- "bb" binary-binary.
- ".approx" use analytic approximations.
- ".s" use simulation.

The same formulation can also be used to evaluate the effect of a predictor in the presence of a confounder by swapping the roles of X1 and X2.

Throughout, X1 denotes the primary predictor and X2 denotes the mediator. Coefficients b1 and b2 correspond to X1 and X2 respectively, and f1 and f2 denote prevalences when variables are binary. rho denotes the correlation between X1 and X2.

The function is also applicable to the analogous problem of detecting the effect of a primary predictor in the presence of a confounder by treating X2 as the primary predictor and X1 as the confounder.

Let b1\* (b1star) denote the coefficient of X1 in the reduced model excluding X2, and b1 the coefficient in the full model. Mediation can be expressed as attenuation b1\* - b1 or proportion of total effect explained (PTE):

$$PTE = \frac{b_1^* - b_1}{b_1^*}$$

Testing mediation is equivalent (linear models) or approximately equivalent (GLMs and Cox models) to testing the coefficient of X2 in the full model. Sample size is therefore derived from a Wald test on b2 using the expected information matrix.

For binary variables, use  $\sqrt{f(1-f)}$  for the corresponding standard deviation.

Linear models parameters are based on Hsieh et al. (1998) and use a variance inflation factor  $1/(1-\rho^2)$ , where  $\rho$  is the correlation between X1 and X2. Logistic and Cox models generally require simulation to estimate information, while Poisson models admit analytic solutions in most cases.

The quantity PTE and b1star parameter for linear model are provided as follows,

```
getPTE <- function(b1, b2, rho, sdX1=1, sdX2=1)
  b2*sdX2*rho/((b1+b2*sdX2*rho/sdX1)*sdX1)
getb1star <- function(b1, b2, rho, sdX1=1, sdX2=1) b1+b2*sdX2*rho/sdX1
```

For logistic model, the approximate solution due to Hsieh is implemented in the function `logistic.approx`, and can be used for all four cases. Arguments are p, b2, rho, sdX2, alpha, and gamma. Simulating the information matrix of the logistic model provides somewhat more accurate sample size estimates than the Hsieh approximation. The functions for cases CpCm, BpCm, CpBm, and BpBm are respectively `logistic.ccs`, `logistic.bcs`, `logistic.cbs`, and `logistic.bbs`, as for the Poisson and Cox models. Arguments for these functions include p, b1, sdX1 or f1, b2, sdX2 or f2, rho, alpha, gamma, and ns. As in other functions, sdX1, sdX2, alpha, and gamma are set to the defaults listed above. These four functions call the utility functions and `getb0` (to calculate the intercept parameter from the others), which are supplied.

```
getb0 <- function(p, X, b1, b2)
{
  stopifnot(is.numeric(p), length(p) == 1, p > 0, p < 1)
  eta0 <- X[,2] * b1 + X[,3] * b2
  f <- function(b0) mean(plogis(b0+eta0)) - p
  uniroot(f, interval = c(-50,50), tol = 1e-10)$root
}
```

For Poisson model, The function implementing the approximate solution based on the variance inflation factor is `poisson.approx`, and can be used for all four cases. Arguments are EY (the marginal mean of the Poisson outcome), b2, sdX2, rho, alpha and gamma, with sdX2, alpha and gamma set



to the usual defaults; For cases CpCm and BpCm (continuous mediators), the approximate formula is also the analytic solution. For these cases, we supply redundant functions `poisson.cc` and `poisson.bc`, with the same arguments and defaults as for `poisson.approx` (it's the same function). For the two cases with binary mediators, the functions are `poisson.cb` and `poisson.bb`. In addition to `m`, `b2`, `f2`, `rho`, `alpha`, and `gamma`, `b1` and `sdx1` or `f1` must be specified. Defaults are as usual. Functions using simulation for the Poisson model are available: `poisson.ccs`, `poisson.bcs`, `poisson.cbs`, and `poisson.bbs`. As in the logistic case, these require arguments `b1` and `sdx1` or `f1`. For this case, however, the analytic functions are faster, avoid simulation error, and should be used. We include these functions as templates that could be adapted to other joint predictor distributions.

For Cox model, the function implementing the approximate solution, using the variance inflation factor and derived by Schmoor et al. (2000) is `cox.approx`, and can be used for all four cases. Arguments are `b2`, `sdx2`, `rho`, `alpha`, `gamma`, and `f`. The approximation works very well for cases CpCm and BpCm (continuous mediators), but is a bit less accurate for cases CpBm and BpBm (binary mediators). We get some improvement for those cases using the simulation approach. This approach is implemented for all four, as functions `cox.ccs`, `cox.bcs`, `cox.cbs`, and `cox.bbs`. Arguments are `b1`, `sdx1` or `f1`, `b2`, `sdx2` or `f2`, `rho`, `alpha`, `gamma`, `f`, and `ns`, with defaults as described above. Slight variants of these functions, `cox.ccs2`, `cox.bcs2`, `cox.cbs2`, and `cox.bbs2`, make it possible to allow for early censoring of a fraction `fc` of observations; but in our experience this has virtually no effect, even with values of `fc` of 0.5. The default for `fc` is 0.

A summary of the arguments is as follows, noting that additional parameter seed can be supplied for simulation-based method.

| Model                  | Arguments                                                                                                                                                               | Description     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Linear models:         |                                                                                                                                                                         |                 |
| <code>linear1</code>   | <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code> , <code>sd<sub>y</sub></code>                                                                         | linear          |
| <code>linear2</code>   | <code>b1star</code> , <code>PTE</code> , <code>rho</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>y</sub></code>                                                  | lineara         |
| <code>linear3</code>   | <code>b1star</code> , <code>b2</code> , <code>PTE</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>sd<sub>y</sub></code>                    | linearb         |
| <code>linear4</code>   | <code>b1star</code> , <code>b1</code> , <code>b2</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>sd<sub>y</sub></code>                     | linearc         |
| Logistic models:       |                                                                                                                                                                         |                 |
| <code>logistic1</code> | <code>p</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code>                                                                                      | logistic.approx |
| <code>logistic2</code> | <code>p</code> , <code>b1</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                   | logistic.ccs    |
| <code>logistic3</code> | <code>p</code> , <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                                | logistic.bcs    |
| <code>logistic4</code> | <code>p</code> , <code>b1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>sd<sub>x1</sub></code> , <code>ns</code>                                | logistic.cbs    |
| <code>logistic5</code> | <code>p</code> , <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>ns</code>                                             | logistic.bbs    |
| Poisson models:        |                                                                                                                                                                         |                 |
| <code>poisson1</code>  | <code>m</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code>                                                                                      | poisson.approx  |
| <code>poisson2</code>  | <code>m</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code>                                                                                      | poisson.cc      |
| <code>poisson3</code>  | <code>m</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code>                                                                                      | poisson.bc      |
| <code>poisson4</code>  | <code>m</code> , <code>b1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>sd<sub>x1</sub></code>                                                  | poisson.cb      |
| <code>poisson5</code>  | <code>m</code> , <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code>                                                               | poisson.bb      |
| <code>poisson6</code>  | <code>m</code> , <code>b1</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                   | poisson.ccs     |
| <code>poisson7</code>  | <code>m</code> , <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>rho</code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                                | poisson.bcs     |
| <code>poisson8</code>  | <code>m</code> , <code>b1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>sd<sub>x1</sub></code> , <code>ns</code>                                | poisson.cbs     |
| <code>poisson9</code>  | <code>m</code> , <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>ns</code>                                             | poisson.bbs     |
| Cox models:            |                                                                                                                                                                         |                 |
| <code>cox1</code>      | <code>b2</code> , <code>rho</code> , <code>f</code> , <code>sd<sub>x2</sub></code>                                                                                      | cox.approx      |
| <code>cox2</code>      | <code>b1</code> , <code>b2</code> , <code>rho</code> , <code>f</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                   | cox.ccs         |
| <code>cox3</code>      | <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>rho</code> , <code>f</code> , <code>sd<sub>x2</sub></code> , <code>ns</code>                                | cox.bcs         |
| <code>cox4</code>      | <code>b1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>f</code> , <code>sd<sub>x1</sub></code> , <code>ns</code>                                | cox.cbs         |
| <code>cox5</code>      | <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>f2</code> , <code>rho</code> , <code>f</code> , <code>ns</code>                                             | cox.bbs         |
| <code>cox6</code>      | <code>b1</code> , <code>b2</code> , <code>rho</code> , <code>f</code> , <code>fc</code> , <code>sd<sub>x1</sub></code> , <code>sd<sub>x2</sub></code> , <code>ns</code> | cox.ccs2        |
| <code>cox7</code>      | <code>b1</code> , <code>f1</code> , <code>b2</code> , <code>rho</code> , <code>f</code> , <code>fc</code> , <code>sd<sub>x2</sub></code> , <code>ns</code>              | cox.bcs2        |

|      |                                               |          |
|------|-----------------------------------------------|----------|
| cox8 | b1, b2, f2, rho, f, fc, sd <sub>x1</sub> , ns | cox.cbs2 |
| cox9 | b1, f1, b2, f2, rho, f, fc, ns                | cox.bbs2 |

## Value

A list containing:

- desc Description of the selected method.
- n Required sample size.
- d Required number of events (Cox models only).

## References

Hsieh FY, Bloch DA, Larsen MD (1998). "A simple method of sample size calculation for linear and logistic regression." *Stat Med*, **17**(14), 1623-34. doi:[10.1002/\(sici\)10970258\(19980730\)17:14<1623::aid-sim871>3.0.co;2s](https://doi.org/10.1002/(sici)10970258(19980730)17:14<1623::aid-sim871>3.0.co;2s).

Schmoor C, Sauerbrei W, Schumacher M (2000). "Sample size considerations for the evaluation of prognostic factors in survival analysis." *Stat Med*, **19**(4), 441-52. doi:[10.1002/\(sici\)1097-0258\(20000229\)19:4<441::aidsim349>3.0.co;2n](https://doi.org/10.1002/(sici)1097-0258(20000229)19:4<441::aidsim349>3.0.co;2n).

Vittinghoff E, Sen S, McCulloch CE (2009). "Sample size calculations for evaluating mediation." *Stat Med*, **28**(4), 541-57. doi:[10.1002/sim.3491](https://doi.org/10.1002/sim.3491).

## See Also

[ab](#)

## Examples

```
## Not run:
## linear model
# CpCm
opts <- list(b2=0.5, rho=0.3, sdx2=1, sdy=1)
masize("linear1",opts)
# BpBm
opts <- list(b2=0.75, rho=0.3, f2=0.25, sdx2=sqrt(0.25*0.75), sdy=3)
masize("linear1",opts,gamma=0.1)

## logistic model
# CpBm
opts <- list(p=0.25, b2=log(0.5), rho=0.5, sdx2=0.5)
masize("logistic1",opts)
opts <- list(p=0.25, b1=log(1.5), sdx1=1, b2=log(0.5), f2=0.5, rho=0.5, ns=10000,
            seed=1234)
masize("logistic4",opts)
opts <- list(p=0.25, b1=log(1.5), sdx1=1, b2=log(0.5), f2=0.5, rho=0.5, ns=10000,
            seed=1234)
opts <- list(p=0.25, b1=log(1.5), sdx1=4.5, b2=log(0.5), f2=0.5, rho=0.5, ns=50000,
            seed=1234)
masize("logistic4",opts)

## Poisson model
# BpBm
opts <- list(m=0.5, b2=log(1.25), rho=0.3, sdx2=sqrt(0.25*0.75))
```

```

masize("poisson1",opts)
opts <- list(m=0.5, b1=log(1.4), f1=0.25, b2=log(1.25), f2=0.25, rho=0.3)
masize("poisson5",opts)
opts <- c(opts,ns=10000, seed=1234)
masize("poisson9",opts)

## Cox model
# BpBm
opts <- list(b2=log(1.5), rho=0.45, f=0.2, sdx2=sqrt(0.25*0.75))
masize("cox1",opts)
opts <- list(b1=log(2), f1=0.5, b2=log(1.5), f2=0.25, rho=0.45, f=0.2, seed=1234)
masize("cox5",c(opts, ns=10000))
masize("cox5",c(opts, ns=50000))

## End(Not run)

```

MCMCgrm

*Mixed modeling with genetic relationship matrices***Description**

Mixed modeling with genetic relationship matrices

**Usage**

```

MCMCgrm(
  model,
  prior,
  data,
  GRM,
  eps = 0,
  n.thin = 10,
  n.burnin = 3000,
  n.iter = 13000,
  ...
)

```

**Arguments**

|          |                                                                         |
|----------|-------------------------------------------------------------------------|
| model    | statistical model.                                                      |
| prior    | a list of priors for parameters in the model above.                     |
| data     | a data.frame containing outcome and covariates.                         |
| GRM      | a relationship matrix.                                                  |
| eps      | a small number added to the diagonal of the a nonpositive definite GRM. |
| n.thin   | thinning parameter in the MCMC.                                         |
| n.burnin | the number of burn-in's.                                                |
| n.iter   | the number of iterations.                                               |
| ...      | other options as appropriate for MCMCglmm.                              |

## Details

Mixed modeling with genomic relationship matrix. This is appropriate with relationship matrix derived from family structures or unrelated individuals based on whole genome data.

The function was created to address a number of issues involving mixed modelling with family data or population sample with whole genome data. First, the implementation will shed light on the uncertainty involved with polygenic effect in that posterior distributions can be obtained. Second, while the model can be used with the MCMCglmm package there is often issues with the specification of pedigree structures but this is less of a problem with genetic relationship matrices. We can use established algorithms to generate kinship or genomic relationship matrix as input to the MCMCglmm function. Third, it is more intuitive to specify function arguments in line with other packages such as R2OpenBUGS, R2jags or glmmBUGS. In addition, our experiences of tuning the model would help to reset the input and default values.

## Value

The returned value is an object as generated by MCMCglmm.

## Author(s)

Jing Hua Zhao

## References

Hadfield JD (2010). "MCMC Methods for Multi-Response Generalized Linear Mixed Models: The MCMCglmm R Package." *Journal of Statistical Software*, **33**(2), 1 - 22. doi:[10.18637/jss.v033.i02](https://doi.org/10.18637/jss.v033.i02).

## Examples

```
## Not run:
### with kinship

# library(kinship)
# fam <- with(l51,makefamid(id,fid,mid))
# s <-with(l51, makekinship(fam, id, fid, mid))
# K <- as.matrix(s)*2

### with gap

s <- kin.morgan(l51)
K <- with(s,kin.matrix*2)
prior <- list(R=list(V=1, nu=0.002), G=list(G1=list(V=1, nu=0.002)))
m <- MCMCgrm(qt~1,prior,l51,K)
save(m,file="l51.m")
pdf("l51.pdf")
plot(m)
dev.off()

# A real analysis on bats
## data
bianfu.GRM <- read.table("bianfu.GRM.txt", header = TRUE)
bianfu.GRM[1:5,1:6]
Data <- read.table(file = "PHONE.txt", header = TRUE,
                   colClasses=c(rep("factor",3),rep("numeric",7)))

## MCMCgrm
library("MCMCglmm")
```

```

GRM <- as.matrix(bianfu.GRM[,-1])
colnames(GRM) <- rownames(GRM) <- bianfu.GRM[,1]
library(gap)
names(Data)[1] <- "id"
prior <- list(G = list(G1 = list(V = 1, nu = 0.002)), R = list(V = 1, nu = 0.002))
model1.1 <- MCMCgrm(WEIGHTHT ~ 1, prior, Data, GRM, n.burnin=100, n.iter=1000, verbose=FALSE)
## an alternative
names(Data)[1] <- "animal"
N <- nrow(Data)
i <- rep(1:N, rep(N, N))
j <- rep(1:N, N)
s <- Matrix::spMatrix(N, N, i, j, as.vector(GRM))
Ginv <- Matrix::solve(s)
class(Ginv) <- "dgCMatrix"
rownames(Ginv) <- Ginv@Dimnames[[1]] <- with(Data, animal)
model1.2 <- MCMCglmm(WEIGHTHT ~ 1, random= ~ animal, data = Data,
  ginverse=list(animal=Ginv), prior = prior, burnin=100, nitt=1000, verbose=FALSE)
## without missing data
model1.3 <- MCMCglmm(Peak_Freq ~ WEIGHTHT, random = ~ animal,
  data=subset(Data,!is.na(Peak_Freq)&!is.na(WEIGHTHT)),
  ginverse=list(animal=Ginv), prior = prior, burnin=100, nitt=1000, verbose=FALSE)

## End(Not run)

```

METAL\_forestplot

*forest plot as R/meta's forest for METAL outputs*

## Description

forest plot as R/meta's forest for METAL outputs

## Usage

```

METAL_forestplot(
  tbl,
  all,
  rsid,
  flag = "",
  package = "meta",
  method = "REML",
  split = FALSE,
  ...
)

```

## Arguments

|                      |                                                                     |
|----------------------|---------------------------------------------------------------------|
| <code>tbl</code>     | Meta-analysis summary statistics.                                   |
| <code>all</code>     | statistics from all contributing studies.                           |
| <code>rsid</code>    | SNPID-rsid mapping file.                                            |
| <code>flag</code>    | a variable in <code>tbl</code> such as <code>cis/trans</code> type. |
| <code>package</code> | "meta" or "metafor" package.                                        |

|        |                                                              |
|--------|--------------------------------------------------------------|
| method | an explicit flag for fixed/random effects model.             |
| split  | when TRUE, individual prot-MarkerName.pdf will be generated. |
| ...    | Additional arguments to meta::forest or metafor::forest.     |

## Details

This functions takes a meta-data from METAL (tbl) and data from contributing studies (all) for forest plot. It also takes a SNPID-rsid mapping (rsid) as contributing studies often involve discrepancies in rsid so it is appropriate to use SNPID, i.e., chr:pos\_A1\_A2 (A1<=A2).

The study-specific and total sample sizes (N) can be customised from METAL commands. By default, the input triplets each contain a MarkerName variable which is the unique SNP identifier (e.g., chr:pos:a1:a2) and the tbl argument has variables A1 and A2 as produced by METAL while the all argument has EFFECT\_ALLELE and REFERENCE\_ALLELE as with a study variable indicating study name. Another variable common the tbl and all is prot variable as the function was developed in a protein based meta-analysis. As noted above, the documentation example also has variable N. From these all information is in place for generation of a list of forest plots through a batch run.

CUSTOMVARIABLE N  
 LABEL N as N  
 WEIGHTLABEL N

## Value

It will generate a forest plot specified by pdf for direction-adjusted effect sizes.

## Author(s)

Jing Hua Zhao

## References

Schwarzer G (2007). "meta: An R package for meta-analysis." *R News*, **7**, 40-45. [https://cran.r-project.org/doc/Rnews/Rnews\\_2007-3.pdf](https://cran.r-project.org/doc/Rnews/Rnews_2007-3.pdf). Willer CJ, Li Y, Abecasis GR (2010). "METAL: fast and efficient meta-analysis of genomewide association scans." *Bioinformatics*, **26**(17), 2190-1. [doi:10.1093/bioinformatics/btq340](https://doi.org/10.1093/bioinformatics/btq340).

## Examples

```
## Not run:
data(OPG, package="gap.datasets")
meta::settings.meta(method.tau="DL")
METAL_forestplot(OPGtbl,OPGall,OPGrsid,width=8.75,height=5,digits.TE=2,digits.se=2,
  col.diamond="black",col.inside="black",col.square="black")
METAL_forestplot(OPGtbl,OPGall,OPGrsid,package="metafor",method="FE",xlab="Effect",
  showweights=TRUE)

## End(Not run)
```

metap

*Meta-analysis of p-values with heterogeneity and random effects***Description**

Performs GWAS-style meta-analysis by combining p-values across studies. Implements Fisher's method, fixed-effect weighted Stouffer Z, and random-effects Z meta-analysis with heterogeneity statistics.

**Usage**

```
metap(
  data,
  N,
  sided = c("two", "one"),
  verbose = TRUE,
  prefixp = "p",
  prefixn = "n",
  prefixbeta = NULL,
  prefixdir = NULL
)
```

**Arguments**

|            |                                                        |
|------------|--------------------------------------------------------|
| data       | data.frame containing study results.                   |
| N          | number of studies.                                     |
| sided      | "two" (default) or "one" for one-sided p-values.       |
| verbose    | logical; print summary output.                         |
| prefixp    | prefix for p-value columns (default "p").              |
| prefixn    | prefix for sample-size columns (default "n").          |
| prefixbeta | optional prefix for beta columns (used for direction). |
| prefixdir  | optional prefix for sign columns (+1/-1).              |

**Details**

This function implements the meta-analysis approach used in the Genetic Investigation of ANThropometric Traits (GIANT) consortium.

Missing studies are automatically excluded per row, so the number of contributing studies may vary across variants.

**Fisher's method:**

$$X^2 = -2 \sum_{i=1}^k \log(p_i) \sim \chi_{2k}^2$$

**Fixed-effect weighted Stouffer Z:**

Convert p-values to Z:

$$z_i = \Phi^{-1}(1 - p_i/2)$$

Sample-size weights:

$$w_i = \sqrt{n_i}$$

$$Z_{FE} = \frac{\sum w_i z_i}{\sqrt{\sum w_i^2}}$$

**Heterogeneity:**

$$Q = \sum w_i (z_i - \bar{z})^2$$

$$I^2 = \max(0, (Q - (k - 1))/Q)$$

**Random-effects Z meta-analysis:**

DerSimonian–Laird variance:

$$\tau^2 = \max\left(0, \frac{Q - (k - 1)}{\sum w_i - \sum w_i^2 / \sum w_i}\right)$$

Random-effects weights:

$$w_i^* = 1/(1/w_i + \tau^2)$$

$$Z_{RE} = \frac{\sum w_i^* z_i}{\sqrt{\sum w_i^*}}$$

## Value

data.frame with

- fisher\_p Fisher combined p-value
- stouffer\_FE Fixed-effect Z meta p-value
- stouffer\_RE Random-effects Z meta p-value
- Q Cochran heterogeneity statistic
- I2 Proportion heterogeneity
- tau2 Between-study variance

## Author(s)

ChatGPT (Jing Hua Zhao)

## Examples

```
## Not run:
## -----
## Classic GIANT consortium example (historical demo)
## Speliotes, Elizabeth K., M.D. [ESPELIOTES@PARTNERS.ORG]
## 22-2-2008 MRC-Epid JHZ
## -----

s <- data.frame(
  p1 = 0.1^rep(8:2, each = 7),
  n1 = rep(32000, 49),
  p2 = 0.1^rep(8:2, times = 7),
  n2 = rep(8000, 49)
)
```



```

res <- metap(s, 2)
head(res)

## Visual comparison equal vs unequal N (original demo)

np <- 7
p <- 0.1^((np + 1):2)
z <- qnorm(1 - p/2)
n <- c(32000, 8000)

grid <- expand.grid(i = seq_len(np), j = seq_len(np))
za <- z[grid$i]; zb <- z[grid$j]

metaz_equalN <- (sqrt(n[1])*za + sqrt(n[1])*zb)/sqrt(n[1]+n[1])
metaz_unequalN <- (sqrt(n[1])*za + sqrt(n[2])*zb)/sqrt(n[1]+n[2])

q <- -log10(sort(p, decreasing=TRUE))
t1 <- matrix(-log10(sort(2*pnorm(-abs(metaz_equalN))), decreasing=TRUE), np, np)
t2 <- matrix(-log10(sort(2*pnorm(-abs(metaz_unequalN))), decreasing=TRUE), np, np)

par(mfrow=c(1,2), bg="white", mar=c(4.2,3.8,0.2,0.2))
persp(q,q,t1, main="Equal sample sizes")
persp(q,q,t2, main="Unequal sample sizes")

## -----
## New example: missing studies + heterogeneity
## -----

set.seed(1)
dat <- data.frame(
  p1 = runif(100,1e-6,0.05),
  n1 = sample(5000:20000,100,TRUE),
  p2 = runif(100,1e-6,0.05),
  n2 = sample(5000:20000,100,TRUE)
)

## simulate missing studies
dat$p2[sample(1:100,20)] <- NA

res <- metap(dat,2)
summary(res$I2)

## End(Not run)

```

## Description

Performs inverse-variance weighted fixed- and random-effects meta-analysis across multiple studies supplied in wide format.

## Usage

```
metareg(data, N, verbose = FALSE, prefixb = "b", prefixse = "se")
```

**Arguments**

|          |                                                                                             |
|----------|---------------------------------------------------------------------------------------------|
| data     | A data frame containing regression coefficients and standard errors in wide format.         |
| N        | Integer. Number of studies included in the meta-analysis.                                   |
| verbose  | Logical. If TRUE, prints a completion message.                                              |
| prefixb  | Character. Prefix for regression coefficients. Default is "b" (columns b1, b2, . . . , bN). |
| prefixse | Character. Prefix for standard errors. Default is "se" (columns se1, se2, . . . , seN).     |

**Details**

The function is designed for high-throughput settings (e.g. GWAS), where each row represents an independent meta-analysis.

The function accepts wide-format input with estimates  $b_1, \dots, b_N$  and standard errors  $se_1, \dots, se_N$ . Missing values are automatically ignored on a per-row basis.

**Fixed effects model:**

For  $k$  studies, inverse-variance weights are defined as

$$w_i = 1/se_i^2$$

The pooled estimate is

$$\beta_f = \frac{\sum_i b_i w_i}{\sum_i w_i}$$

with standard error

$$se_f = \sqrt{1/\sum_i w_i}$$

and test statistic

$$z_f = \beta_f/se_f$$

with p-value

$$p_f = 2\Phi(-|z_f|)$$

.

**Random effects model (DerSimonian–Laird):**

Cochran's Q statistic:

$$Q = \sum_i w_i (b_i - \beta_f)^2$$

Between-study variance:

$$\tau^2 = \max\left(0, \frac{Q - (k - 1)}{\sum w_i - \sum w_i^2 / \sum w_i}\right)$$

Corrected weights:

$$w_i^* = 1/(1/w_i + \tau^2)$$

Random-effects pooled estimate:

$$\beta_r = \frac{\sum_i b_i w_i^*}{\sum_i w_i^*}$$

with standard error

$$se_r = \sqrt{1 / \sum_i w_i^*}$$

and p-value

$$p_r = 2 \Phi(-|z_r|)$$

Heterogeneity p-value:

$$p_{heter} = P(\chi_{k-1}^2 > Q)$$

The heterogeneity statistic is reported as

$$I^2 = \max(0, (Q - (k - 1)) / Q)$$

## Value

A data frame with one row per meta-analysis containing:

- beta\_f Fixed-effects pooled estimate
- se\_f Standard error of fixed-effects estimate
- z\_f Z statistic (fixed effects)
- p\_f P value (fixed effects)
- beta\_r Random-effects pooled estimate
- se\_r Standard error of random-effects estimate
- z\_r Z statistic (random effects)
- p\_r P value (random effects)
- p\_heter Cochran's Q heterogeneity test p-value
- i2  $I^2$  heterogeneity statistic
- k Number of studies contributing to each row
- eps Smallest double-precision number used for stability

## Author(s)

ChatGPT

## References

Higgins JP, Thompson SG, Deeks JJ, Altman DG (2003). "Measuring inconsistency in meta-analyses." *BMJ*, **327**(7414), 557-60. doi:10.1136/bmj.327.7414.557.

## Examples

```
## Not run:
df <- data.frame(
  b1 = 1, se1 = 2,
  b2 = 2, se2 = 6,
  b3 = 3, se3 = 8
)
metareg(df, 3)

df2 <- data.frame(
  b1 = c(1,2), se1 = c(2,4),
  b2 = c(2,3), se2 = c(4,6),
  b3 = c(3,4), se3 = c(6,8)
)
metareg(df2, 3)

## End(Not run)
```

---

mht.control

*Controls for Manhattan plot*


---

## Description

Parameter specification helper for [mhtplot\(\)](#). This function creates a list of graphical and behavioural settings used when generating Manhattan plots.

## Usage

```
mht.control(
  type = "p",
  usepos = FALSE,
  logscale = TRUE,
  base = 10,
  cutoffs = NULL,
  colors = NULL,
  labels = NULL,
  gap = NULL,
  cex = 0.4,
  lab.cex = 1,
  axis.cex = 1.2,
  axis.lwd = 1.2,
  axis.tck = -0.02,
  yline = 3,
  xline = 3,
  verbose = FALSE
)
```

## Arguments

|        |                                                                          |
|--------|--------------------------------------------------------------------------|
| type   | Character. Either "p" (points) or "l" (lines).                           |
| usepos | Logical. Use real chromosomal positions instead of ordinal marker order. |

|          |                                                                                                                      |
|----------|----------------------------------------------------------------------------------------------------------------------|
| logscale | Logical. If TRUE, values are transformed using $-\log(\text{base})(\text{value})$ before plotting.                   |
| base     | Numeric. Base of logarithm used when logscale = TRUE.                                                                |
| cutoffs  | Numeric vector of horizontal reference lines to draw.                                                                |
| colors   | Vector of chromosome colours. Recycled as needed.                                                                    |
| labels   | Optional chromosome labels for the x-axis.                                                                           |
| gap      | Numeric. Gap inserted between chromosomes on the x-axis.                                                             |
| cex      | Numeric. Scaling factor for plotted points.                                                                          |
| lab.cex  | Numeric. Scaling factor for chromosome labels on the x-axis.                                                         |
| axis.cex | Numeric. Scaling factor for axis tick labels. Increase when exporting high-resolution figures.                       |
| axis.lwd | Numeric. Line width for axes and tick marks.                                                                         |
| axis.tck | Numeric. Length and direction of tick marks. Negative values draw ticks outward (recommended for publication plots). |
| yline    | Numeric. Margin line for the y-axis label.                                                                           |
| xline    | Numeric. Margin line for the x-axis label.                                                                           |
| verbose  | Logical. Print plotting progress messages.                                                                           |

**Value**

a named list of control parameters for `mhtplot()`.

**See Also**

`mhtplot()`, `hmht.control()`

**Examples**

```
mht.control()
```

---

mhtplot

*Manhattan plot*


---

**Description**

Draw a Manhattan plot for genome-wide association studies (GWAS) or any genome-indexed numeric score.

**Usage**

```
mhtplot(data, control = mht.control(), hcontrol = hmht.control(), ...)
```

## Arguments

|          |                                                                                                                                                                                                                                                                                           |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data     | A matrix or data frame with at least three columns. The first three columns are interpreted as: <ol style="list-style-type: none"> <li>1. chromosome</li> <li>2. position</li> <li>3. value (typically p-value) Column names are ignored. Factors are automatically converted.</li> </ol> |
| control  | A list produced by <code>mht.control()</code> controlling plot behaviour.                                                                                                                                                                                                                 |
| hcontrol | A list produced by <code>hmht.control()</code> defining highlighted markers or regions and labels.                                                                                                                                                                                        |
| ...      | Additional graphical arguments passed to <code>graphics::plot()</code> (e.g. <code>pch</code> , <code>bg</code> , <code>xlab</code> , <code>ylab</code> , <code>ylim</code> ). If <code>mar</code> is supplied, the default margins are not modified.                                     |

## Details

Produces a Manhattan plot in which genomic markers are arranged along the x-axis by chromosome and position.

By default the y-axis shows  $-\log_{10}(p)$ , the standard GWAS significance scale. Set `logscale = FALSE` in `mht.control()` to plot raw values instead.

Rows containing missing values are removed automatically before plotting.

### Chromosome handling:

Chromosomes may be numeric or character. The prefix "chr" is removed automatically. Chromosomes are ordered numerically first, followed by character chromosomes (e.g. X, Y, MT).

### X-axis spacing:

- `usepos = FALSE` (default): markers are spaced evenly within chromosomes.
- `usepos = TRUE`: real chromosomal positions are used.
- `gap` inserts spacing between chromosomes when using real positions.

### Colours:

Chromosome colours alternate by default. Custom colours can be supplied via `colors` in `mht.control()`; colours are recycled as needed.

### Axis tuning:

Axis appearance can be controlled using:

- `axis.cex` — tick label size
- `axis.lwd` — axis and tick thickness
- `axis.tck` — tick length and direction

These settings are particularly useful when exporting high-resolution figures.

### Horizontal thresholds:

Horizontal reference lines can be drawn using the `cutoffs` parameter.

### Highlighting regions:

Highlighted regions are defined using `hmht.control()`. Matching markers are recoloured, labelled, and optionally boxed. Matching is performed by chromosome and position range.

**Value**

invisibly returns NULL.

**Mathematical background**

A Manhattan plot visualises genome-wide association statistics by mapping genetic markers to a two-dimensional coordinate system.

**Y-axis transformation**

P-values are transformed to improve visibility of small values:

$$y = -\log_{10}(p)$$

Small p-values therefore appear as large positive values and form the characteristic peaks used to identify strong associations.

**Genome linearisation**

Markers originate from multiple chromosomes and must be mapped onto a single continuous axis. For chromosome  $c$ :

$$x_i = pos_i + offset_c$$

where the chromosome offset is

$$offset_c = \sum_{k < c} \max(pos_k)$$

This produces a continuous genome coordinate system in which chromosomes appear sequentially without overlap.

**Chromosome label placement**

Chromosome labels are positioned at the midpoint of each chromosome:

$$mid_c = (\min(x_c) + \max(x_c))/2$$

This centres labels regardless of chromosome length.

**Plot limits**

The plotting region is defined using axis tick locations so that ticks and plotting limits coincide exactly.

Conceptually, a Manhattan plot is defined by two transformations:

$$x = \text{linearised genome coordinate}$$

$$y = -\log_{10}(p)$$

All other elements (colouring, thresholds, highlighting) are visual annotations applied to these transformed coordinates.

**Author(s)**

Jing Hua Zhao

**See Also**

`mht.control()`, `hmht.control()`

**Examples**

```
## -----
## 1. Minimal example
## -----
test <- matrix(c(
  1,1,4,
  1,1,6,
  1,10,3,
  2,1,5,
  2,2,6,
  2,4,8),
  byrow=TRUE, ncol=3)
mhtplot(test)
## Raw values instead of -log10
mhtplot(test, mht.control(logscale = FALSE))

## -----
## 2. Simulated GWAS dataset
## -----
set.seed(1)
affy <- c(40220,41400,33801,32334,32056,31470,25835,27457,22864,
          28501,26273,24954,19188,15721,14356,15309,11281,14881,
          6399,12400,7125,6207)
n.markers <- sum(affy)
n.chr <- length(affy)
gwas <- data.frame(
  chr = rep(1:n.chr, affy),
  pos = 1:n.markers,
  p   = runif(n.markers)
)
mhtplot(gwas)

## -----
## 3. Publication-style figure
## -----
ops <- mht.control(
  axis.cex = 1.4,
  axis.lwd = 2,
  axis.tck = -0.03
)
mhtplot(gwas, control = ops, pch = 19)

## -----
## 4. Real positions + chromosome gaps
## -----
ops2 <- mht.control(usepos = TRUE, gap = 10000)
mhtplot(gwas, control = ops2, pch = 19)

## -----
## 5. Genome-wide significance thresholds
## -----
```



```

ops3 <- mht.control(cutoffs = c(5, 7.3))
mhtplot(gwas, control = ops3, pch = 19)

## -----
## 6. Highlight selected genes
## -----
hdata <- data.frame(
  chr = c(1,3,5),
  pos = c(10000,50000,90000),
  p = c(1e-8,1e-7,1e-9),
  gene = c("GENE1","GENE2","GENE3")
)
hops <- hmht.control(
  data = hdata,
  colors = "red",
  boxed = TRUE
)
mhtplot(gwas, control = ops, hcontrol = hops, pch = 19)
## A real study (data from gap.datasets package)
if (requireNamespace("gap.datasets", quietly = TRUE)) {
  data("mhtdata", package = "gap.datasets")

  data <- with(mhtdata, cbind(chr, pos, p))
  glist <- c("IRS1","SPRY2","FTO","GRIK3","SNED1","HTR1A","MARCH3",
    "WISP3","PPP1R3B","RP1L1","FDFT1","SLC39A14","GFRA1","MC4R")
  hdata <- subset(mhtdata, gene %in% glist)[c("chr","pos","p","gene")]

  ops <- mht.control(colors = rep(c("lightgray","gray"),11),
    labels = paste("chr",1:22,sep=""),
    yline = 1.5, xline = 3)
  hops <- hmht.control(data = hdata, colors = "red", boxed = TRUE)

  mhtplot(data, ops, hops, pch = 19)
  title("Manhattan plot with genes highlighted")
}

## -----
## 7. Export high-resolution PNG
## -----
f <- tempfile(fileext = ".png")
png(f, height = 3600, width = 6000, res = 600)
opar <- par()
par(cex = 0.7)
mhtplot(gwas, control = ops, pch = 19)
par(opar)
dev.off()

## -----
## 8. Miamiplot (see also vignette)
## -----
if (requireNamespace("gap.datasets", quietly = TRUE)) {
  data("mhtdata", package = "gap.datasets")
  mhtdata <- within(mhtdata,{pr=p})
  miamiplot(mhtdata,chr="chr",bp="pos",p="p",pr="pr",snp="rsn")
}

```

---

|               |                                 |
|---------------|---------------------------------|
| mhtplot.trunc | <i>Truncated Manhattan plot</i> |
|---------------|---------------------------------|

---

## Description

Truncated Manhattan plot

## Usage

```
mhtplot.trunc(
  x,
  chr = "CHR",
  bp = "BP",
  p = NULL,
  log10p = NULL,
  z = NULL,
  snp = "SNP",
  col = c("gray10", "gray60"),
  chrlabs = NULL,
  suggestiveline = -log10(1e-05),
  genomewideline = -log10(5e-08),
  highlight = NULL,
  annotatelog10P = NULL,
  annotateTop = FALSE,
  cex.mtext = 1.5,
  cex.text = 0.7,
  mtext.line = 2,
  y.ax.space = 5,
  y.brk1 = NULL,
  y.brk2 = NULL,
  trunc.yaxis = TRUE,
  cex.axis = 1.2,
  delta = 0.05,
  ...
)
```

## Arguments

|                |                                          |
|----------------|------------------------------------------|
| x              | A data.frame.                            |
| chr            | Chromosome column name.                  |
| bp             | Base-pair position column name.          |
| p              | P-value column name.                     |
| log10p         | Column containing -log10(P).             |
| z              | Z-statistic column name.                 |
| snp            | SNP/variant identifier column name.      |
| col            | Point colours alternating by chromosome. |
| chrlabs        | Optional chromosome labels.              |
| suggestiveline | Horizontal suggestive significance line. |

|                |                                                     |
|----------------|-----------------------------------------------------|
| genomewideline | Horizontal genome-wide significance line.           |
| highlight      | SNPs to highlight.                                  |
| annotatelog10P | Threshold for annotation.                           |
| annotateTop    | Annotate only top SNP per chromosome.               |
| cex.mtext      | Axis title size.                                    |
| cex.text       | SNP label size.                                     |
| mtext.line     | Axis title line offset.                             |
| y.ax.space     | Y-axis tick spacing.                                |
| y.brk1         | Lower truncation breakpoint.                        |
| y.brk2         | Upper truncation breakpoint.                        |
| trunc.yaxis    | Enable truncated y-axis.                            |
| cex.axis       | Axis tick label size.                               |
| delta          | Fractional window around highlighted SNPs.          |
| ...            | Additional graphical parameters passed to points(). |

## Details

Draws a Manhattan plot with optional y-axis truncation for extremely significant associations commonly observed in large-scale GWAS or protein GWAS analyses.

## Value

Invisibly returns the processed plotting data.

Example FTO locus dataset and truncated Manhattan plot

This example demonstrates the use of `mhtplot.trunc()` on a single-chromosome FTO locus dataset with and without y-axis truncation.

## Examples

```
txt <- "
CHR POS SNP Z log10P
16 53804965 rs10852521 -39.75000 344.8039
16 53805207 rs11075985 43.88235 419.8925
16 53819877 rs11075989 43.94118 421.0149
16 53819893 rs11075990 -43.94118 421.0149
16 53809247 rs1121980 43.76471 417.6523
16 53845487 rs11642841 38.52941 324.0426
16 53842908 rs12149832 42.23529 389.0756
16 53800954 rs1421085 -45.76471 456.5538
16 53803574 rs1558902 45.88235 458.8962
16 53813367 rs17817449 -46.87500 478.8994
16 53828066 rs17817964 44.29412 427.7808
16 53804340 rs1861866 39.81250 345.8844
16 53818460 rs3751812 44.41176 430.0480
16 53822651 rs7185735 -43.94118 421.0149
16 53810686 rs7193144 -44.29412 427.7808
16 53821862 rs7201850 42.35294 391.2377
16 53821615 rs7202116 -43.94118 421.0149
16 53813450 rs8043757 -42.22222 388.8357
16 53798523 rs8047395 37.76471 311.3650
```

```

16 53816275 rs8050136 46.75000 476.3569
16 53816752 rs8051591 -44.00000 422.1388
16 53803156 rs8055197 39.81250 345.8844
16 53812614 rs8057044 39.75000 344.8039
16 53806280 rs9922047 -39.62500 342.6480
16 53831771 rs9922619 43.37500 410.2743
16 53831146 rs9922708 43.25000 407.9218
16 53801549 rs9923147 43.82353 418.7716
16 53819198 rs9923233 43.94118 421.0149
16 53801985 rs9923544 43.82353 418.7716
16 53820503 rs9926289 40.66667 360.8208
16 53799905 rs9928094 -43.88235 419.8925
16 53799977 rs9930333 -44.00000 422.1388
16 53830452 rs9930501 -40.94118 365.6883
16 53830465 rs9930506 -43.31250 409.0972
16 53827179 rs9931494 -42.94118 402.1387
16 53830491 rs9932754 -41.00000 366.7356
16 53816838 rs9935401 46.81250 477.6273
16 53819169 rs9936385 -44.23529 426.6494
16 53799507 rs9937053 43.94118 421.0149
16 53820527 rs9939609 44.05882 423.2642
16 53800568 rs9939973 43.88235 419.8925
16 53800754 rs9940128 43.82353 418.7716
16 53800629 rs9940646 -43.82353 418.7716
16 53825488 rs9941349 42.58824 395.5801
"

FTO <- read.table(text = txt, header = TRUE, stringsAsFactors = FALSE)
par(mar = c(5, 6, 2, 1))
mhtplot.trunc(
  x = FTO,
  chr = "CHR",
  bp = "POS",
  log10p = "log10P",
  snp = "SNP",
  cex = 1.0,
  cex.axis = 1.2,
  cex.mtext = 1.5,
  col = "navy"
)
title("FTO locus without truncation")

par(mar = c(5, 6, 2, 1))
mhtplot.trunc(
  x = FTO,
  chr = "CHR",
  bp = "POS",
  log10p = "log10P",
  snp = "SNP",
  trunc.yaxis = TRUE,
  y.brk1 = 200,
  y.brk2 = 350,
  y.ax.space = 50,
  genomewideline = -log10(5e-8),
  suggestiveline = NULL,
  highlight = c("rs1421085", "rs1558902", "rs17817449",
    "rs8050136", "rs9939609"),
  annotatelog10P = 420,

```

```

    annotateTop = FALSE,
    cex = 1.2,
    cex.text = 0.9,
    cex.axis = 1.2,
    cex.mtext = 1.5,
    col = "steelblue4"
  )
  title("FTO locus with truncated y-axis")

```

mhtplot2

*Manhattan plot with annotations*

## Description

Manhattan plot with annotations

## Usage

```
mhtplot2(data, control = mht.control(), hcontrol = hmht.control(), ...)
```

## Arguments

|          |                                                                                                                                                                    |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data     | a data frame with three columns representing chromosome, position and p values.                                                                                    |
| control  | A list produced by <code>mht.control()</code> controlling plot behaviour.                                                                                          |
| hcontrol | A list produced by <code>hmht.control()</code> defining highlighted markers or regions and labels.                                                                 |
| ...      | Additional graphical arguments passed to <code>graphics::plot()</code> (e.g. pch, bg, xlab, ylab, ylim). If mar is supplied, the default margins are not modified. |

## Details

To generate Manhattan plot with annotations. The function is generic and for instance could be used for genomewide p values or any random variable that is uniformly distributed. By default, a log10-transformation is applied. Note that with real chromosomal positions, it is also appropriate to plot and some but not all chromosomes.

It is possible to specify options such as xlab, ylim and font family when the plot is requested for data in other context. The example uses only chromosomes 14 and 20 of (den Hoed et al. 2013).

## Value

shown on or saved to the appropriate device.

## Author(s)

Jing Hua Zhao

## References

den Hoed M, Eijgelsheim M, Esko T, Brundel BJ, Peal DS, Evans DM, Nolte IM, Segrè AV, Holm H, Handsaker RE, Westra HJ, Johnson T, Isaacs A, Yang J, Lundby A, Zhao JH, Kim YJ, Go MJ, Almgren P, Bochud M, Boucher G, Cornelis MC, Gudbjartsson D, Hadley D, van der Harst P, Hayward C, den Heijer M, Igl W, Jackson AU, Kutalik Z, Luan J, Kemp JP, Kristiansson K, Ladenvall C, Lorentzon M, Montasser ME, Njajou OT, O'Reilly PF, Padmanabhan S, St Pourcain B, Rankinen T, Salo P, Tanaka T, Timpson NJ, Vitart V, Waite L, Wheeler W, Zhang W, Draisma HH, Feitosa MF, Kerr KF, Lind PA, Mihailov E, Onland-Moret NC, Song C, Weedon MN, Xie W, Yengo L, Absher D, Albert CM, Alonso A, Arking DE, de Bakker PI, Balkau B, Barlassina C, Benaglio P, Bis JC, Bouatia-Naji N, Brage S, Chanock SJ, Chines PS, Chung M, Darbar D, Dina C, Dörr M, Elliott P, Felix SB, Fischer K, Fuchsberger C, de Geus EJ, Goyette P, Gudnason V, Harris TB, Hartikainen AL, Havulinna AS, Heckbert SR, Hicks AA, Hofman A, Holewijn S, Hoogstra-Berends F, Hottenga JJ, Jensen MK, Johansson A, Junttila J, Kääb S, Kanon B, Ketkar S, Khaw KT, Knowles JW, Kooner AS, others (2013). "Identification of heart rate-associated loci and their effects on cardiac conduction and rhythm disorders." *Nat Genet*, **45**(6), 621-31. doi:10.1038/ng.2610.

## Examples

```
## Not run:
mdata <- within(hr1420,{
  c1<-colour==1
  c2<-colour==2
  c3<-colour==3
  colour[c1] <- 62
  colour[c2] <- 73
  colour[c3] <- 552
})
mdata <- mdata[,c("CHR", "POS", "P", "gene", "colour")]
ops <- mht.control(colors=rep(c("lightgray", "gray"),11),yline=1.5,xline=2)
hops <- hmht.control(data=subset(mdata,!is.na(gene)),boxed=TRUE)
v <- "Verdana"
ifelse(Sys.info()['sysname']=="Windows", windowsFonts(ffamily=windowsFont(v)),
  ffamily <- v)
tiff("mh.tiff", width=.03937*189, height=.03937*189/2, units="in", res=1200,
  compress="lzw")
par(las=2, xpd=TRUE, cex.axis=1.8, cex=0.4)
mhplot2(with(mdata,cbind(CHR,POS,P,colour)),ops,hops,pch=19,
  ylab=expression(paste(plain("-"),log[10],plain("p-value")),sep=" "),
  family="ffamily")
axis(2,pos=2,at=seq(0,25,5),family="ffamily",cex=0.5,cex.axis=1.1)
dev.off()

# To exemplify the use of chr, pos and p without gene annotation
# in response to query from Vallejo, Roger <Roger.Vallejo@ARS.USDA.GOV>
opar <- par()
par(cex=0.4)
ops <- mht.control(colors=rep(c("lightgray", "lightblue"),11),yline=2.5,xline=2)
mhplot2(data.frame(mhtdata[,c("chr", "pos", "p")],gene=NA,color=NA),ops,xlab="",ylab="")
axis(2,at=1:16)
title("data in mhplot used by mhplot2")
par(opar)

## End(Not run)
```

---

**mia***Multiple imputation analysis for hap*

---

**Description**

Multiple imputation analysis for hap

**Usage**

```
mia(  
  hapfile = "hap.out",  
  assfile = "assign.out",  
  miafile = "mia.out",  
  so = 0,  
  ns = 0,  
  mi = 0,  
  allsnps = 0,  
  sas = 0  
)
```

**Arguments**

|         |                                                 |
|---------|-------------------------------------------------|
| hapfile | hap haplotype output file name.                 |
| assfile | hap assignment output file name.                |
| miafile | mia output file name.                           |
| so      | to generate results according to subject order. |
| ns      | do not sort in subject order.                   |
| mi      | number of multiple imputations used in hap.     |
| allsnps | all loci are SNPs.                              |
| sas     | produce SAS data step program.                  |

**Details**

This command reads outputs from hap session that uses multiple imputations, i.e. -mi# option. To simplify matters it assumes -ss option is specified together with -mi option there.

This is a very naive version of MIANALYZE, but can produce results for PROC MIANALYZE of SAS.

It simply extracts outputs from hap.

**Value**

The returned value is a list.

**Note**

adapted from hap, in fact cline.c and cline.h are not used. keywords utilities

## References

Zhao JH and W Qian (2003) Association analysis of unrelated individuals using polymorphic genetic markers. RSS 2003, Hassalt, Belgium

Clayton DG (2001) SNPHAP. <https://github.com/chr1swallace/snphap>.

## See Also

[hap](#)

## Examples

```
## Not run:
# 4 SNP example, to generate hap.out and assign.out alone
data(fsnps)
hap(id=fsnps[,1],data=fsnps[,3:10],nloci=4)

# to generate results of imputations
control <- hap.control(ss=1,mi=5)
hap(id=fsnps[,1],data=fsnps[,3:10],nloci=4,control=control)

# to extract information from the second run above
mia(so=1,ns=1,mi=5)
file.show("mia.out")

## commands to check out where the output files are as follows:
## Windows
# system("command.com")
## Unix
# system("csh")

## End(Not run)
```

---

miamiplot

*Miami plot*


---

## Description

Miami plot

## Usage

```
miamiplot(
  x,
  chr = "CHR",
  bp = "BP",
  p = "P",
  pr = "PR",
  snp = "SNP",
  col = c("midnightblue", "chartreuse4"),
  col2 = c("royalblue1", "seagreen1"),
  ymax = NULL,
```



```

    highlight = NULL,
    highlight.add = NULL,
    pch = 19,
    cex = 0.75,
    cex.lab = 1,
    xlab = "Chromosome",
    ylab = "-log10(P) [y>0]; log10(P) [y<0]",
    lcols = c("red", "black"),
    lwds = c(5, 2),
    ltys = c(1, 2),
    main = "",
    ...
)

```

### Arguments

|               |                            |
|---------------|----------------------------|
| x             | Input data.                |
| chr           | Chromosome.                |
| bp            | Position.                  |
| p             | P value.                   |
| pr            | P value of the other GWAS. |
| snp           | Marker.                    |
| col           | Colors.                    |
| col2          | Colors.                    |
| ymax          | Max y.                     |
| highlight     | Highlight flag.            |
| highlight.add | Highlight meta-data.       |
| pch           | Symbol.                    |
| cex           | cex.                       |
| cex.lab       | cex for labels.            |
| xlab          | Label for x-axis.          |
| ylab          | Label for y-axis.          |
| lcols         | Colors.                    |
| lwds          | lwd.                       |
| ltys          | lty.                       |
| main          | Main title.                |
| ...           | Additional options.        |

### Details

The function allows for contrast of genomewide P values from two GWASs. It is conceptually simpler than at the first sight since it involves only one set of chromosomal positions.

### Value

None.

## Examples

```
## Not run:
mhtdata <- within(mhtdata,{pr=p})
miamiplot(mhtdata,chr="chr",bp="pos",p="p",pr="pr",snp="rsn")

## End(Not run)
```

---

miamiplot2

*Miami Plot*


---

## Description

Miami Plot

## Usage

```
miamiplot2(
  gwas1,
  gwas2,
  name1 = "GWAS 1",
  name2 = "GWAS 2",
  chr1 = "chr",
  chr2 = "chr",
  pos1 = "pos",
  pos2 = "pos",
  p1 = "p",
  p2 = "p",
  z1 = NULL,
  z2 = NULL,
  sug = 1e-05,
  sig = 5e-08,
  pcutoff = 0.1,
  topcols = c("green3", "darkgreen"),
  botcols = c("royalblue1", "navy"),
  yAxisInterval = 5
)
```

## Arguments

|       |                                                                                            |
|-------|--------------------------------------------------------------------------------------------|
| gwas1 | The first of two GWAS datasets to plot, in the upper region.                               |
| gwas2 | The second of two GWAS datasets to plot, in the lower region.                              |
| name1 | The name of the first dataset, plotted above the upper plot region. Defaults to "GWAS 1".  |
| name2 | The name of the second dataset, plotted below the lower plot region. Defaults to "GWAS 2". |
| chr1  | The name of the column containing chromosome number in gwas1. Defaults to "chr".           |
| chr2  | The name of the column containing chromosome number in gwas2. Defaults to "chr".           |

|               |                                                                                                                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pos1          | The name of the column containing SNP position in gwas1. Defaults to "pos".                                                                                                                                             |
| pos2          | The name of the column containing SNP position in gwas2. Defaults to "pos".                                                                                                                                             |
| p1            | The name of the column containing p-values in gwas1. Defaults to "p".                                                                                                                                                   |
| p2            | The name of the column containing p-values in gwas2. Defaults to "p".                                                                                                                                                   |
| z1            | The name of the column containing z-values in gwas1. Defaults to "NULL".                                                                                                                                                |
| z2            | The name of the column containing z-values in gwas2. Defaults to "NULL".                                                                                                                                                |
| sug           | The threshold for suggestive significance, plotted as a light grey dashed line.                                                                                                                                         |
| sig           | The threshold for genome-wide significance, plotted as a dark grey dashed line.                                                                                                                                         |
| pcutoff       | The p-value threshold below which SNPs will be ignored. Defaults to 0.1. It is not recommended to set this higher as it will narrow the central gap between the two plot region where the chromosome number is plotted. |
| topcols       | A vector of two colours to plot alternating chromosomes in for the upper plot. Defaults to green3 and darkgreen.                                                                                                        |
| botcols       | A vector of two colours to plot alternating chromosomes in for the lower plot. Defaults to royalblue1 and navy.                                                                                                         |
| yAxisInterval | The interval between tick marks on the y-axis. Defaults to 5, 2 may be more suitable for plots with larger minimum p-values.                                                                                            |

## Details

Creates a Miami plot to compare results from two genome-wide association analyses.

## Value

In addition to creating a Miami plot, the function returns a data frame containing x coordinates for chromosome start positions (required for [labelManhattan](#))

## Note

Extended to handle extreme P values.

## Author(s)

Jonathan Marten

## Examples

```
## Not run:
# miamipLOT2(gwas1, gwas2)
# chrmaxpos <- miamipLOT2(gwas1, gwas2)
gwas <- within(mhtdata[c("chr", "pos", "p")], {z=qnrm(p/2)})
chrmaxpos <- miamipLOT2(gwas, gwas, name1="Batch 2", name2="Batch 1", z1="z", z2="z")
labelManhattan(chr=c(2,16), pos=c(226814165, 52373776), name=c("AnonymousGene", "FT0"),
               gwas, gwasZLab="z", chrmaxpos=chrmaxpos)

## End(Not run)
```

---

|    |                                                                                    |
|----|------------------------------------------------------------------------------------|
| mr | <i>Mendelian Randomization wrapper (IVW, Egger, Weighted Median, Penalised WM)</i> |
|----|------------------------------------------------------------------------------------|

---

## Description

Mendelian Randomization wrapper (IVW, Egger, Weighted Median, Penalised WM)

## Usage

```
mr(data, X, Y, alpha = 0.05, other_plots = FALSE)
```

## Arguments

|             |                                                                  |
|-------------|------------------------------------------------------------------|
| data        | Data frame containing SNP summary statistics in wide format.     |
| X           | Character string. Exposure trait name.                           |
| Y           | Character vector. Outcome trait names.                           |
| alpha       | Significance level used for confidence intervals (default 0.05). |
| other_plots | Logical. If TRUE, produces metafor funnel and forest plots.      |

## Details

Performs two-sample Mendelian Randomization using summary statistics stored in a wide data frame with columns named:

- SNP
- b.trait : effect estimate
- SE.trait : standard error

For each outcome in Y, the function:

1. Extracts instruments for exposure X
2. Computes IVW, MR-Egger, Weighted Median and Penalised Weighted Median estimates
3. Computes Cochran's Q heterogeneity statistic
4. Produces MR scatter plots

Methods implemented:

- IVW (inverse-variance weighted regression)
- MR-Egger regression
- Weighted median estimator
- Penalised weighted median
- Cochran's Q heterogeneity statistic (metafor)

Required packages: metafor, ggplot2, cowplot. Functions `weighted.median()` and `mr.boot()` must be available in the environment (e.g. from Mendelian randomization toolkits).

## Value

Invisible list with components:

- `r` Matrix of MR estimates for each exposure–outcome pair.
- `plots` List of ggplot objects (MR scatter plots).

## Examples

```
## Not run:
if (requireNamespace("metafor", quietly = TRUE) &&
    requireNamespace("ggplot2", quietly = TRUE) &&
    requireNamespace("cowplot", quietly = TRUE)) {
  txt <- "
rs188743906  0.6804 0.1104  0.00177 0.01660      NA      NA
rs2289779    -0.0788 0.0134  0.00104 0.00261 -0.007543 0.0092258
rs117804300 -0.2281 0.0390 -0.00392 0.00855  0.109372 0.0362219
rs7033492    -0.0968 0.0147 -0.00585 0.00269  0.022793 0.0119903
rs10793962   0.2098 0.0212  0.00378 0.00536 -0.014567 0.0138196
rs635634     -0.2885 0.0153 -0.02040 0.00334  0.077157 0.0117123
rs176690     -0.0973 0.0142  0.00293 0.00306 -0.000007 0.0107781
rs147278971 -0.2336 0.0378 -0.01240 0.00792  0.079873 0.0397491
rs11562629   0.1155 0.0181  0.00960 0.00378 -0.010040 0.0151460
"

v <- c("SNP", "b.LIF.R", "SE.LIF.R", "b.FEV1", "SE.FEV1", "b.CAD", "SE.CAD")
mrdat <- setNames(as.data.frame(scan(text = txt,
                                   what = list("", 0, 0, 0, 0, 0), quiet=TRUE)), v)
res <- mr(mrdat, "LIF.R", c("CAD", "FEV1"), other_plots=TRUE)
r <- as.data.frame(res$r, stringsAsFactors=FALSE)
rownames(r) <- r$IV
r$IV <- NULL
r[] <- lapply(r, function(x) as.numeric(as.character(x)))
b_cols <- grep("^b[A-Z]", names(r), value=TRUE)
se_cols <- paste0("se", substring(b_cols, 2))
keep <- se_cols %in% names(r)
b_cols <- b_cols[keep]; se_cols <- se_cols[keep]
if(length(b_cols) > 0){
  pvals <- sapply(seq_along(b_cols), function(i)
    2 * pnorm(-abs(r[[b_cols[i]]] / r[[se_cols[i]]])))
  pvals <- as.data.frame(pvals)
  colnames(pvals) <- substring(b_cols, 2)
  pvals[] <- lapply(pvals, format.pval, digits=3, eps=1e-4)
  results_table <- cbind(round(r,3), pvals)
} else {
  results_table <- round(r,3)
}
results_table
res$plots
}
```

## End(Not run)

mr\_forestplot

*Mendelian Randomization forest plot***Description**

Mendelian Randomization forest plot

**Usage**

```
mr_forestplot(dat, sm = "", title = "", ...)
```

**Arguments**

|       |                                                               |
|-------|---------------------------------------------------------------|
| dat   | A data.frame with outcome id, effect size and standard error. |
| sm    | Summary measure such as OR, RR, MD.                           |
| title | Title of the meta-analysis.                                   |
| ...   | Additional arguments passed to meta::forest().                |

**Details**

Wrapper around meta::forest() for multi-outcome Mendelian Randomization. Works for binary and continuous outcomes, with or without summary statistics.

**Examples**

```
## Not run:
## Example data -----
tnfb <- '
      "multiple sclerosis" 0.69058600 0.059270400
      "systemic lupus erythematosus" 0.76687500 0.079000500
      "sclerosing cholangitis" 0.62671500 0.075954700
      "juvenile idiopathic arthritis" -1.17577000 0.160293000
      "psoriasis" 0.00582586 0.000800016
      "rheumatoid arthritis" -0.00378072 0.000625160
      "inflammatory bowel disease" -0.14334200 0.025272500
      "ankylosing spondylitis" -0.00316852 0.000626225
      "hypothyroidism" -0.00432054 0.000987324
      "allergic rhinitis" 0.00393075 0.000926002
      "IgA glomerulonephritis" -0.32696600 0.105262000
      "atopic eczema" -0.00204018 0.000678061
'

tnfb <- as.data.frame(scan(file = textConnection(tnfb), what = list("",0,0)))
names(tnfb) <- c("outcome","Effect","StdErr")
tnfb$outcome <- gsub("\\b(^[a-z])","\\U\\1", tnfb$outcome, perl = TRUE)

## 1) Default meta-style forest plot (b, SE, CI + weights) -----
mr_forestplot(
  tnfb,
  colgap.forest.left = "0.05cm",
  fontsize = 14,
  leftcols = c("studlab","effect","seTE","ci"),
```

```

leftlabs = c("Outcome","b","SE","95% CI"),
rightcols = c("w.common","w.random"),
rightlabs = c("Weight (FE)","Weight (RE)"),
common = FALSE, random = FALSE,
print.I2 = FALSE, print.pval.Q = FALSE, print.tau2 = FALSE,
spacing = 1.6, digits.TE = 2, digits.seTE = 2
)

## 2) MR summary (OR + CI only) -----
mr_forestplot(
  tnfb,
  sm = "OR",
  backtransf = TRUE,
  colgap.forest.left = "0.05cm",
  fontsize = 14,
  leftcols = "studlab",
  leftlabs = "Outcome",
  rightcols = c("effect","ci"),
  rightlabs = c("OR","95% CI"),
  sortvar = tnfb$Effect,
  common = FALSE, random = FALSE,
  print.I2 = FALSE, print.pval.Q = FALSE, print.tau2 = FALSE,
  spacing = 1.6
)

## 3) MR summary with P-values -----
mr_forestplot(
  tnfb,
  sm = "OR",
  backtransf = TRUE,
  colgap.forest.left = "0.05cm",
  fontsize = 14,
  leftcols = "studlab",
  leftlabs = "Outcome",
  rightcols = c("effect","ci","pval"),
  rightlabs = c("OR","95% CI","P"),
  digits = 3, digits.pval = 2, scientific.pval = TRUE,
  sortvar = tnfb$Effect,
  common = FALSE, random = FALSE,
  print.I2 = FALSE, print.pval.Q = FALSE, print.tau2 = FALSE,
  addrow = TRUE,
  spacing = 1.6
)

## End(Not run)

```

---

mtdt

---

*Transmission/disequilibrium test of a multiallelic marker*


---

## Description

Transmission/disequilibrium test of a multiallelic marker

## Usage

```
mtdt(x, n.sim = 0)
```

## Arguments

|       |                            |
|-------|----------------------------|
| x     | the data table.            |
| n.sim | the number of simulations. |

## Details

This function calculates transmission-disequilibrium statistics involving multiallelic marker. Inside the function are `tril` and `triu` used to obtain lower and upper triangular matrices.

## Value

It returned list contains the following components:

- SE Spielman-Ewens Chi-square from the observed data.
- ST Stuart or score Statistic from the observed data.
- pSE the simulated p value.
- sSE standard error of the simulated p value.
- pST the simulated p value.
- sST standard error of the simulated p value.

## Author(s)

Mike Miller, Jing Hua Zhao

## References

- Miller MB (1997). "Genomic scanning and the transmission/disequilibrium test: analysis of error rates." *Genet Epidemiol*, **14**(6), 851-6. doi:10.1002/(sici)10982272(1997)14:6<854::aidgepi48>3.3.co;2-7.
- Sham PC, Curtis D (1995). "An extended transmission/disequilibrium test (TDT) for multi-allele marker loci." *Ann Hum Genet*, **59**(3), 323-36. doi:10.1111/j.14691809.1995.tb00751.x.
- Spielman RS, Ewens WJ (1996). "The TDT and other family-based tests for linkage disequilibrium and association." *Am J Hum Genet*, **59**(5), 983-9.
- Zhao JH, Sham PC, Curtis D (1999). "A program for the Monte Carlo evaluation of significance of the extended transmission/disequilibrium test." *Am J Hum Genet*, **64**, 1484-1485.

## See Also

[bt](#)



## Examples

```
## Not run:
x <- matrix(c(0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 3, 0,0, 0, 2, 3, 0, 0, 0, 0,
              2,3,26,35, 7,0, 2,10,11, 3, 4, 1,
              2,3,22,26, 6,2, 4, 4,10, 2, 2, 0,
              0,1, 7,10, 2,0, 0, 2, 2, 1, 1, 0,
              0,0, 1, 4, 0,1, 0, 1, 0, 0, 0, 0,
              0,2, 5, 4, 1,1, 0, 0, 0, 2, 0, 0,
              0,0, 2, 6, 1,0, 2, 0, 2, 0, 0, 0,
              0,3, 6,19, 6,0, 0, 2, 5, 3, 0, 0,
              0,0, 3, 1, 1,0, 0, 0, 1, 0, 0, 0,
              0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 0, 0,0, 0, 0, 0, 0, 0, 0),nrow=12)

# See note to bt for the score test obtained by SAS

mtdt(x)

## End(Not run)
```

---

|       |                                                                                         |
|-------|-----------------------------------------------------------------------------------------|
| mtdt2 | <i>Transmission/disequilibrium test of a multiallelic marker by Bradley-Terry model</i> |
|-------|-----------------------------------------------------------------------------------------|

---

## Description

Transmission/disequilibrium test of a multiallelic marker by Bradley-Terry model

## Usage

```
mtdt2(x, verbose = TRUE, n.sim = NULL, ...)
```

## Arguments

|         |                                                 |
|---------|-------------------------------------------------|
| x       | the data table.                                 |
| verbose | To print out test statistics if TRUE.           |
| n.sim   | Number of simulations.                          |
| ...     | other options compatible with the BTm function. |

## Details

This function calculates transmission-disequilibrium statistics involving multiallelic marker according to Bradley-Terry model.

## Value

It returned list contains the following components:

- c2b A data frame in four-column format showing transmitted vs nontransmitted counts.
- BTm A fitted Bradley-Terry model object.

- X2 Allele-wise, genotype-wise and goodness-of-fit Chi-squared statistics.
- df Degrees of freedom.
- p P value.
- pn Monte Carlo p values when n.sim is specified.

### Author(s)

Jing Hua Zhao keywords models keywords htest

### References

Firth D (2005). “Bradley-Terry Models in R.” *Journal of Statistical Software*, **12**(1), 1 - 12. [doi:10.18637/jss.v012.i01](https://doi.org/10.18637/jss.v012.i01).

Turner H, Firth D (2010) Bradley-Terry models in R: The BradleyTerry2 package. <https://cran.r-project.org/web/packages/BradleyTerry2/vignettes/BradleyTerry.pdf>.

### See Also

[mtdt](#)

### Examples

```
## Not run:
x <- matrix(c(0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 3, 0,0, 0, 2, 3, 0, 0, 0, 0,
              2,3,26,35, 7,0, 2,10,11, 3, 4, 1,
              2,3,22,26, 6,2, 4, 4,10, 2, 2, 0,
              0,1, 7,10, 2,0, 0, 2, 2, 1, 1, 0,
              0,0, 1, 4, 0,1, 0, 1, 0, 0, 0, 0,
              0,2, 5, 4, 1,1, 0, 0, 0, 2, 0, 0,
              0,0, 2, 6, 1,0, 2, 0, 2, 0, 0, 0,
              0,3, 6,19, 6,0, 0, 2, 5, 3, 0, 0,
              0,0, 3, 1, 1,0, 0, 0, 1, 0, 0, 0,
              0,0, 0, 2, 0,0, 0, 0, 0, 0, 0, 0,
              0,0, 1, 0, 0,0, 0, 0, 0, 0, 0, 0),nrow=12)

xx <- mtdt2(x,refcat="12")

## End(Not run)
```

---

muvar

*Means and variances under 1- and 2- locus (biallelic) QTL model*

---

### Description

Means and variances under 1- and 2- locus (biallelic) QTL model

**Usage**

```
muvar(
  n.loci = 1,
  y1 = c(0, 1, 1),
  y12 = c(1, 1, 1, 1, 1, 0, 0, 0, 0),
  p1 = 0.99,
  p2 = 0.9
)
```

**Arguments**

|        |                                                                                                |
|--------|------------------------------------------------------------------------------------------------|
| n.loci | number of loci, 1=single locus, 2=two loci.                                                    |
| y1     | the genotypic means of aa, Aa and AA.                                                          |
| y12    | the genotypic means of aa, Aa and AA at the first locus and bb, Bb and BB at the second locus. |
| p1     | the frequency of the lower allele, or the that for the first locus under a 2-locus model.      |
| p2     | the frequency of the lower allele at the second locus.                                         |

**Details**

Function muvar() gives means and variances under 1-locus and 2-locus QTL model (simple); in the latter case it gives results from different avenues. This function is included for experimental purpose and yet to be generalized.

**Value**

Currently it does not return any value except screen output; the results can be kept via R's sink() command or via modifying the C/R codes.

**Note**

Adapted from an earlier C program written for the above book.

**Author(s)**

Jing Hua Zhao

**References**

Sham P (1997). *Statistics in Human Genetics*. Wiley. ISBN 978-0470689288.

**Examples**

```
## Not run:
# the default 1-locus model
muvar(n.loci=1,y1=c(0,1,1),p1=0.5)

# the default 2-locus model
muvar(n.loci=2,y12=c(1,1,1,1,1,0,0,0,0),p1=0.99,p2=0.9)

## End(Not run)
```

mvmeta

*Multivariate fixed-effects meta-analysis via generalized least squares***Description**

Performs multivariate meta-analysis by pooling study-specific parameter estimates using generalized least squares (GLS) under a fixed-effects model.

**Usage**

```
mvmeta(b, V)
```

**Arguments**

**b** Matrix of study estimates (studies  $\times$  parameters).  
**V** may be supplied with

- a matrix of upper-triangular rows
- a list of covariance matrices
- a 3D array ( $p \times p \times k$ )

**Details**

The function accepts a matrix of parameter estimates and the corresponding within-study covariance matrices (stored in upper-triangular vector form).

This approach is appropriate when combining correlated effect estimates, for example correlation coefficients of SNPs across studies.

The function fits a multivariate **fixed-effects meta-analysis** using generalized least squares (GLS).

For study  $i = 1, \dots, k$ , let  $d_i$  be the vector of observed parameter estimates and  $\Psi_i$  the corresponding within-study covariance matrix:

$$d_i \sim N(\beta, \Psi_i)$$

where  $\beta$  is the vector of common (pooled) parameters.

The study estimates are stacked into a single vector

$$d = (d_1^T, \dots, d_k^T)^T$$

with block-diagonal covariance matrix

$$\Psi = \text{blockdiag}(\Psi_1, \dots, \Psi_k).$$

The model can then be written in GLS regression form

$$d = X\beta + \varepsilon, \quad \varepsilon \sim N(0, \Psi)$$

where  $X$  is a block design matrix that repeats an identity matrix for each study (intercept-only multivariate meta-analysis). Missing outcomes are automatically removed when constructing  $d$ ,  $\Psi$  and  $X$ .

The pooled estimator is the GLS estimator

$$\hat{\beta} = (X^T \Psi^{-1} X)^{-1} X^T \Psi^{-1} d.$$

Heterogeneity is assessed using the multivariate Cochran Q statistic

$$Q = (d - X\hat{\beta})^T \Psi^{-1} (d - X\hat{\beta}),$$

which is asymptotically  $\chi^2_{N-p}$ , where  $N$  is the number of observed estimates and  $p$  the number of pooled parameters.

This implementation corresponds to the multivariate fixed-effects model described in Hartung et al. (2008, Example 11.3).

### Value

An object of class "mvmeta" with the following elements:

- beta: pooled estimates
- vcov: covariance matrix of pooled estimates
- se: standard errors
- z: z statistics
- pval: p values
- ci: 95% confidence intervals
- X2, df, p: heterogeneity test
- logLik: model log-likelihood
- k: number of studies
- p\_outcomes: number of pooled outcomes

### Author(s)

Jing Hua Zhao

### References

Hartung J, Knapp G, Sinha BK (2008). *Statistical Meta-analysis with Applications*. Wiley. ISBN 978-0-470-29089-7.

### See Also

[metareg](#)

### Examples

```
## Not run:
# Example 11.3 from Hartung et al.
b <- matrix(c(
  0.808, 1.308, 1.379, NA, NA,
  NA, 1.266, 1.828, 1.962, NA,
  NA, 1.835, NA, 2.568, NA,
  NA, 1.272, NA, NA, 2.038,
  1.171, 2.024, 2.423, 3.159, NA,
  0.681, NA, NA, NA, NA), ncol=5, byrow=TRUE)

psi1 <- psi2 <- psi3 <- psi4 <- psi5 <- psi6 <- matrix(0,5,5)
psi1[1,1] <- 0.0985; psi1[1,2] <- 0.0611; psi1[1,3] <- 0.0623
psi1[2,2] <- 0.1142; psi1[2,3] <- 0.0761; psi1[3,3] <- 0.1215
```

```

psi2[2,2] <- 0.0713; psi2[2,3] <- 0.0539; psi2[2,4] <- 0.0561
psi2[3,3] <- 0.0938; psi2[3,4] <- 0.0698; psi2[4,4] <- 0.0981

psi3[2,2] <- 0.1228; psi3[2,4] <- 0.1119; psi3[4,4] <- 0.1790
psi4[2,2] <- 0.0562; psi4[2,5] <- 0.0459; psi4[5,5] <- 0.0815

psi5[1,1] <- 0.0895; psi5[1,2] <- 0.0729; psi5[1,3] <- 0.0806
psi5[1,4] <- 0.0950; psi5[2,2] <- 0.1350; psi5[2,3] <- 0.1151
psi5[2,4] <- 0.1394; psi5[3,3] <- 0.1669; psi5[3,4] <- 0.1609
psi5[4,4] <- 0.2381

psi6[1,1] <- 0.0223

V <- rbind(psi1[upper.tri(psi1,diag=TRUE)],
           psi2[upper.tri(psi2,diag=TRUE)],
           psi3[upper.tri(psi3,diag=TRUE)],
           psi4[upper.tri(psi4,diag=TRUE)],
           psi5[upper.tri(psi5,diag=TRUE)],
           psi6[upper.tri(psi6,diag=TRUE)])

fit <- mvmeta(b, V)
summary(fit)
logLik(fit)
AIC(fit)
BIC(fit)

## End(Not run)

```

---

pbsize

*Power for population-based association design*


---

## Description

Power for population-based association design

## Usage

```
pbsize(kp, gamma = 4.5, p = 0.15, alpha = 5e-08, beta = 0.2)
```

## Arguments

|       |                                                       |
|-------|-------------------------------------------------------|
| kp    | population disease prevalence.                        |
| gamma | genotype relative risk assuming multiplicative model. |
| p     | frequency of disease allele.                          |
| alpha | type I error rate.                                    |
| beta  | type II error rate.                                   |

## Details

This function implements Scott et al. (1997) statistics for population-based association design. This is based on a contingency table test and accurate level of significance can be obtained by Fisher's exact test.

**Value**

The returned value is scalar containing the required sample size.

**Author(s)**

Jing Hua Zhao extracted from rm.c.

**References**

Scott WK, Pericak-Vance MA, Haines JL (1997). "Genetic analysis of complex diseases." *Science*, **275**(5304), 1327; author reply 1329-30. Rosner B (2000). *Fundamentals of biostatistics*, 5 edition. Duxbury, Pacific Grove, CA. ISBN 9780534370688. Armitage P, Colton T (eds.) (2005). *Encyclopedia of biostatistics*, 2 edition. John Wiley, Chichester, West Sussex, England ; Hoboken, NJ. ISBN 9780470849071.

**See Also**

[fbsize](#)

**Examples**

```
kp <- c(0.01,0.05,0.10,0.2)
models <- matrix(c(
  4.0, 0.01,
  4.0, 0.10,
  4.0, 0.50,
  4.0, 0.80,
  2.0, 0.01,
  2.0, 0.10,
  2.0, 0.50,
  2.0, 0.80,
  1.5, 0.01,
  1.5, 0.10,
  1.5, 0.50,
  1.5, 0.80), ncol=2, byrow=TRUE)
outfile <- "pbsize.txt"
cat("gamma", "p", "p1", "p5", "p10", "p20\n", sep="\t", file=outfile)
for(i in 1:dim(models)[1])
{
  g <- models[i,1]
  p <- models[i,2]
  n <- vector()
  for(k in kp) n <- c(n, ceiling(pbsize(k,g,p)))
  cat(models[i,1:2], n, sep="\t", file=outfile, append=TRUE)
  cat("\n", file=outfile, append=TRUE)
}
table5 <- read.table(outfile, header=TRUE, sep="\t")
unlink(outfile)

# Alzheimer's disease
g <- 4.5
p <- 0.15
alpha <- 5e-8
beta <- 0.2
z1alpha <- qnorm(1-alpha/2) # 5.45
z1beta <- qnorm(1-beta)
```

```

q <- 1-p
pi <- 0.065 # 0.07 and zbeta generate 163
k <- pi*(g*p+q)^2
s <- (1-pi*g^2)*p^2+(1-pi*g)^2*p*q+(1-pi)*q^2
# LGL formula
lambda <- pi*(g^2*p+q-(g*p+q)^2)/(1-pi*(g*p+q)^2)
# mine
lambda <- pi*p*q*(g-1)^2/(1-k)
n <- (z1alpha+z1beta)^2/lambda
cat("\nPopulation-based result: Kp =",k, "Kq =",s, "n =",ceiling(n),"n")

```

pbsize2

*Power for case-control association design*

## Description

Power for case-control association design

## Usage

```

pbsize2(
  N,
  fc = 0.5,
  alpha = 0.05,
  gamma = 4.5,
  p = 0.15,
  kp = 0.1,
  model = "additive"
)

```

## Arguments

|       |                                                                                             |
|-------|---------------------------------------------------------------------------------------------|
| N     | The sample size.                                                                            |
| fc    | The proportion of cases in the sample.                                                      |
| alpha | Type I error rate.                                                                          |
| gamma | The genotype relative risk (GRR).                                                           |
| p     | Frequency of the risk allele.                                                               |
| kp    | The prevalence of disease in the population.                                                |
| model | Disease model, i.e., "multiplicative", "additive", "dominant", "recessive", "overdominant". |

## Details

This extends [pbsize](#) from a multiplicative model for a case-control design under a range of disease models. Essentially, for given sample sizes(s), a proportion of which (fc) being cases, the function calculates power estimate for a given type I error (alpha), genotype relative risk (gamma), frequency of the risk allele (p), the prevalence of disease in the population (kp) and optionally a disease model (model). A major difference would be the consideration of case/control ascertainment in [pbsize](#).

Internally, the function obtains a baseline risk to make the disease model consistent with Kp as in [tscc](#) and should produce accurate power estimate. It provides power estimates for given sample size(s) only.



**Value**

The returned value is the power for the specified design.

**See Also**

The design follows that of [pbsize](#).

**Examples**

```
## Not run:
# single calculation
m <- c("multiplicative","recessive","dominant","additive","overdominant")
for(i in 1:5) print(pbsize2(N=50,alpha=5e-2,gamma=1.1,p=0.1,kp=0.1, model=m[i]))

# a range of sample sizes
pbsize2(p=0.1, N=c(25,50,100,200,500), gamma=1.2, kp=.1, alpha=5e-2, model='r')

# a power table
m <- sapply(seq(0.1,0.9, by=0.1),
            function(x) pbsize2(p=x, N=seq(100,1000,by=100),
                                gamma=1.2, kp=.1, alpha=5e-2, model='recessive'))
colnames(m) <- seq(0.1,0.9, by=0.1)
rownames(m) <- seq(100,1000,by=100)
print(round(m,2))

## End(Not run)
```

---

pedtodot

*Converting pedigree(s) to dot file(s)*

---

**Description**

Converting pedigree(s) to dot file(s)

**Usage**

```
pedtodot(
  pedfile,
  makeped = FALSE,
  sink = TRUE,
  page = "B5",
  url = "https://jinghuazhao.github.io/",
  height = 0.5,
  width = 0.75,
  rotate = 0,
  dir = "none"
)
```

## Arguments

|         |                                                                                                                                                   |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| pedfile | a pedigree file in GAS or LINKAGE format, note if individual's ID is character then it is necessary to specify as.is=T in the read.table command. |
| makeped | a logical variable indicating if the pedigree file is post-makeped.                                                                               |
| sink    | a logical variable indicating if .dot file(s) are created.                                                                                        |
| page    | a string indicating the page size, e.g, A4, A5, B5, Legal, Letter, Executive, "x,y", where x, y is the customized page size.                      |
| url     | Unified Resource Locator (URL) associated with the diagram(s).                                                                                    |
| height  | the height of node(s).                                                                                                                            |
| width   | the width of node(s).                                                                                                                             |
| rotate  | if set to 90, the diagram is in landscape.                                                                                                        |
| dir     | direction of edges, i.e., "none", "forward", "back", "both". This will be useful if the diagram is viewed by Ineato.                              |

## Details

This function converts GAS or LINKAGE formatted pedigree(s) into .dot file for each pedigree to be used by dot in graphviz, which is a flexible package for graphics freely available.

Note that a single PostScript (PDF) file can be obtained by dot, fdp, or neato.

```
dot -Tps <dot file> -o <ps file>
```

or

```
fdp -Tps <dot file> -o <ps file>
```

or

```
neato -Tps <dot file> -o <ps file>
```

See relevant documentations for other formats.

To preserve the original order of pedigree(s) in the data, you can examine the examples at the end of this document.

Under Cygwin/Linux/Unix, the PostScript file can be converted to Portable Document Format (PDF) default to Acrobat.

```
ps2pdf <ps file>
```

Use ps2pdf12, ps2pdf13, or ps2pdf14 for appropriate versions of Acrobat according to information given on the headline of <ps file>.

Under Linux, you can also visualize the .dot file directly via command,

```
dotty <dot file> &
```

We can extract the code below (or within pedtodot.Rd) to pedtodot and then use command:

```
sh pedtodot <pedigree file>
```

**Value**

For each pedigree, the function generates a .dot file to be used by dot. The collection of all pedigrees (\*.dot) can also be put together.

**Note**

This is based on the gawk script program pedtodot by David Duffy with minor changes.

**Author(s)**

David Duffy, Jing Hua Zhao

**See Also**

package sem in CRAN and Rgraphviz in BioConductor <https://www.bioconductor.org/>.

**Examples**

```
## Not run:
# example as in R News and Bioinformatics (see also plot.pedigree in package kinship)
# it works from screen paste only
p1 <- scan(nlines=16,what=list(0,0,0,0,0,"",""))
 1  2  3  2  2  7/7  7/10
 2  0  0  1  1  -/-  -/-
 3  0  0  2  2  7/9  3/10
 4  2  3  2  2  7/9  3/7
 5  2  3  2  1  7/7  7/10
 6  2  3  1  1  7/7  7/10
 7  2  3  2  1  7/7  7/10
 8  0  0  1  1  -/-  -/-
 9  8  4  1  1  7/9  3/10
10  0  0  2  1  -/-  -/-
11  2 10  2  1  7/7  7/7
12  2 10  2  2  6/7  7/7
13  0  0  1  1  -/-  -/-
14 13 11  1  1  7/8  7/8
15  0  0  1  1  -/-  -/-
16 15 12  2  1  6/6  7/7

p2 <- as.data.frame(p1)
names(p2) <-c("id","fid","mid","sex","aff","GABRB1","D4S1645")
p3 <- data.frame(pid=10081,p2)
pedtodot(p3)
#
# Three examples of pedigree-drawing
# assuming pre-MakePed LINKAGE file in which IDs are characters
pre<-read.table("pheno.pre",as.is=TRUE)[,1:6]
pedtodot(pre)
dir()
# for post-MakePed LINKAGE file in which IDs are integers
ped <-read.table("pheno.ped")[,1:10]
pedtodot(ped,makeped=TRUE)
dir()
# for a single file with a list of pedigrees ordered data
sink("gaw14.dot")
pedtodot(ped,sink=FALSE)
```

```

sink()
file.show("gaw14.dot")
# more details
pedtodot(ped,sink=FALSE,page="B5",url="https://jinghuazhao.github.io/")

# An example from Richard Mott and in the demo
filespec <- system.file("tests/ped.1.3.pre")
pre <- read.table(filespec,as.is=TRUE)
pre
pedtodot(pre,dir="forward")

## End(Not run)

```

---

|                   |                                       |
|-------------------|---------------------------------------|
| pedtodot_verbatim | <i>Pedigree-drawing with graphviz</i> |
|-------------------|---------------------------------------|

---

## Description

Pedigree-drawing with graphviz

## Usage

```
pedtodot_verbatim(f)
```

## Arguments

|   |                                                                                                                          |
|---|--------------------------------------------------------------------------------------------------------------------------|
| f | A data.frame containing pedigrees, each with pedigree id, individual id, father id, mother id, sex and affection status. |
|---|--------------------------------------------------------------------------------------------------------------------------|

## Details

Read a GAS or LINKAGE pedigree and write a Graphviz DOT representation for each pedigree.

This is a verbatim translation of the original pedtodot implemented in Bash/awk. To check independently, try `xclip < pedtodot_verbatim.R` and paste into an R session.

## Value

For each pedigree, a Graphviz DOT file is written to the working directory.

## Note

Adapted from Bash/awk script by David Duffy

## Examples

```

## Not run:
# pedigree p3 in pedtodot
pedtodot_verbatim(p3)

## End(Not run)

```

pfc

*Probability of familial clustering of disease***Description**

Probability of familial clustering of disease

**Usage**

```
pfc(famdata, enum = 0)
```

**Arguments**

|         |                                                                                    |
|---------|------------------------------------------------------------------------------------|
| famdata | collective information of sib size, number of affected sibs and their frequencies. |
| enum    | a switch taking value 1 if all possible tables are to be enumerated.               |

**Details**

To calculate exact probability of familial clustering of disease

**Value**

The returned value is a list containing (tailp,sump,nenum are only available if enum=1:

- p the probability of familial clustering.
- stat the deviances, chi-squares based on binomial and hypergeometric distributions, the degrees of freedom should take into account the number of marginals used.
- tailp the exact statistical significance.
- sump sum of the probabilities used for error checking.
- nenum the total number of tables enumerated.

**Note**

Adapted from family.for by Dani Zelterman, 25/7/03

**Author(s)**

Dani Zelterman, Jing Hua Zhao

**References**

Yu C, Zelterman D (2001). "Exact inference for family disease clusters." *Communications in Statistics - Theory and Methods*, **30**(11), 2293-2305. doi:[10.1081/STA100107686](https://doi.org/10.1081/STA100107686).

Yu C, Zelterman D (2002). "Statistical inference for familial disease clusters." *Biometrics*, **58**(3), 481-91. doi:[10.1111/j.0006341x.2002.00481.x](https://doi.org/10.1111/j.0006341x.2002.00481.x).

**See Also**

[kin.morgan](#)

## Examples

```
## Not run:
# IPF among 203 siblings of 100 COPD patients from Liang KY, SL Zeger,
# Qaquish B. Multivariate regression analyses for categorical data
# (with discussion). J Roy Stat Soc B 1992, 54:3-40

# the degrees of freedom is 15
famtest<-c(
  1, 0, 36,
  1, 1, 12,
  2, 0, 15,
  2, 1, 7,
  2, 2, 1,
  3, 0, 5,
  3, 1, 7,
  3, 2, 3,
  3, 3, 2,
  4, 0, 3,
  4, 1, 3,
  4, 2, 1,
  6, 0, 1,
  6, 2, 1,
  6, 3, 1,
  6, 4, 1,
  6, 6, 1)
test<-t(matrix(famtest,nrow=3))
famp<-pfc(test)

## End(Not run)
```

---

pfc.sim

---

*Probability of familial clustering of disease*


---

## Description

Probability of familial clustering of disease

## Usage

```
pfc.sim(famdata, n.sim = 1e+06, n.loop = 1)
```

## Arguments

|         |                                                                                    |
|---------|------------------------------------------------------------------------------------|
| famdata | collective information of sib size, number of affected sibs and their frequencies. |
| n.sim   | number of simulations in a single Monte Carlo run.                                 |
| n.loop  | total number of Monte Carlo runs.                                                  |

## Details

To calculate probability of familial clustering of disease using Monte Carlo simulation.

**Value**

The returned value is a list containing:

- n.sim a copy of the number of simulations in a single Monte Carlo run.
- n.loop the total number of Monte Carlo runs.
- p the observed p value.
- tailpl accumulated probabilities at the lower tails.
- tailpu simulated p values.

**Note**

Adapted from runi.for from Chang Yu, 5/6/4

**Author(s)**

Chang Yu, Dani Zelterman

**References**

Yu C, Zelterman D (2001). "Exact inference for family disease clusters." *Communications in Statistics - Theory and Methods*, **30**(11), 2293-2305. doi:[10.1081/STA100107686](https://doi.org/10.1081/STA100107686).

**See Also**

[pfc](#)

**Examples**

```
## Not run:
# Li FP, Fraumeni JF Jr, Mulvihill JJ, Blattner WA, Dreyfus MG, Tucker MA,
# Miller RW. A cancer family syndrome in twenty-four kindreds.
# Cancer Res 1988, 48(18):5358-62.

# family_size #_of_affected frequency

famtest<-c(
  1, 0, 2,
  1, 1, 0,
  2, 0, 1,
  2, 1, 4,
  2, 2, 3,
  3, 0, 0,
  3, 1, 2,
  3, 2, 1,
  3, 3, 1,
  4, 0, 0,
  4, 1, 2,
  5, 0, 0,
  5, 1, 1,
  6, 0, 0,
  6, 1, 1,
  7, 0, 0,
  7, 1, 1,
  8, 0, 0,
```

```

8, 1, 1,
8, 2, 1,
8, 3, 1,
9, 3, 1)

test<-matrix(famtest,byrow=T,ncol=3)

famp<-pfc.sim(test)

## End(Not run)

```

pgc

*Preparing weight for GENECOUNTING***Description**

Preparing weight for GENECOUNTING

**Usage**

```
pgc(data, handle.miss = 1, is.genotype = 0, with.id = 0)
```

**Arguments**

|                          |                                                                                |
|--------------------------|--------------------------------------------------------------------------------|
| <code>data</code>        | the multilocus genotype data for a set of individuals.                         |
| <code>handle.miss</code> | a flag to indicate if missing data is kept, 0 = no, 1 = yes.                   |
| <code>is.genotype</code> | a flag to indicate if the data is already in the form of genotype identifiers. |
| <code>with.id</code>     | a flag to indicate if the unique multilocus genotype identifier is generated.  |

**Details**

This function is a R port of the GENECOUNTING/PREPARE program which takes an array of genotyp data and collapses individuals with the same multilocus genotype. This function can also be used to prepare for the genotype table in testing Hardy-Weinberg equilibrium.

**Value**

The returned value is a list containing:

- `cdata` the collapsed genotype data.
- `wt` the frequency weight.
- `obscom` the observed number of combinations or genotypes.
- `idsave` optional, available only if `with.id = 1`.

**Note**

Built on `pgc.c`.

**Author(s)**

Jing Hua Zhao



## References

Zhao JH, Sham PC (2003). “Generic number systems and haplotype analysis.” *Comput Methods Programs Biomed*, **70**(1), 1-9. doi:10.1016/s01692607(01)001936.

## See Also

[genecounting](#), [hwe.hardy](#)

## Examples

```
## Not run:
require(gap.datasets)
data(hla)
x <- hla[,3:8]

# do not handle missing data
y<-pgc(x,handle.miss=0,with.id=1)
hla.gc<-genecounting(y$data,y$wt)

# handle missing but with multilocus genotype identifier
pgc(x,handle.miss=1,with.id=1)

# handle missing data with no identifier
pgc(x,handle.miss=1,with.id=0)

## End(Not run)
```

---

plot.hap.score

*Plot haplotype frequencies versus haplotype score statistics*

---

## Description

Method function to plot a class of type hap.score

## Usage

```
## S3 method for class 'hap.score'
plot(x, ...)
```

## Arguments

|     |                                                                 |
|-----|-----------------------------------------------------------------|
| x   | The object returned from hap.score (which has class hap.score). |
| ... | Optional arguments.                                             |

## Value

Nothing is returned.

This is a plot method function used to plot haplotype frequencies on the x-axis and haplotype-specific scores on the y-axis. Because hap.score is a class, the generic plot function can be used, which in turn calls this plot.hap.score function.

## References

Schaid DJ, Rowland CM, Tines DE, Jacobson RM, Poland GA (2002) Score tests for association of traits with haplotypes when linkage phase is ambiguous. *Amer J Hum Genet* 70:425-34

## See Also

[hap.score](#)

## Examples

```
## Not run:
save <- hap.score(y, geno, trait.type = "gaussian")

# Example illustrating generic plot function:
plot(save)

# Example illustrating specific method plot function:
plot.hap.score(save)

## End(Not run)
```

---

|                 |                                 |
|-----------------|---------------------------------|
| print.hap.score | <i>Print a hap.score object</i> |
|-----------------|---------------------------------|

---

## Description

Method function to print a class of type hap.score

## Usage

```
## S3 method for class 'hap.score'
print(x, ...)
```

## Arguments

|     |                                                                 |
|-----|-----------------------------------------------------------------|
| x   | The object returned from hap.score (which has class hap.score). |
| ... | Optional arguments.                                             |

## Value

Nothing is returned.

This is a print method function used to print information from hap.score class, with haplotype-specific information given in a table. Because hap.score is a class, the generic print function can be used, which in turn calls this print.hap.score function.

## References

Schaid DJ, Rowland CM, Tines DE, Jacobson RM, Poland GA (2002) Score tests for association of traits with haplotypes when linkage phase is ambiguous. *Amer J Hum Genet* 70:425-34

**See Also**[hap.score](#)**Examples**

```
## Not run:
save <- hap.score(y, geno, trait.type = "gaussian")

# Example illustrating generic print function:
print(save)

# Example illustrating specific method print function:
print.hap.score(save)

## End(Not run)
```

---

|        |                                     |
|--------|-------------------------------------|
| pvalue | <i>P value for a normal deviate</i> |
|--------|-------------------------------------|

---

**Description**

P value for a normal deviate

**Usage**

```
pvalue(z, decimals = 2)
```

**Arguments**

|          |                           |
|----------|---------------------------|
| z        | normal deviate.           |
| decimals | number of decimal places. |

**Value**

P value as a string variable.

**Examples**

```
pvalue(-1.96)
```

qqfun

*Quantile-comparison plots***Description**

Quantile-comparison plots

**Usage**

```
qqfun(
  x,
  distribution = "norm",
  ylab = deparse(substitute(x)),
  xlab = paste(distribution, "quantiles"),
  main = NULL,
  las = par("las"),
  envelope = 0.95,
  labels = FALSE,
  col = palette()[4],
  lcol = palette()[2],
  xlim = NULL,
  ylim = NULL,
  lwd = 1,
  pch = 1,
  bg = palette()[4],
  cex = 0.4,
  line = c("quartiles", "robust", "none"),
  ...
)
```

**Arguments**

|              |                                                                                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| x            | vector of numeric values.                                                                                                                                          |
| distribution | root name of comparison distribution – e.g., norm for the normal distribution; t for the t-distribution.                                                           |
| ylab         | label for vertical (empirical quantiles) axis.                                                                                                                     |
| xlab         | label for horizontal (comparison quantiles) axis.                                                                                                                  |
| main         | label for plot.                                                                                                                                                    |
| las          | if 0, ticks labels are drawn parallel to the axis; set to 1 for horizontal labels (see <a href="#">graphics::par</a> ).                                            |
| envelope     | confidence level for point-wise confidence envelope, or FALSE for no envelope.                                                                                     |
| labels       | vector of point labels for interactive point identification, or FALSE for no labels.                                                                               |
| col          | color for points; the default is the <i>fourth</i> entry in the current color palette (see <a href="#">grDevices::palette</a> and <a href="#">graphics::par</a> ). |
| lcol         | color for lines; the default is the <i>second</i> entry as above.                                                                                                  |
| xlim         | the x limits (x1, x2) of the plot. Note that x1 > x2 is allowed and leads to a reversed axis.                                                                      |
| ylim         | the y limits of the plot.                                                                                                                                          |

|      |                                                                                                                                                                                                                                |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| lwd  | line width; default is 1 (see <a href="#">graphics::par</a> ). Confidence envelopes are drawn at half this line width.                                                                                                         |
| pch  | plotting character for points; default is 1 (a circle, see <a href="#">graphics::par</a> ).                                                                                                                                    |
| bg   | background color of points.                                                                                                                                                                                                    |
| cex  | factor for expanding the size of plotted symbols; the default is .4.                                                                                                                                                           |
| line | "quartiles" to pass a line through the quartile-pairs, or "robust" for a robust-regression line; the latter uses the <code>r1m</code> function in the MASS package. Specifying <code>line = "none"</code> suppresses the line. |
| ...  | arguments such as <code>df</code> to be passed to the appropriate quantile function.                                                                                                                                           |

## Details

Plots empirical quantiles of a variable against theoretical quantiles of a comparison distribution.

Draws theoretical quantile-comparison plots for variables and for studentized residuals from a linear model. A comparison line is drawn on the plot either through the quartiles of the two distributions, or by robust regression.

Any distribution for which quantile and density functions exist in R (with prefixes `q` and `d`, respectively) may be used. Studentized residuals are plotted against the appropriate t-distribution.

This is adapted from [car::qq.plot](#) with different values for points and lines, more options, more transparent code and examples in the current setting. Another similar but sophisticated function is [lattice::qqmath](#).

## Value

These functions are used only for their side effect (to make a graph).

## Author(s)

John Fox, Jing Hua Zhao

## References

Davison AC (2003). *Statistical Models (Cambridge Series in Statistical and Probabilistic Mathematics)*. Cambridge University Press (2003-08-04). doi:10.1017/CBO9780511815850. Leemis LM, McQueston JT (2008). "Univariate Distribution Relationships." *The American Statistician*, 62(1), 45-53. doi:10.1198/000313008X270448.

## See Also

[stats::qqnorm](#), [qqunif](#), [gcontrol2](#)

## Examples

```
## Not run:
p <- runif(100)
alpha <- 1/log(10)

qqfun(p,distribution="unif")
qqfun(-log10(p),distribution="exp",rate=alpha,pch=21)

library(car)
qq.plot(p,dist="unif")
```

```
qq.plot(-log10(p),dist="exp",rate=alpha)

library(lattice)
qqmath(~ -log10(p), distribution=function(p) qexp(p,rate=alpha))

## End(Not run)
```

qqunif

*Q-Q plot for uniformly distributed random variables***Description**

Q-Q plot for uniformly distributed random variables

**Usage**

```
qqunif(
  u,
  type = "unif",
  logscale = TRUE,
  base = 10,
  col = palette()[4],
  lcol = palette()[2],
  ci = FALSE,
  alpha = 0.05,
  ...
)
```

**Arguments**

|                       |                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------|
| <code>u</code>        | A vector of uniformly distributed random variables.                                               |
| <code>type</code>     | Distribution type: "unif" for uniform order statistics or "exp" for exponential order statistics. |
| <code>logscale</code> | Logical; use log scale.                                                                           |
| <code>base</code>     | Base of the logarithm.                                                                            |
| <code>col</code>      | Color for points.                                                                                 |
| <code>lcol</code>     | Color for the diagonal reference line.                                                            |
| <code>ci</code>       | Logical; show confidence intervals.                                                               |
| <code>alpha</code>    | Significance level for confidence intervals.                                                      |
| <code>...</code>      | Additional graphical arguments passed to <code>qqplot()</code> .                                  |

**Details**

This function produces a Q-Q plot for a random variable following a uniform distribution, optionally on a logarithmic scale.

For type = "exp", the plot is based on exponential order statistics, which is generally more appropriate than directly log-transforming the expected uniform order statistics.

**Value**

Invisibly returns the list produced by `qqplot()` with components:

- x** Expected quantiles.
- y** Observed quantiles.

**Author(s)**

Jing Hua Zhao

**References**

Balakrishnan N, Nevzorov VB (2003). *A Primer on Statistical Distributions*. Wiley, Hoboken, N.J. ISBN 9780471427988. doi:[10.1002/0471722227](https://doi.org/10.1002/0471722227). Casella G, Berger RL (2002). *Statistical Inference*, 2 edition. Duxbury. ISBN 978-0-534-24312-8. Davison AC (2003). *Statistical Models (Cambridge Series in Statistical and Probabilistic Mathematics)*. Cambridge University Press (2003-08-04). doi:[10.1017/CBO9780511815850](https://doi.org/10.1017/CBO9780511815850).

**See Also**

[qqfun](#)

**Examples**

```
## Not run:
u_obs <- runif(1000)
r <- qqunif(u_obs, pch=21, bg="blue", bty="n")
u_exp <- r$y
hits <- u_exp >= 2.30103
points(r$x[hits], u_exp[hits], pch=21, bg="green")
legend("topleft", sprintf("GC.lambda = %.4f", gc.lambda(u_obs)))

## End(Not run)
```

---

qtl2dplot

2D QTL plot

---

**Description**

2D QTL plot

**Usage**

```
qtl2dplot(
  d,
  chrLen = gap::hg19,
  snp_name = "SNP",
  snp_chr = "Chr",
  snp_pos = "bp",
  gene_chr = "p.chr",
  gene_start = "p.start",
```

```

    gene_end = "p.end",
    trait = "p.target.short",
    gene = "p.gene",
    TSS = FALSE,
    cis = "cis",
    value = "log10p",
    plot = TRUE,
    cex.labels = 0.6,
    cex.points = 0.6,
    xlab = "QTL position",
    ylab = "Gene position"
  )

```

### Arguments

|            |                                                              |
|------------|--------------------------------------------------------------|
| d          | Data to be used.                                             |
| chrLen     | lengths of chromosomes for specific build: hg18, hg19, hg38. |
| snp_name   | variant name.                                                |
| snp_chr    | variant chromosome.                                          |
| snp_pos    | variant position.                                            |
| gene_chr   | gene chromosome.                                             |
| gene_start | gene start position.                                         |
| gene_end   | gene end position.                                           |
| trait      | trait name.                                                  |
| gene       | gene name.                                                   |
| TSS        | to use TSS when TRUE.                                        |
| cis        | cis variant when TRUE.                                       |
| value      | A specific value to show.                                    |
| plot       | to plot when TRUE.                                           |
| cex.labels | Axis label extension factor.                                 |
| cex.points | Data point extension factor.                                 |
| xlab       | X-axis title.                                                |
| ylab       | Y-axis title.                                                |

### Details

This function is both used as its own for a 2d plot and/or generate data for a plotly counterpart.

### Value

positional information.

### Examples

```

## Not run:
# SCALLOP/INF
d <- read.csv("~/INF/work/INF1.merge.cis.vs.trans", as.is=TRUE)
inf <- qtl2dplot(d, cex.points=-d[["log10p"]]/50)
# A qtlClaaifier/qtl2dplot coupling example:

```



```

cis.vs.trans <- read.csv("~/Caprion/analysis/work/caprion_dr.cis.vs.trans",as.is=TRUE)
caprion <- qtl2dplot(cis.vs.trans,chrLen=gap::hg19,
  snp_name="SNP",snp_chr="SNPChrom",snp_pos="SNPPos",
  gene_chr="geneChrom",gene_start="geneStart",gene_end="geneEnd",
  trait="prot",gene="Gene",
  TSS=TRUE,cis="cis",plot=TRUE,
  cex.labels=0.6,cex.points=cis.vs.trans[["log10p"]]/40,
  xlab="pQTL position",ylab="Gene position")
head(cis.vs.trans)
  Gene      SNP prot log10p geneChrom geneStart geneEnd SNPChrom  SNPPos cis
YWHAB 8:111907280_A_T 1433B   7.38      20  43530174 43535076      8 111907280 FALSE
A2M 14:34808001_A_T  A2MG   7.51      12  9220421  9268445     14 34808001 FALSE
APEH 1:12881809_A_G  ACPH   7.83       3 49711834 49720772      1 12881809 FALSE
PGD 2:121896327_A_G  6PGD   7.79       1 10459174 10479803      2 121896327 FALSE
SERPINF2 17:1618262_C_T  A2AP  12.59      17  1648289 1657825     17 1618262  TRUE
PGLS 19:54327869_G_T  6PGL   9.87      19 17622481 17631887     19 54327869 FALSE

## End(Not run)

```

qtl2dplotly

*2D QTL cis/trans visualization (Plotly)*

## Description

Interactive visualization of QTL–gene relationships in 2D genomic space, separating cis and trans associations into stable Plotly traces.

## Usage

```

qtl2dplotly(
  d,
  chrLen = gap::hg19,
  qtl.id = "SNPid:",
  qtl.prefix = "QTL:",
  qtl.gene = "Gene:",
  target.type = "Protein",
  TSS = FALSE,
  xlab = "QTL position",
  ylab = "Gene position",
  ...
)

```

## Arguments

|            |                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------|
| d          | Data frame containing QTL data. Must include SNP, chromosome, position, gene mapping, and cis/trans classification. |
| chrLen     | Chromosome length reference (default: gap::hg19).                                                                   |
| qtl.id     | Prefix for SNP identifier in tooltip.                                                                               |
| qtl.prefix | Prefix for QTL coordinate label.                                                                                    |
| qtl.gene   | Prefix for gene coordinate label.                                                                                   |

|                          |                                                                |
|--------------------------|----------------------------------------------------------------|
| <code>target.type</code> | Label for target entity type.                                  |
| <code>TSS</code>         | Logical. If TRUE, uses transcription start site.               |
| <code>xlab</code>        | Label for x-axis.                                              |
| <code>ylab</code>        | Label for y-axis.                                              |
| <code>...</code>         | Additional arguments passed to <code>gap::qtl2dplot()</code> . |

### Details

This function wraps `gap::qtl2dplot()` and converts the output into an interactive Plotly scatter plot.

Unlike the original implementation, this version:

- avoids `color = ~cistrans` (prevents row dropping in Plotly)
- uses explicit traces for cis/trans groups
- ensures stable HTML widget rendering
- eliminates "Ignoring observations" warnings

### Value

A plotly object.

### Examples

```
## Not run:
INF <- Sys.getenv("INF")
d <- read.csv(file.path(INF, "work", "INF1.merge.cis.vs.trans"), as.is=TRUE)
r <- qtl2dplotly(d)
htmlwidgets::saveWidget(r, file="qtl2dplotly.html")
r

## End(Not run)
```

---

qtl3dplotly

3D QTL plot (cis/trans genomic visualization)

---

### Description

Interactive 3D visualization of QTL–gene relationships using a scatter3D plot.

### Usage

```
qtl3dplotly(
  d,
  chrLen = gap::hg19,
  zmax = 300,
  qtl.id = "SNPid:",
  qtl.prefix = "QTL:",
  qtl.gene = "Gene:",
  target.type = "Protein",
```

```

    TSS = FALSE,
    xlab = "QTL position",
    ylab = "Gene position",
    ...
  )

```

### Arguments

|                          |                                                            |
|--------------------------|------------------------------------------------------------|
| <code>d</code>           | Data frame in <code>qtl2dplot()</code> -compatible format. |
| <code>chrLen</code>      | Chromosome lengths (e.g. <code>gap::hg19, hg38</code> ).   |
| <code>zmax</code>        | Maximum value for z-axis truncation.                       |
| <code>qtl.id</code>      | Prefix for SNP identifier.                                 |
| <code>qtl.prefix</code>  | Prefix for QTL coordinate label.                           |
| <code>qtl.gene</code>    | Prefix for gene coordinate label.                          |
| <code>target.type</code> | Label for target entity type.                              |
| <code>TSS</code>         | Logical; use transcription start site if TRUE.             |
| <code>xlab</code>        | X-axis label.                                              |
| <code>ylab</code>        | Y-axis label.                                              |
| <code>...</code>         | Additional arguments passed to <code>qtl2dplot()</code> .  |

### Details

This function extends `gap::qtl2dplot()` into a 3D representation by adding a z-axis (typically  $-\log_{10}(p)$  or similar association strength).

Features:

- caps extreme values at `zmax`
- colors cis/trans associations
- provides interactive hover labels

### Value

A plotly 3D scatter object.

### Examples

```

## Not run:
suppressMessages(library(dplyr))
INF <- Sys.getenv("INF")
d <- read.csv(
  file.path(INF, "work", "INF1.merge.cis.vs.trans"),
  as.is = TRUE
) %>%
  dplyr::mutate(log10p = -log10p)
r <- qtl3dplotly(d, zmax = 300)
htmlwidgets::saveWidget(r, file = "qtl3dplotly.html")
r

## End(Not run)

```

---

|               |                                   |
|---------------|-----------------------------------|
| qtlClassifier | <i>A QTL cis/trans classifier</i> |
|---------------|-----------------------------------|

---

**Description**

A QTL cis/trans classifier

**Usage**

```
qtlClassifier(geneSNP, SNPPos, genePos, radius)
```

**Arguments**

|         |                                                                                 |
|---------|---------------------------------------------------------------------------------|
| geneSNP | data.frame with columns on gene, SNP and biomarker (e.g., expression, protein). |
| SNPPos  | data.frame containing SNP, chromosome and position.                             |
| genePos | data.frame containing gene, chromosome, start and end positions.                |
| radius  | flanking distance.                                                              |

**Details**

The function obtains QTL (simply called SNP here) cis/trans classification based on gene positions.

**Value**

It returns a geneSNP-prefixed data.frame with the following columns:

- geneChrom gene chromosome.
- geneStart gene start.
- geneEnd gene end.
- SNPChrom pQTL chromosome.
- SNPPos pQTL position.
- Type cis/trans labels.

**Note**

This is adapted from iBMQ/eqtlClassifier as an xQTL (x=e, p, me, ...) classifier.

**See Also**

[cis.vs.trans.classification](#)

## Examples

```
## Not run:
merged <- read.delim("INF1.merge",as.is=TRUE)
hits <- merge(merged[c("CHR","POS","MarkerName","prot","log10p")],
             inf1[c("prot","uniprot")],by="prot")
names(hits) <- c("prot","Chr","bp","SNP","log10p","uniprot")

options(width=200)
geneSNP <- merge(hits[c("prot","SNP","log10p")],
                inf1[c("prot","gene")],by="prot")[c("gene","SNP","prot","log10p")]
SNPPos <- hits[c("SNP","Chr","bp")]
genePos <- inf1[c("gene","chr","start","end")]
cvt <- qtlClassifier(geneSNP,SNPPos,genePos,1e6)
cvt
cistrans <- cis.vs.trans.classification(hits,inf1,"uniprot")
cis.vs.trans <- with(cistrans,data)
cistrans.check <- merge(cvt[c("gene","SNP","Type")],cis.vs.trans[c("p.gene","SNP","cis.trans")],
                      by.x=c("gene","SNP"),by.y=c("p.gene","SNP"))
with(cistrans.check,table(Type,cis.trans))

## End(Not run)
```

---

qtlFinder

*Distance-based signal identification*


---

## Description

Distance-based signal identification

## Usage

```
qtlFinder(
  d,
  Chromosome = "Chromosome",
  Position = "Position",
  MarkerName = "MarkerName",
  Allele1 = "Allele1",
  Allele2 = "Allele2",
  EAF = "Freq1",
  Effect = "Effect",
  StdErr = "StdErr",
  log10P = "log10P",
  N = "N",
  radius = 1e+06,
  collapse.hla = TRUE,
  build = "hg19"
)
```

## Arguments

**d**                      Input data frame containing summary association statistics.

|              |                                                                                                                                      |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Chromosome   | Chromosome column.                                                                                                                   |
| Position     | Genomic position column (bp).                                                                                                        |
| MarkerName   | Variant identifier (e.g. rsID or SNP ID).                                                                                            |
| Allele1      | Effect allele.                                                                                                                       |
| Allele2      | Other allele.                                                                                                                        |
| EAF          | Effect allele frequency.                                                                                                             |
| Effect       | Effect estimate.                                                                                                                     |
| StdErr       | Standard error of the effect estimate.                                                                                               |
| log10P       | Column containing log10(P) values, typically as produced by METAL. Values are converted internally to -log10(P) for ranking signals. |
| N            | Sample size.                                                                                                                         |
| radius       | Maximum gap (bp) allowed between consecutive variants when defining a locus.                                                         |
| collapse.hla | Logical flag indicating whether variants within the HLA region should be collapsed.                                                  |
| build        | Genome build used to define HLA boundaries ("hg19" or "hg38").                                                                       |

### Details

Variants are merged into non-overlapping loci using `valr::bed_merge(max_dist = radius)`. Variants connected through a chain of gaps less than or equal to radius belong to the same locus.

The most significant variant (largest  $-\log_{10}(P)$ ) within each merged locus is selected as the lead signal.

When `collapse.hla=TRUE`, signals within the HLA region are collapsed using HLA boundaries:

- hg19/GRCh37: chr6:28477797-33448354
- hg38/GRCh38: chr6:28510020-33480577

### Value

A tibble containing one lead variant per non-overlapping genomic locus.

Loci are defined by single-linkage clustering of variants separated by at most radius base pairs. Each locus is represented by its most significant variant (largest  $-\log_{10}(P)$ ).

#### Output columns:

- **chrom**: chromosome
- **start, end**: position of the lead variant
- **rsid**: variant identifier
- **a1, a2**: effect and other allele
- **EAF**: effect allele frequency
- **b**: effect size
- **SE**: standard error
- **mlog10p**:  $-\log_{10}(P)$  used for ranking
- **n**: sample size
- **regionStart, regionEnd**: boundaries of the merged locus

#### Notes:

Each row corresponds to a non-overlapping locus defined using `valr::bed_merge(max_dist = radius)`.

The lead variant is selected as the SNP with the highest `mlog10p`.

## Examples

```
## Not run:
threshold <- log10(5*10^-8)
varlist <- c(
  "Chromosome", "Position", "MarkerName", "Allele1", "Allele2",
  "Freq1", "FreqSE", "MinFreq", "MaxFreq",
  "Effect", "StdErr", "log10P", "Direction",
  "HetISq", "HetChiSq", "HetDf", "logHetP", "N"
)
tbl <- "~/Caprion/analysis/METAL_dr/ZPI_dr-1.tbl.gz"
sumstats <- read.table(tbl, col.names = varlist, check.names = FALSE)
res1 <- qtlFinder(sumstats)
res1
subset(res1, mlog10p >= -threshold)
d <- subset(sumstats, log10P <= threshold)
res2 <- qtlFinder(d)
res2
sentinels <- "~/Caprion/analysis/METAL_dr/sentinels/ZPI_dr.p.gz"
d2 <- read.table(sentinels, col.names = varlist, check.names = FALSE)
res3 <- qtlFinder(d2)
res3
# A tibble: 2 × 14
# Groups:   chrom, regionStart, regionEnd [2]
  chrom    start    end rsid  a1    a2    EAF    b    SE mlog10p    n
  <chr>    <dbl>    <dbl> <chr> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <int>
1 chr13 113800622 113800622 13:11. . . t    c    0.373 -0.541 0.0306   69.0 2491
2 chr14  94750486  94750486 14:94. . . t    c    0.989  1.52  0.136   28.5 2491
# 3 more variables: regionStart <dbl>, regionEnd <dbl>, .overlap <int>

## End(Not run)
```

---

read.ms.output

A utility function to read ms output

---

## Description

A utility function to read ms output

## Usage

```
read.ms.output(
  msout,
  is.file = TRUE,
  xpose = TRUE,
  verbose = TRUE,
  outfile = NULL,
  outfileonly = FALSE
)
```

**Arguments**

|                          |                                                                                  |
|--------------------------|----------------------------------------------------------------------------------|
| <code>msout</code>       | an ms output.                                                                    |
| <code>is.file</code>     | a flag indicating ms output as a system file or an R object.                     |
| <code>xpose</code>       | a flag to obtain the tranposed format as it is (when TRUE).                      |
| <code>verbose</code>     | when TRUE, display on screen every 1000 for large nsam.                          |
| <code>outfile</code>     | to save the haplotypes in a tab-delimited ASCII file.                            |
| <code>outfileonly</code> | to reset gametes to NA when nsam/nreps is very large and is useful with outfile. |

**Details**

This function reads in the output of the program ms, a program to generate samples under a variety of neutral models.

The argument indicates either a file name or a vector of character strings, one string for each line of the output of ms. As with the second case, it is appropriate with `system(,intern=TRUE)`, see example below.

**Value**

The returned value is a list storing the results:

- call system call to ms.
- seed random number seed to ms.
- nsam number of copies of the locus in each sample.
- nreps the number of independent samples to generate.
- segsites a vector of the numbers of segregating sites.
- times vectors of time to most recent ancestor (TMRCA) and total tree lengths.
- positions positions of polymorphic sites on a scale of (0,1).
- gametes a list of haplotype arrays.
- probs the probability of the specified number of segregating sites given the genealogical history of the sample and the value to -t option.

**Author(s)**

D Davison, RR Hudson, JH Zhao

**References**

Hudson RR (2002). "Generating samples under a Wright-Fisher neutral model of genetic variation." *Bioinformatics*, **18**(2), 337-8. doi:[10.1093/bioinformatics/18.2.337](https://doi.org/10.1093/bioinformatics/18.2.337).

Press WH, SA Teukolsky, WT Vetterling, BP Flannery (1992). Numerical Recipes in C. Cambridge University Press, Cambridge.



**Examples**

```
## Not run:
# Assuming ms is on the path

system("ms 5 4 -s 5 > ms.out")
msout1 <- read.ms.output("ms.out")

system("ms 50 4 -s 5 > ms.out")
msout2 <- read.ms.output("ms.out",outfile="out",outfileonly=TRUE)

msout <- system("ms 5 4 -s 5 -L", intern=TRUE)
msout3 <- read.ms.output(msout,FALSE)

## End(Not run)
```

ReadGRM

*A function to read GRM file***Description**

A function to read GRM file

**Usage**

```
ReadGRM(prefix = 51)
```

**Arguments**

prefix                      file root.

ReadGRMBin

*A function to read GRM binary files***Description**

A function to read GRM binary files

**Usage**

```
ReadGRMBin(prefix, AllN = FALSE, size = 4)
```

**Arguments**

prefix                      file root.  
 AllN                        a logical variable.  
 size                        size.

**Details**

Modified from GCTA documentation

---

|           |                                     |
|-----------|-------------------------------------|
| revStrand | <i>Allele on the reverse strand</i> |
|-----------|-------------------------------------|

---

**Description**

Allele on the reverse strand

**Usage**

```
revStrand(allele)
```

**Arguments**

allele            Allele to reverse.

**Details**

The function obtains allele on the reverse strand.

**Value**

Allele on the reverse strand.

**Examples**

```
## Not run:
alleles <- c("a","c","G","t")
reverse_strand(alleles)

## End(Not run)
```

---

|             |                       |
|-------------|-----------------------|
| runshinygap | <i>Start shinygap</i> |
|-------------|-----------------------|

---

**Description**

Start shinygap

**Usage**

```
runshinygap(...)
```

**Arguments**

...            Additional arguments passed to the 'runApp' function from the 'shiny' package.

**Details**

This function starts the interactive 'shinygap' shiny web application that allows for flexible model specification.

The 'shiny' based web application allows for flexible model specification for the implemented study designs.

**Value**

These are design specific.

---

|     |                                    |
|-----|------------------------------------|
| s2k | <i>Statistics for 2 by K table</i> |
|-----|------------------------------------|

---

**Description**

Statistics for 2 by K table

**Usage**

```
s2k(y1, y2)
```

**Arguments**

|    |                                                                   |
|----|-------------------------------------------------------------------|
| y1 | a vector containing the first row of a 2 by K contingency table.  |
| y2 | a vector containing the second row of a 2 by K contingency table. |

**Details**

This function calculates one-to-others and maximum accumulated chi-squared statistics for a 2 by K contingency table.

**Value**

The returned value is a list containing:

- x2a the one-to-other chisquare.
- x2b the maximum accumulated chisquare.
- col1 the column index for x2a.
- col2 the column index for x2b.
- p the corresponding p value.

**Note**

The lengths of y1 and y2 should be the same.

**Author(s)**

Chihiro Hirotsu, Jing Hua Zhao

**References**

Hirotsu C, Aoki S, Inada T, Kitao Y (2001). "An exact test for the association between the disease and alleles at highly polymorphic loci with particular interest in the haplotype analysis." *Biometrics*, **57**, 769-778.

**Examples**

```
## Not run:
# an example from Mike Neale
# termed 'ugly' contingency table by Patrick Sullivan
y1 <- c(2,15,16,35,132,30,25,7,12,24,10,10,0)
y2 <- c(0, 6,31,49,120,27,15,8,14,25, 3, 9,3)

result <- s2k(y1,y2)

## End(Not run)
```

---

sentinels

*Sentinel identification from GWAS summary statistics*


---

**Description**

Sentinel identification from GWAS summary statistics

**Usage**

```
sentinels(
  p,
  pid,
  st,
  debug = FALSE,
  flanking = 1e+06,
  chr = "Chrom",
  pos = "End",
  b = "Effect",
  se = "StdErr",
  log_p = NULL,
  snp = "MarkerName",
  sep = ", "
)
```

**Arguments**

|          |                                               |
|----------|-----------------------------------------------|
| p        | an object containing GWAS summary statistics. |
| pid      | a phenotype (e.g., protein) name in pGWAS.    |
| st       | row number as in p.                           |
| debug    | a flag to show the actual data.               |
| flanking | the width of flanking region.                 |
| chr      | Chromosome name.                              |
| pos      | Position.                                     |
| b        | Effect size.                                  |
| se       | Standard error.                               |
| log_p    | log(P).                                       |
| snp      | Marker name.                                  |
| sep      | field delimiter.                              |

## Details

This function accepts an object containing GWAS summary statistics for signal identification as defined by flanking regions. As the associate P value could be extremely small, the effect size and its standard error are used.

A distance-based approach was consequently used and reframed as an algorithm here. It takes as input signals multiple correlated variants in particular region(s) which reach genomewide significance and output three types of sentinels in a region-based manner. For a given protein and a chromosome, the algorithm proceeds as follows:

### Algorithm sentinels

Step 1. for a particular collection of genomewide significant variants on a chromosome, the width of the region is calculated according to the start and end chromosomal positions and if it is smaller than the flanking distance, the variant with the smallest P value is taken as sentinel (I) otherwise goes to step 2.

Step 2. The variant at step 1 is only a candidate and a flanking region is generated. If such a region contains no variant the candidate is recorded as sentinel (II) and a new iteration starts from the variant next to the flanking region.

Step 3. When the flanking is possible at step 2 but the P value is still larger than the candidate at step 2, the candidate is again recorded as sentinel (III) but next iteration starts from the variant just after the variant at the end position; otherwise the variant is updated as a new candidate where the next iteration starts.

Note Type I signals are often seen from variants in strong LD at a cis region, type II results seen when a chromosome contains two trans signals, type III results seen if there are multiple trans signals.

Typically, input to the function are variants reaching certain level of significance and the function identifies minimum p value at the flanking interval; in the case of another variant in the flanking window has smaller p value it will be used instead.

For now key variables in p are "MarkerName", "End", "Effect", "StdErr", "P.value", where "End" is as in a bed file indicating marker position, and the function is set up such that row names are numbered as 1:nrow(p); see example below. When log\_p is specified, log(P) is used instead, which is appropriate with output from METAL with LOGPVALUE ON. In this case, the column named log(P) in the output is actually log<sub>10</sub>(P).

## Value

The function give screen output.

## Examples

```
## Not run:
## OPG as a positive control in our pGWAS
require(gap.datasets)
data(OPG)
p <- reshape::rename(OPGtbl1, c(Chromosome="Chrom", Position="End"))
chrs <- with(p, unique(Chrom))
for(chr in chrs)
{
  ps <- subset(p[c("Chrom", "End", "MarkerName", "Effect", "StdErr")], Chrom==chr)
  row.names(ps) <- 1:nrow(ps)
  sentinels(ps, "OPG", 1)
}
subset(OPGrsid, MarkerName=="chr8:120081031_C_T")
```

```

subset(OPGrsid,MarkerName=="chr17:26694861_A_G")
## log(P)
p <- within(p, {logp <- log(P.value)})
for(chr in chrs)
{
  ps <- subset(p[c("Chrom","End","MarkerName","logp")], Chrom==chr)
  row.names(ps) <- 1:nrow(ps)
  sentinels(ps, "OPG", 1, log_p="logp")
}
## to obtain variance explained
tbl <- within(OPGtbl, chi2n <- (Effect/StdErr)^2/N)
s <- with(tbl, aggregate(chi2n,list(prot),sum))
names(s) <- c("prot", "h2")
sd <- with(tbl, aggregate(chi2n,list(prot),sd))
names(sd) <- c("p1","sd")
m <- with(tbl, aggregate(chi2n,list(prot),length))
names(m) <- c("p2","m")
h2 <- cbind(s,sd,m)
ord <- with(h2, order(h2))
sink("h2.dat")
print(h2[ord, c("prot","h2","sd","m")], row.names=FALSE)
sink()
png("h2.png", res=300, units="in", width=12, height=8)
np <- nrow(h2)
with(h2[ord,], {
  plot(h2, cex=0.4, pch=16, xaxt="n", xlab="protein", ylab=expression(h^2))
  xtick <- seq(1, np, by=1)
  axis(side=1, at=xtick, labels = FALSE)
  text(x=xtick, par("usr")[3],labels = prot, srt = 75, pos = 1, xpd = TRUE, cex=0.5)
})
dev.off()
write.csv(tbl,file="INF1.csv",quote=FALSE,row.names=FALSE)

## End(Not run)

```

## Description

These are a set of functions specifically for single nucleotide polymorphisms (SNPs), which are biallelic markers. This is particularly relevant to the genomewide association studies (GWAS) using GeneChips and in line with the classic generalised single-locus model. snpHWE is from Abecasis's website and yet to be adapted for chromosome X.

## Usage

snpHWE(g)

PARn(p, RRlist)

snpPVE(beta, se, N)

snpPAR(RR, MAF, unit = 2)

**Arguments**

|                     |                                      |
|---------------------|--------------------------------------|
| <code>g</code>      | Observed genotype vector.            |
| <code>p</code>      | genotype frequencies.                |
| <code>RRlist</code> | A list of RRs.                       |
| <code>beta</code>   | Regression coefficient.              |
| <code>se</code>     | Standard error for beta.             |
| <code>N</code>      | Sample size.                         |
| <code>RR</code>     | Relative risk.                       |
| <code>MAF</code>    | Minar allele frequency.              |
| <code>unit</code>   | Unit to exponentiate for homozygote. |

**Details**

snpHWE gives an exact Hardy-Weinberg Equilibrium (HWE) test and it return -1 in the case of misspecification of genotype counts.

snpPAR calculates the the population attributable risk (PAR) for a particular SNP. Internally, it calls for an internal function PARn, given a set of frequencies and associate relative risks (RR). Other 2x2 table statistics familiar to epidemiologists can be added when necessary.

snpPVE provides proportion of variance explained (PVE) estimate based on the linear regression coefficient and standard error. For logistic regression, we can have similar idea for log(OR) and log(SE(OR)).

**Author(s)**

Jing Hua Zhao, Shengxu Li

---

snptest\_sample

*A utility to generate SNPTEST sample file*


---

**Description**

A utility to generate SNPTEST sample file

**Usage**

```
snptest_sample(
  data,
  sample_file = "snptest.sample",
  ID_1 = "ID_1",
  ID_2 = "ID_2",
  missing = "missing",
  C = NULL,
  D = NULL,
  P = NULL
)
```

Arguments

|             |                             |
|-------------|-----------------------------|
| data        | Data to be used.            |
| sample_file | Output filename.            |
| ID_1        | ID_1 as in the sample file. |
| ID_2        | ID_2 as in the sample file. |
| missing     | Missing data column.        |
| C           | Continuous variables.       |
| D           | Discrete variables.         |
| P           | Phenotypic variables.       |

Value

Output file in SNPTEST’s sample format.

Examples

```
## Not run:
d <- data.frame(ID_1=1, ID_2=1, missing=0, PC1=1, PC2=2, D1=1, P1=10)
snptest_sample(d, C=paste0("PC", 1:2), D=paste0("D", 1:1), P=paste0("P", 1:1))

## End(Not run)
```

---

|       |                                                            |
|-------|------------------------------------------------------------|
| tsccl | <i>Power calculation for two-stage case-control design</i> |
|-------|------------------------------------------------------------|

---

Description

Power calculation for two-stage case-control design

Usage

```
tsccl(model, GRR, p1, n1, n2, M, alpha.genome, pi.samples, pi.markers, K)
```

Arguments

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| model        | any in c("multiplicative", "additive", "dominant", "recessive").           |
| GRR          | genotype relative risk.                                                    |
| p1           | the estimated risk allele frequency in cases.                              |
| n1           | total number of cases.                                                     |
| n2           | total number of controls.                                                  |
| M            | total number of markers.                                                   |
| alpha.genome | false positive rate at genome level.                                       |
| pi.samples   | sample% to be genotyped at stage 1.                                        |
| pi.markers   | markers% to be selected (also used as the false positive rate at stage 1). |
| K            | the population prevalence.                                                 |



## Details

This function gives power estimates for two-stage case-control design for genetic association.

The false positive rates are calculated as follows,

$$P(|z_1| > C_1)P(|z_2| > C_2, \text{sign}(z_1) = \text{sign}(z_2))$$

and

$$P(|z_1| > C_1)P(|z_j| > C_j | |z_1| > C_1)$$

for replication-based and joint analyses, respectively; where  $C_1$ ,  $C_2$ , and  $C_j$  are thresholds at stages 1, 2 replication and joint analysis,

$$z_1 = z(p_1, p_2, n_1, n_2, \text{pi.samples})$$

$$z_2 = z(p_1, p_2, n_1, n_2, 1 - \text{pi.samples})$$

$$z_j = \text{sqrt}(\text{pi.samples}) * z_1 + \text{sqrt}(1 - \text{pi.samples}) * z_2$$

## Value

The returned value is a list containing a copy of the input plus output as follows,

- model any in c("multiplicative", "additive", "dominant", "recessive").
- GRR genotype relative risk.
- p1 the estimated risk allele frequency in cases.
- pprime expected risk allele frequency in cases.
- p expected risk allele frequency in controls.
- n1 total number of cases.
- n2 total number of controls.
- M total number of markers.
- alpha.genome false positive rate at genome level.
- pi.samples sample% to be genotyped at stage 1.
- pi.markers markers% to be selected (also used as the false positive rate at stage 1).
- K the population prevalence.
- C thresholds for no stage, stage 1, stage 2, joint analysis.
- power power corresponding to C.

## Note

solve.skol is adapted from CaTS.

## Author(s)

Jing Hua Zhao

## References

Skol AD, Scott LJ, Abecasis GR, Boehnke M (2006). "Joint analysis is more efficient than replication-based analysis for two-stage genome-wide association studies." *Nat Genet*, **38**(2), 209-13. doi:10.1038/ng1706.

**Examples**

```
## Not run:
K <- 0.1
p1 <- 0.4
n1 <- 1000
n2 <- 1000
M <- 300000
alpha.genome <- 0.05
GRR <- 1.4
p1 <- 0.4
pi.samples <- 0.2
pi.markers <- 0.1

options(echo=FALSE)
cat("sample%,marker%,GRR,(thresholds x 4)(power estimates x 4)","\\n")
for(GRR in c(1.3,1.35,1.40))
{
  cat("\\n")
  for(pi.samples in c(1.0,0.5,0.4,0.3,0.2))
  {
    if(pi.samples==1.0) s <- 1.0
    else s <- c(0.1,0.05,0.01)
    for(pi.markers in s)
    {
      x <- tsc("multiplicative",GRR,p1,n1,n2,M,alpha.genome,
              pi.samples,pi.markers,K)
      l <- c(pi.samples,pi.markers,GRR,x$C,x$power)
      l <- sprintf("%.2f %.2f %.2f, %.2f %.2f %.2f %.2f, %.2f %.2f %.2f %.2f",
                  l[1],l[2],l[3],l[4],l[5],l[6],l[7],l[8],l[9],l[10],l[11])
      cat(l,"\\n")
    }
    cat("\\n")
  }
}
options(echo=TRUE)

## End(Not run)
```

whscore

*Whittemore-Halpern scores for allele-sharing***Description**

Whittemore-Halpern scores for allele-sharing

**Usage**

whscore(allele, type)

**Arguments**

allele            a matrix of alleles of affected pedigree members.  
 type            0 = pairs, 1 = all.

**Details**

Allele sharing score statistics.

**Value**

The returned value is the value of score statistic.

**Note**

adapted from GENEHUNTER.

**Author(s)**

Leonid Kruglyak, Jing Hua Zhao

**References**

Kruglyak L, Daly MJ, Reeve-Daly MP, Lander ES (1996). "Parametric and nonparametric linkage analysis: a unified multipoint approach." *Am J Hum Genet*, **58**(6), 1347-63.

Whittemore AS, Halpern J (1994). "A class of tests for linkage using affected pedigree members." *Biometrics*, **50**(1), 118-27.

Whittemore AS, Halpern J (1994). "Probability of gene identity by descent: computation and applications." *Biometrics*, **50**(1), 109-17.

**Examples**

```
## Not run:
c<-matrix(c(1,1,1,2,2,2),ncol=2)
whscore(c,type=1)
whscore(c,type=2)

## End(Not run)
```

---

WriteGRM

*A function to write GRM file*


---

**Description**

A function to write GRM file

**Usage**

```
WriteGRM(prefix = 51, id, N, GRM)
```

**Arguments**

|        |              |
|--------|--------------|
| prefix | file root.   |
| id     | id.          |
| N      | sample size. |
| GRM    | a GRM.       |

---

|             |                                            |
|-------------|--------------------------------------------|
| WriteGRMBin | <i>A function to write GRM binary file</i> |
|-------------|--------------------------------------------|

---

**Description**

A function to write GRM binary file

**Usage**

```
WriteGRMBin(prefix, grm, N, id, size = 4)
```

**Arguments**

|        |              |
|--------|--------------|
| prefix | file root.   |
| grm    | a GRM.       |
| N      | Sample size. |
| id     | id.          |
| size   | size.        |

---

|    |                                                  |
|----|--------------------------------------------------|
| xy | <i>Conversion of chromosome names to strings</i> |
|----|--------------------------------------------------|

---

**Description**

Conversion of chromosome names to strings

**Usage**

```
xy(x)
```

**Arguments**

|   |                                             |
|---|---------------------------------------------|
| x | (alpha)numeric value indicating chromosome. |
|---|---------------------------------------------|

**Details**

This function converts x=1:24 to 1:22, X, Y

**Value**

As indicated.

# Index

- \* **GWAS**
  - labelManhattan, 86
  - miamipLOT2, 122
- \* **Manhattan**
  - labelManhattan, 86
  - miamipLOT2, 122
- \* **Miami**
  - labelManhattan, 86
  - miamipLOT2, 122
- \* **annotation**
  - labelManhattan, 86
- \* **datagen**
  - b2r, 14
  - gif, 55
  - kin.morgan, 84
- \* **datasets**
  - hg18, 69
  - hg19, 70
  - hg38, 70
- \* **distribution**
  - METAL\_forestplot, 101
  - qqfun, 148
  - qqunif, 150
- \* **dplot**
  - pedtodot, 137
- \* **hplot**
  - asplot, 13
  - ESplot, 34
  - METAL\_forestplot, 101
  - plot.hap.score, 145
  - qqunif, 150
- \* **htest**
  - AE3, 11
  - chow.test, 22
  - comp.score, 31
  - gcp, 48
  - hwe.cc, 75
  - hwe.jags, 79
  - klem, 85
  - MCMCgrm, 99
- \* **internal**
  - gap, 40
- \* **meta-analysis**
  - metareg, 105
- \* **misc**
  - fbsize, 35
  - masize, 94
  - pbsize, 134
  - pbsize2, 136
  - tscC, 168
- \* **models**
  - AE3, 11
  - BFDp, 15
  - bt, 17
  - FPRP, 37
  - gc.em, 43
  - gcontrol, 45
  - gcontrol2, 47
  - gcp, 48
  - genecounting, 50
  - hap, 62
  - hap.em, 65
  - hap.score, 67
  - LD22, 87
  - LDk1, 89
  - metareg, 105
  - mtdt, 127
  - muvar, 130
  - pfc, 141
  - pfc.sim, 142
  - s2k, 163
- \* **print**
  - print.hap.score, 146
- \* **regression**
  - hap.score, 67
  - qqfun, 148
- \* **univar**
  - qqfun, 148
  - qqunif, 150
- \* **utilities**
  - pgc, 144
  - read.ms.output, 159
  - sentinels, 164
  - snpHWE, 166
  - whscore, 170
- \* **utilties**

- muvar, 130
- a2g, 4
- ab, 5, 98
- ACDE, 7
- ACE\_CI, 10
- AE3, 11
- allele.recode, 12
- asplot, 13
- b2r, 14
- BFDP, 15, 38
- bt, 17, 128
- car::qq.plot, 149
- ccsize, 19
- chow.test, 22
- chr\_pos\_a1\_a2, 23
- ci2ms, 24
- circos.cis.vs.trans.plot, 26
- circos.cnvplot, 27
- circos.mhtplot, 27
- circos.mhtplot2, 28
- cis.vs.trans.classification, 29, 156
- cnvplot, 30
- comp.score, 31
- cs, 33
- ESplot, 34
- fbsize, 35, 135
- fbsize(), 9
- FPRP, 16, 37
- g2a, 39
- gap, 40
- gap-package (gap), 40
- gc.em, 43, 51, 88
- gc.lambda, 45
- gcontrol, 45
- gcontrol2, 47, 149
- gcp, 48
- genecounting, 43, 44, 49, 50, 64, 85, 88, 145
- genetics::genotype, 78
- genetics::HWE.test, 78
- geno.recode, 52
- get\_b\_se, 52
- get\_pve\_se, 53
- get\_sdy, 54
- gif, 55, 84
- graphics::par, 148, 149
- graphics::plot(), 110, 117
- graphics::text, 87
- grDevices::palette, 148
- grid2d, 56
- h2.jags, 57
- h2\_mzdz, 60
- h2G, 59
- h2GE, 59
- h2L, 60
- hap, 62, 66, 120
- hap.control, 64
- hap.em, 65
- hap.score, 67, 146, 147
- hg18, 69
- hg19, 70
- hg38, 70
- hmht.control, 70
- hmht.control(), 109, 110, 112, 117
- htr, 23, 71
- hwe, 73, 76, 78
- hwe.cc, 75
- hwe.hardy, 77, 80, 145
- hwe.jags, 79
- inv\_chr\_pos\_a1\_a2, 82
- invnormal, 81
- ixy, 83
- KCC, 83
- kin.morgan, 84, 141
- klem, 85
- labelManhattan, 86, 87, 123
- lattice::qqmath, 149
- LD22, 15, 87, 90
- LDkl, 44, 51, 66, 88, 89
- log10p, 91
- log10pvalue, 91
- logp, 92
- makeped, 92
- makeRLEplot, 93
- masize, 94
- MCMCgrm, 99
- METAL\_forestplot, 101
- metap, 103
- metareg, 105, 133
- mht.control, 108
- mht.control(), 71, 110, 112, 117
- mhtplot, 109
- mhtplot(), 70, 71, 108, 109
- mhtplot.trunc, 114
- mhtplot2, 117
- mia, 119
- miamipLOT, 120

miamiplo2, [87](#), [122](#)  
mr, [124](#)  
mr\_forestplot, [126](#)  
mtdt, [18](#), [127](#), [130](#)  
mtdt2, [129](#)  
muvar, [130](#)  
mvmeta, [15](#), [132](#)  
  
nlme::lme, [7](#)  
  
PARn (snpHWE), [166](#)  
pbsize, [20](#), [36](#), [134](#), [136](#), [137](#)  
pbsize(), [9](#)  
pbsize2, [136](#)  
pedtodot, [137](#)  
pedtodot\_verbatim, [140](#)  
pfc, [56](#), [141](#), [143](#)  
pfc.sim, [142](#)  
pgc, [144](#)  
plot.hap.score, [145](#)  
print.hap.score, [146](#)  
pvalue, [147](#)  
  
qqfun, [148](#), [151](#)  
qqunif, [149](#), [150](#)  
qtl2dplot, [151](#)  
qtl2dplotly, [153](#)  
qtl3dplotly, [154](#)  
qtlClassifier, [156](#)  
qtlFinder, [157](#)  
  
read.ms.output, [159](#)  
ReadGRM, [161](#)  
ReadGRMBin, [161](#)  
revStrand, [162](#)  
runshinygap, [162](#)  
  
s2k, [163](#)  
sentinels, [164](#)  
snpHWE, [166](#)  
snpPAR (snpHWE), [166](#)  
snpPVE (snpHWE), [166](#)  
snptest\_sample, [167](#)  
stats::qqnorm, [149](#)  
  
tscc, [136](#), [168](#)  
  
whscore, [170](#)  
WriteGRM, [171](#)  
WriteGRMBin, [172](#)  
  
xy, [172](#)